



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校 南京航空航天大学

参赛队号 24102870008

队员姓名 1. 曾实

 2. 仇静轩

 3. 马欣

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： 高速公路应急车道紧急启用模型

摘 要：

高速公路拥堵现象的成因复杂多样，除了交通事故，瓶颈现象尤为突出，这通常源于车辆的集中流动，导致拥堵现象容易扩散。目前，高速公路管理部门在应急车道的使用上还处于探索阶段，多依赖视频监控和管理者经验判断。这样的做法虽然直观，但缺乏理论依据，导致效果难以评估。因此，建立数学模型来识别即将发生拥堵的条件，进行拥堵预警，并评估临时借用应急车道的效果，优化监控点布局，以帮助管理者做出科学决策，具有重要的理论与现实意义。

针对问题一，为统计四个观测点的交通流量、车流密度以及时间平均速度三个交通流参数随时间的变化规律。本问题首先，根据附件提供的视频数据利用 Opencv 模型对视频数据进行预处理，包括**透视变换**、异常值处理操作；其次，基于 YOLOv8 模型逐帧从视频数据中提取四个观测点的交通流参数，绘制每个观测点对应视频的交通流参数随时间的**变化趋势图**；最后，通过对变化趋势图的分析，确定**交通流参数随时间的变化规律**。

为实时预警从第三点到第四点之间路段可能常出现的持续拥堵状态，本问题首先，根据交通流参数建立**交通流拥堵模型**，得出交通拥堵指标随时间的变化趋势；其次，在拥堵模型的基础上，建立**拥堵状态实时预警模型**；最后，通过基于**时空集成学习的交通流时序预测**方法，在交通流拥堵模型初步判断拥堵状态的基础上，加入了**窗口梯度判别机制**，从而建立了交通拥堵状态实时预警模型，并验证了其预测结果的有效性和准确性。

针对问题二，为合理启用高速公路应急车道，首先，结合高速公路通行能力，根据问题一得到的交通流参数，计算**交通流中断概率**，观察四个观测点对应视频交通流中断概率随时间变化的情况；其次，为评估发生交通拥堵的概率，建立了**高速公路应急车道启用模型**。通过分析可得，当车流量较大时，交通拥堵 概率也较大，车流速度下降，更

容易发生拥堵的状况。最后，采用**最大似然估计法**确定交通流量和交通速度的**阈值**，即当**时间平均速度小于 20m/s** 或者**车流量大于 0.5veh/s** 时，启用高速公路应急车道。

针对问题三的实时决策问题，确保**车辆在行驶过程中的道路安全是前提条件**。因此，本问题首先，对车道控制时间与空间变化频次进行约束，设定了**时空安全约束条件**；其次，结合问题一的交通拥堵指标和拥堵实时预警模型，问题二的交通流中断概率模型，**设定了启用应急车道的实时决策策略**，建立了应急车道启用实时决策模型；最后，从**总行程时间、总碰撞暴露时间、平均通行时间、总延误时间和车辆平均速度**这五个指标量化了该模型对应急车道控制的影响。通过对量化结果的分析可知，所建立的应急车道启用实时决策模型有效缓解了交通拥堵。

针对问题四，需要在**科学决策和控制成本**的前提下，合理布置第三个点到第四个点之间路段视频监控点的位置和数量。首先，从**安全性、效率以及成本**三个角度出发，建立了监测点布置策略多目标优化模型；然后，基于 **Kriging 元模型**和**遗传算法**，求解出对应路段视频监控点的最优位置和数量，从而求得**最佳布置方案**。最后，将算法求解出的最佳方案布置到观测点 3 和观测点 4 之间的路段中，可以得出，布置后的视频监控相较于原方案，平均通行时间降低了 22.37%，总延误时间降低了 26.49%，总碰撞暴露时间降低了 19.12%，总行程时间降低了 16.24%，车辆平均速度提升了 7.23%。

关键词： 时空集成学习，实时预警，时空安全约束，应急决策优化，Kriging 元模型和遗传算法

本文的技术路线图如图 1 所示：



图 1 全文总体技术路线图

目录

一、 问题背景与问题重述	7
1.1 问题背景	7
1.2 问题重述	7
二、 基本假设与符号说明	9
2.1 模型基本假设	9
2.2 符号说明	9
三、 视频数据统计分析及预处理	11
3.1 视频数据统计	11
3.2 道路交通流参数	12
3.3 基于 YOLOv8 的视频数据预处理	13
3.4 异常值处理	15
四、 问题一模型的建立与求解	16
4.1 问题分析	16
4.2 交通流参数变化规律分析	17
4.2.1 交通流量随时间变化分析	17
4.2.2 车流密度随时间变化分析	19
4.2.3 时间平均速度随时间变化分析	20
4.2.4 速度-流量分析	22
4.3 交通流拥堵模型的建立与求解	22
4.3.1 交通拥堵指数	22
4.3.2 交通流拥堵模型建立	23
4.3.3 交通流拥堵模型结果分析	23
4.4 交通流拥堵预警模型的建立	24
4.4.1 基于时空集成学习的时间序列预测方法	24
4.4.2 拥堵状态实时预警模型的建立	35
4.4.3 交通流参数预测结果	38
4.4.4 交通拥堵指标的预测结果	40
4.5 交通流参数预测模型的验证与分析	40
4.5.1 交通流参数及其拥堵模型的验证	40
4.5.2 评价指标	43
五、 问题二模型的建立与求解	45
5.1 问题分析	45
5.2 高速公路应急车道启用模型的建立	45
5.3 高速公路应急车道启用模型的求解	46
5.4 最大似然法估计分布参数	47
5.5 速度和流量阈值判定	48

六、 问题三模型的建立与求解	50
6.1 问题分析	50
6.2 应急车道启用实时决策模型的建立	50
6.3 应急车道启用实时决策模型的求解	52
6.4 应急车道实时决策模型的量化分析	55
6.4.1 量化指标体系构建	55
6.4.2 实时决策模型结果分析	56
七、 问题四的求解	59
7.1 问题分析	59
7.2 监控点布置策略多目标优化模型的建立与分析	59
7.3 视频监控点最佳布置方案	62
八、 模型评价	65
8.1 模型的优点	65
8.2 模型的改进和展望	65
九、 参考文献	66
附录	67

一、问题背景与问题重述

1.1 问题背景

高速公路拥堵现象的成因复杂多样，除了交通事故，瓶颈现象尤为突出，这通常源于车辆的集中流动，导致拥堵现象容易扩散。特别是在匝道出入口或桥梁入口等地段，拥堵现象更为明显。此外，一些高速公路因车道数量不足，通行能力受到限制，也是导致拥堵的重要因素。尽管拓宽道路可以有效缓解拥堵，但这并不在本研究的考量范围之内。我们的重点是如何在现有条件下最大化资源利用。在某些特定情况下，若能在拥堵发生之前采取措施，便可有效化解潜在的交通堵塞。高速公路建设时，通常会在右侧增设应急车道，以满足工程抢险和紧急救援的需求。尽管应急车道通常被视为救生通道，限制占用，但若能合理使用，比如通过监控系统及时识别潜在拥堵路段，允许在没有事故的情况下使用应急车道，能有效降低车流密度，避免大规模拥堵。

目前，高速公路管理部门在应急车道的使用上还处于探索阶段，多依赖视频监控和管理者经验判断。这样的做法虽然直观，但缺乏理论依据，导致效果难以评估。因此，建立数学模型来识别即将发生拥堵的条件，进行拥堵预警，并评估临时借用应急车道的效果，优化监控点布局，以帮助管理者做出科学决策，具有重要的理论与现实意义。

1.2 问题重述

高速公路建设时通常会在右侧增设应急车道，用于应对紧急情况。然而，在非紧急情况下，合理利用应急车道可以有效缓解拥堵。本文针对某路段（长度约 5000m，行车道 2+应急车道 1）进行研究，通过四个视频观测点采集数据，建立交通流拥堵模型，探索实时预警机制，并研究如何通过合理布置视频监控点来提高应急车道临时启用决策的科学性和成本效益。

本文需要解决下列四个问题：

问题一：数据分析和拥堵模型的建立

(1) 数据分析：根据四个观测点的视频数据，统计每个观测点的交通流参数随时间的变化规律，包括车流密度、流量、速度等。通过更精细的统计分析，可以揭示交通流变化的趋势和规律，为模型的建立提供更准确的数据支持。

(2) 交通流拥堵模型建立：利用四个观测点的交通流参数，结合道路情况（两行车道），建立交通流拥堵模型。该模型是随着车流密度、流量、速度等参数的变化而变化。

(3) 交通流拥堵预警模型：根据观测点 3 和观测点 4 的第一个视频所统计的每个观测点的交通流参数随时间的变化规律，建立交通流参数预测模型，从而预测并基于交通流拥堵模型判断出从第三点到第四点之间路段可能出现持续拥堵状态，得出实时预警时间。

(4) 交通流参数预测模型验证：利用观测点 3 和观测点 4 的第二个视频数据，来验证根据观测点 3 和观测点 4 的第一个视频所构建的有效性，从而评估模型的准确性和可靠性。

问题二：合理高速公路启用应急车道的建立

(1) 基于问题一中获取的交通流量，车流密度和时间平均速度的结果，得出的交通拥堵指数进行分析；

(2) 构建并求解交通流中断概率模型，从而确定高速公路启用应急车道速度与交通流量的阈值，达到合理启用高速公路应急车道模型。。

问题三：应急车道启用实时决策模型

(1) 决策机制：在确保车辆在行驶过程中的道路安全的前提条件下，根据建立的拥堵模型，制定应急车道临时启用规则。当模型预警第三点到第四点之间路段可能出现持续拥堵时，决策系统会根据预警时间、拥堵程度等因素，决定是否启用应急车道。

(2) 效果评估：选择如总行程时间、总碰撞暴露时间、平均通行时间、总延误时间和车辆平均速度等指标，量化评估应急车道启用模型的效果，包括减少拥堵时间、提升通行效率、降低交通事故风险等方面的效益。

问题四：监控点布置策略多目标优化模型

在确保车辆在行驶过程中的道路安全的前提条件下，需考虑以下三点：

(1) 关键路段：重点关注容易出现拥堵的路段，例如匝道出入口、桥梁入口等。在这些路段增加监控点，可以更准确地监测交通流量变化，提高预警的准确性和及时性。

(2) 数据覆盖：确保视频监控覆盖整个路段，以便全面掌握交通流信息，为模型提供更准确的数据支持。

(3) 成本控制：合理规划监控点位置，避免过度密集，以控制成本，确保资金效益最大化。

二、基本假设与符号说明

2.1 模型基本假设

在数学建模的过程中,在不影响模型意义与计算精度的前提下,为了使模型简单、明确,计算建立了如下假设:

- (1) 数据真实可靠,各个观测点之间的交通流是独立的且互不影响;
- (2) 每个观测点提供的数据能够精确反映车辆的流量、密度和速度,不存在视频数据丢失和观测误差。
- (3) 假设在没有突发交通事故的情况下,交通流在特定路段内是相对稳定的。即在相同的交通条件下(如时间、天气、路况等),交通流的密度、流量和速度之间的关系是可以预测的。
- (4) 假设在特定条件下(如交通流密度达到一定值时),临时启用应急车道能够有效降低车流密度,提高车辆通行速度,从而缓解拥堵现象。

2.2 符号说明

为了增加论文的可读性,如表 1 所示给出了本文模型建立过程中用到的符号及其说明。

表 1 符号说明

符号名称	符号说明
$Q^p(t)$	观测点 p 处在 t 时刻的交通流量数
$k^p(t)$	观测点 p 处在 t 时刻的车流密度
$v^p(t)$	观测点 p 处在 t 时刻的时间平均速度
$Occ^p(t)$	观测点 p 处在 t 时刻的时间占有率
T	观测时间段
$N^p(t)$	观测时段内通过观测点 p 的车辆总数
$\gamma^p(t)$	观测点 p 处 t 时刻的对应时段的交通拥堵指数
TPI	交通拥堵指数
R^2	决定系数
RMSE	均方根误差

MAPE	平均绝对百分比误差
MSE	均方误差
MAE	平均绝对误差
T _{TT}	总行程时间
T _{ET}	总碰撞暴露时间
T _{AT}	平均通行时间
T _{DT}	总延误时间
V _{AV}	车辆平均速度

三、 视频数据统计分析及预处理

3.1 视频数据统计

本题主要提供了四个附件的视频数据，数据来源于 2024 年 5 月 1 日长深高速公路某段上如图 2 所示的四个点位的中午 11 点 35 分至 16 点 10 分的交通监控视频。这四个点位的视频信息如表 2 所示。每个观测点各自有 2-3 个视频段，其中每个观测点的视频段的时间是连续的。第一个点位和第三个点位属于应急车道抓拍视频，观测的时间段为 827 秒，而第二个点位和第四个点位属于特殊所有的车道。在这四个观测段中的道路都是 2 个行车道和 1 个应急车道，且这段区域没有出入口，因此，该高速公路具有封闭性。



图 2 路段中四个观测点的示意图

表 2 数据集介绍表

观测点	开始时间	结束时间	实际时间/s	视频时间/s	倍速
第一个点位 32.31.250.107	11:41:04	13:57:53	8209	827	10
	13:57:57	16:14:26	8193	827	10
第二个点位 32.31.250.105	11:52:28	13:04:05	4300	430	10
	13:04:15	14:15:27	4272	428	10
	14:15:55	15:27:34	4299	433	10
第三个点位 32.31.250.108	11:35:43	13:52:32	8209	827	10
	13:52:37	16:09:11	8194	820	10
第四个点位 32.31.250.103	12:56:47	14:08:01	4274	427	10
	14:08:06	15:20:03	4317	423	10

3.2 道路交通流参数

本文研究高速公路的交通流参数包括交通流量数 $Q^p(t)$ ，密度 $k^p(t)$ ，时间平均速度 $v^p(t)$ 和时间占有率 $Occ^p(t)$ 。

(1) 交通流量数 $Q^p(t)$

交通流量数是道路某断面在给定的观测时间内通过的车辆总数，其计算方法如下：

$$Q^p(t) = \frac{N^p(t)}{T} \quad (1)$$

式中， $Q^p(t)$ 为观测时间段内的交通流量(veh/s)， $N^p(t)$ 为观测时段内通过观测点 p 的车辆总数(veh)， T 为观测时间段(s)。

(2) 车流密度 $k^p(t)$

车流密度是道路某断面在给定的观测时间内通过的车辆密度，其计算方法如下：

$$k^p(t) = \frac{N^p(t)}{L} \quad (2)$$

式中， $k^p(t)$ 为观测时间段内的车流密度(veh/m)， $N^p(t)$ 为观测时段内通过该观测点的车辆总数(veh)， L 为观测距离(m)。在本文中，这里设置 L 为 30 m。

(3) 时间平均速度 $v^p(t)$

时间平均速度是道路观测点 p 在观测时段内通过固定观测点速度的算术平均值，其计算方法如下：

$$v^p(t) = \frac{\sum_{i=1}^{N^p(t)} v_i}{N^p(t)} \quad (3)$$

式中， $v^p(t)$ 为观测时间段内的时间平均速度(m/s)， $N^p(t)$ 为观测时段内通过该观测点的车辆总数(veh)。

(4) 时间占有率 $Occ^p(t)$

时间占有率是道路观测点 p 在观测时段内车辆通过观测点 p 的累计时间占观测总时间的百分比：

$$Occ^p(t) = \frac{\sum_{i=1}^{N^p(t)} t_i}{T} \quad (4)$$

式中， $Occ^p(t)$ 为观测点 p 处在 t 时刻的时间占有率(%)， t_i 为第 i 辆车的占有时间(s)， T 为观测时间段(s)。

3.3 基于 YOLOv8 的视频数据预处理

为提高检测性能，现代目标检测器在正负样本分配策略上进行了大量的研究。典型的动态分配策略包括 YOLOX 的 simOTA、TOOD 的 Task AlignedAssigner 和 RTMDet 的 Dynamic Soft Label Assigner。这些策略通过动态调整正负样本的分配，提升了模型的学习效率和准确性。而 YOLOv5 仍然采用静态分配策略，相对而言缺乏灵活性。鉴于动态分配策略的优越性，YOLOv8 直接引入了 TOOD 的 Task Aligned Assigner。Task Aligned Assigner 的匹配策略可以简单概括为：根据分类和回归分数加权后的综合得分来选择正样本。这种方法有效结合了分类和定位信息，使正样本的选择更加准确，提升了模型的整体性能。

在损失函数的计算中，模型包含两个分支：分类分支和回归分支。

- (1) 分类分支：继续使用二元交叉熵损失函数（Binary Cross-Entropy Loss, BCELoss），用于衡量预测类别与真实类别之间的差异。
- (2) 回归分支：为了与 Distribution Focal Loss 中提出的积分形式表示法相结合，采用了 Distribution Focal Loss，同时使用了 CIoU Loss 来优化边界框的定位。这两个损失共同作用，提高了模型的定位精度。

最终，三个损失值按照一定的权重比例进行加权求和，形成总的损失函数，用于反向传播和模型优化。视频处理过程如图 3 和图 4 所示：

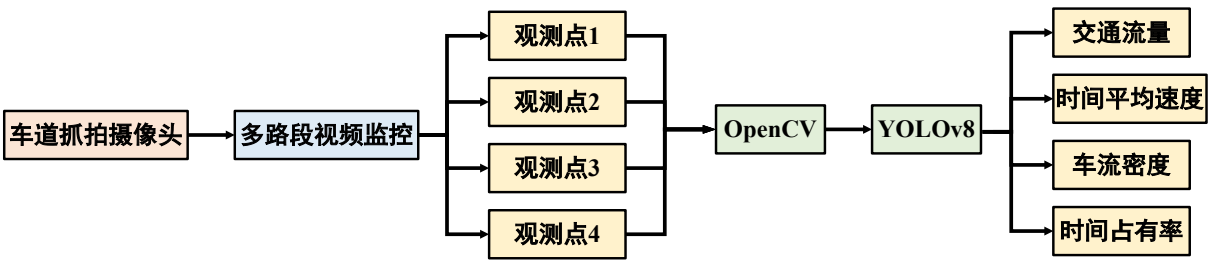


图 3 视频处理示意图

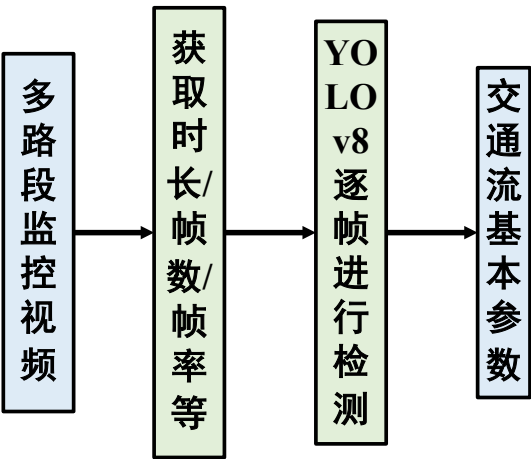


图 4 识别视频划分示意图

基于以上原理，检测车辆并进行一些后续处理，车辆计数和速度计算。在检测过程中，每一帧都可以通过 YOLOv8 检测到如图 5 所示的每辆车的 ID 和速度。

在视频处理过程中，由于透视效应的影响，距离摄像机较近的车辆看起来比远处的车辆更大。在此情形下，即使它们在现实中以相同的速度行驶，在视频中显示的速度也会有所不同。这种近大远小的现象会导致对物体运动的误判，因此，需要对视频中的每一帧进行透视校正。

透视变换是用于校正由于摄像机镜头畸变或拍摄角度所引起的图像失真问题的关键技术。其核心原理是将图像从一个视角投影到另一个视角，校正透视失真，以确保图像中的平行线在校正后仍然保持平行。

具体步骤如下：

1. 确定对应点：首先，在原始图像（源平面）和校正后图像（目标平面）上选取至四对对应点。这些点用于建立两平面之间的空间关系。

2. 计算透视变换矩阵：利用这些对应点，构建一个 3×3 的透视变换矩阵 H 。这个矩阵包含了图像从源平面到目标平面的映射关系，可以通过解线性方程组求解，涉及线性变换和透视投影的参数。

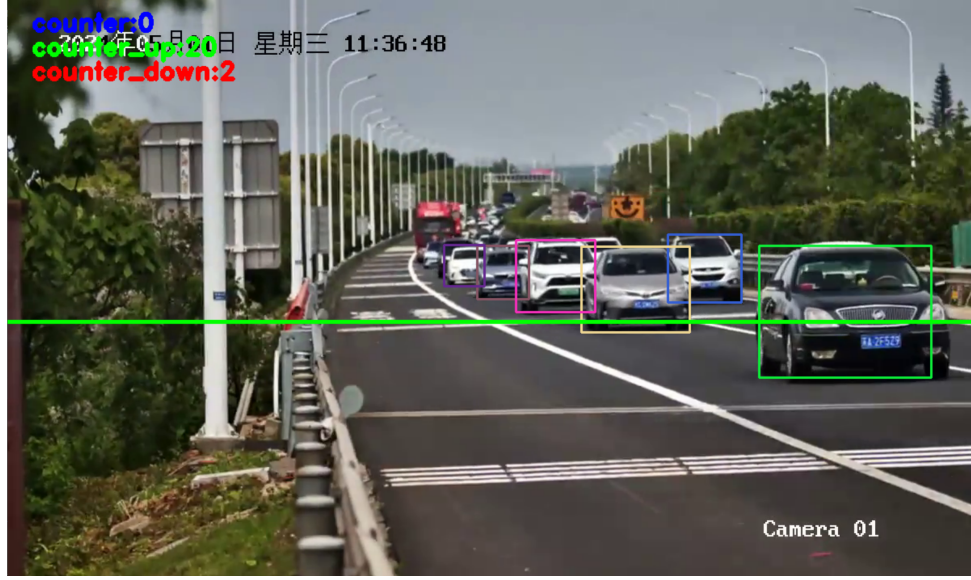
3. 应用变换：将原始图像中的每个像素点通过矩阵 H 映射到校正后的图像位置。这需要使使用齐次坐标表示方法，将二维坐标 (x, y) 扩展为 $(x, y, 1)$ 形式，以便进行矩阵计算。

通过上述步骤，能够有效地校正图像中的透视失真，消除由于摄像机位置和角度造成的误差，使得视频更准确地反映真实场景中的运动情况。这对于需要精确测量和分析的视频应用，例如交通监控尤为重要。

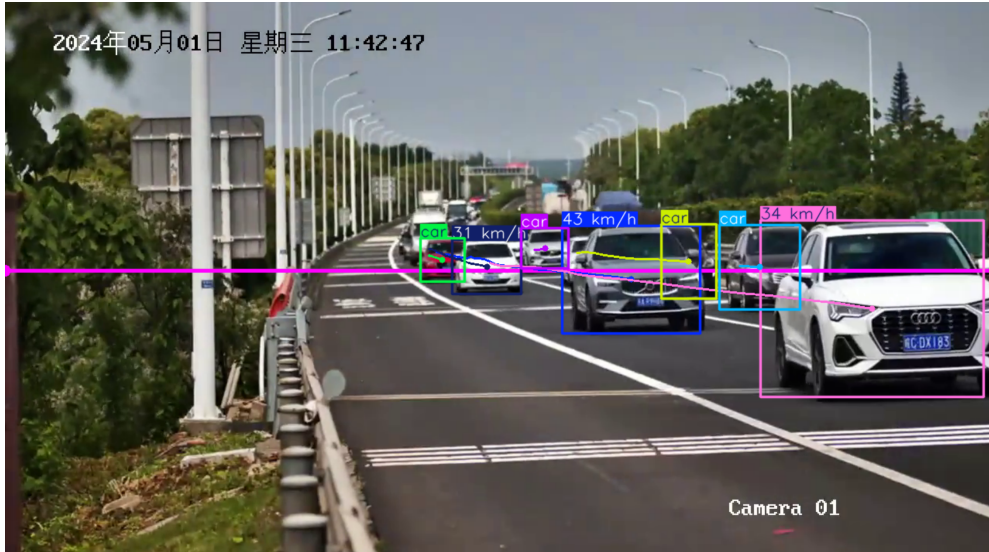
透视变换的数学表达式如下所示：

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} \quad (5)$$

式中， (x, y, w) 是原始图像中的齐次坐标， (x', y', w') 是变换后的坐标。在实际操作中，通常 $w=1$ ，这样变换后的坐标可以通过 $x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$ 和 $y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$ 计算得到的。即近大远小的原则，本文通过代码实现后进行了角度的调整。



(a) 检测交通流量数



(b) 检测时间平均速度

图 5 YOLOv8 检测车辆示意图

3.4 异常值处理

针对本文涉及的数据进行异常值检验，如存在异常值考虑使用拉依达准则进行异常值剔除。具体而言，拉依达准则(3 σ 准则)将对被测变量进行等精度测量，得到 $x_1, x_2, \dots, x_n$ 。接着，计算其算术平均值 $\bar{x}$ 及剩余误差 $v_i = x_i - \bar{x} (i=1, 2, \dots, n)$ ，并按照贝塞尔公式计算出标准误差 σ 。如果某个测量值 x_j 的剩余误差 $v_j (1 \leq j \leq n)$ ，满足 $|v_j| = |x_j - \bar{x}| > 3\sigma$ ，则认为含有较大误差值的坏值，应予剔除。贝塞尔公式如下式：

$$\sigma = \left(\frac{1}{n-1} \sum_{i=1}^n v_i^2 \right)^{\frac{1}{2}} \quad (6)$$

四、问题一模型的建立与求解

4.1 问题分析

问题一具体包含以下四个要点：

(1) 数据分析：根据四个观测点的视频数据，统计每个观测点的交通流参数随时间的变化规律，包括车流密度、流量、速度等。通过更精细的统计分析，可以揭示交通流变化的趋势和规律，为模型的建立提供更准确的数据支持。

(2) 交通流拥堵模型建立：利用四个观测点的交通流参数，结合道路情况（两行车道），建立交通流拥堵模型。该模型是随着车流密度、流量、速度等参数的变化而变化。

(3) 交通流拥堵预警模型：根据观测点 3 和观测点 4 的第一个视频所统计的每个观测点的交通流参数随时间的变化规律，建立交通流参数预测模型，从而预测并基于交通流拥堵模型判断出从第三点到第四点之间路段可能出现持续拥堵状态，得出实时预警时间。

(4) 交通流参数预测模型验证：利用观测点 3 和观测点 4 的第二个视频数据，来验证根据观测点 3 和观测点 4 的第一个视频所构建的有效性，从而评估模型的准确性和可靠性。

这四个要点旨在分析和监测交通流动态，并为实时交通管理提供依据，以避免道路拥堵的发生。通过以上分析，得到问题一的求解思路如图 6 所示。

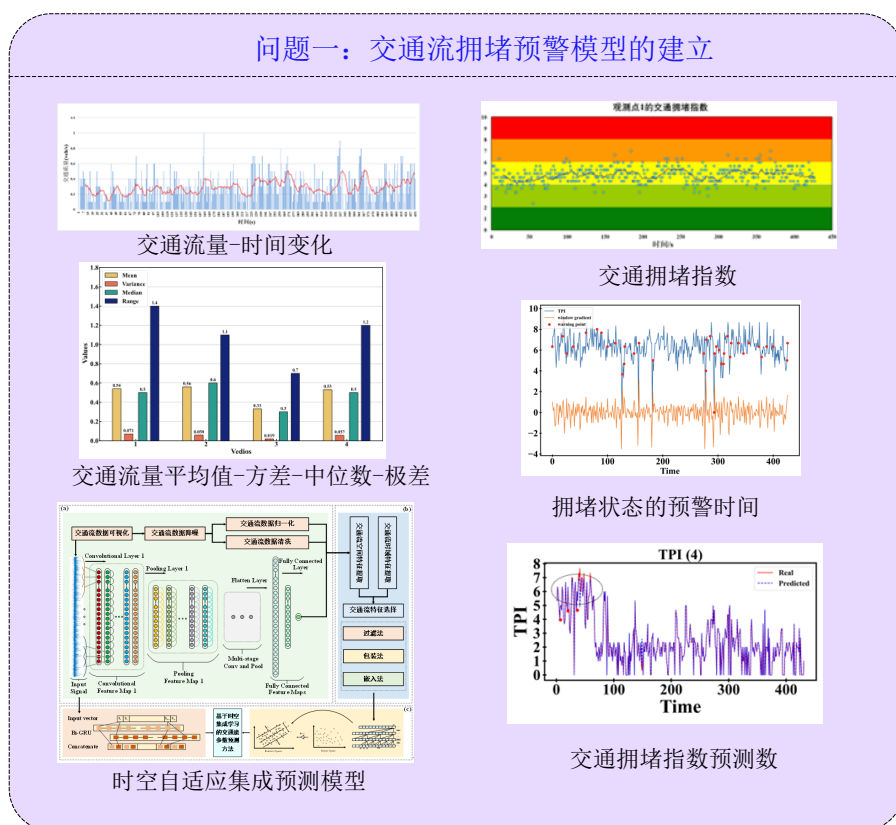


图 6 问题一思路流程图

4.2 交通流参数变化规律分析

交通拥堵是指当交通需求超过道路实际承载能力时，导致车辆行驶速度显著降低的现象。根据产生机理的不同，高速公路拥堵可分为偶发性拥堵和常发性拥堵两种类型，各有其特性：偶发性拥堵由偶然的交通事件诱发，具有随机性，疏散难度大，影响严重；常发性拥堵则在时间和地点上具有规律性，通常发生在交通高峰期、出入口匝道口、收费站和其他道路瓶颈处，发生频率较高，但影响相对较低。无论是哪种拥堵，都会导致车辆速度大幅度降低，进而引起高速公路其他交通参数的变化，这些变化可以通过速度-密度-流量关系曲线来描述，如图 7 所示。

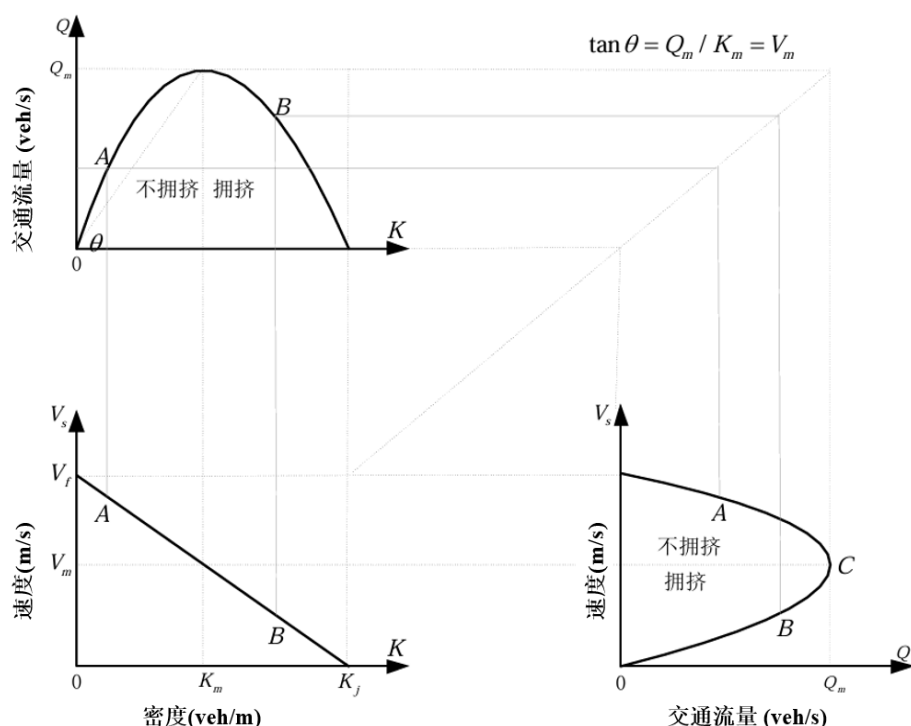


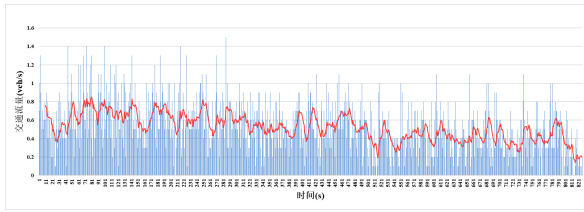
图 7 速度-密度-交通流量关系曲线

4.2.1 交通流量随时间变化分析

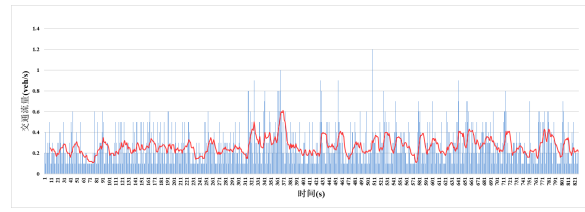
交通流量数是指道路某断面在给定的观测时间内通过的车辆总数，本文计算的是每一帧后的 1 秒内有 25 帧的车辆的数量，即把这 25 帧内重复的车辆删除后在一帧范围中出现的车辆总数。

在观测点 1 的视频 1 中，图中 285 秒左右的时间，单位时间内的车流量达到最大值，此时的车流密度也相应达到最大，时间平均速度逐渐降低，此时，可以开启应急车道。且在观测点 1 的视频 2 中，可以看出后半段交通流量逐渐降低。

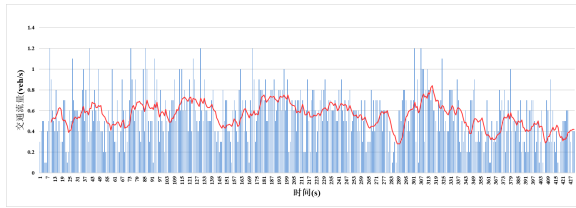
观测点 4 的视频 2 中，交通流量逐渐降低，可能的原因是观测点 4 是服务区入口，而从观测点 1 经过观测点 4，中间都处于封闭状态，在观测点 4 的前段视频即视频 1 中，大部分车辆都已经驶出服务区，所以在观测点 4 的视频 2 中，交通流量逐渐趋于 0。



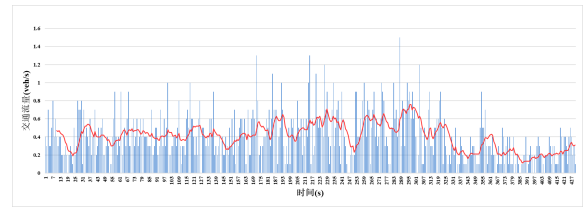
观测点 1 视频 1



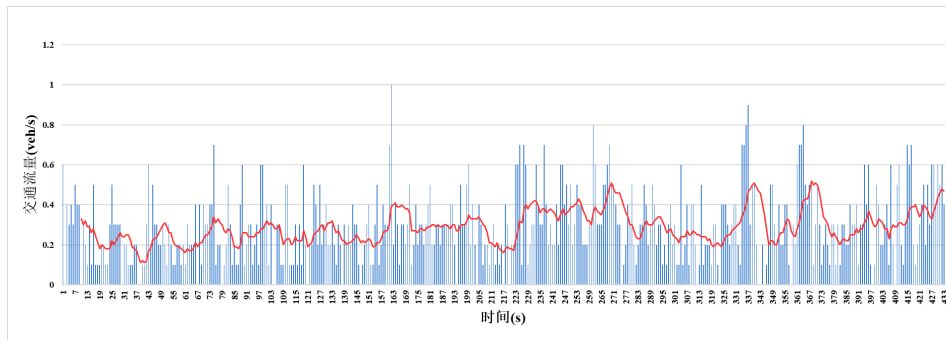
观测点 1 视频 2



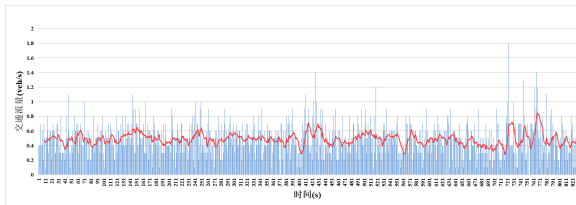
观测点 2 视频 1



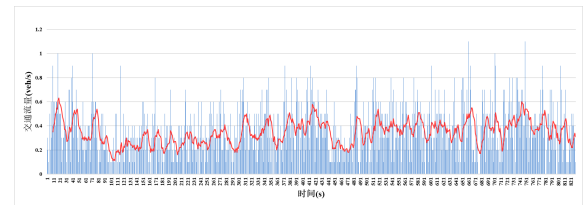
观测点 2 视频 2



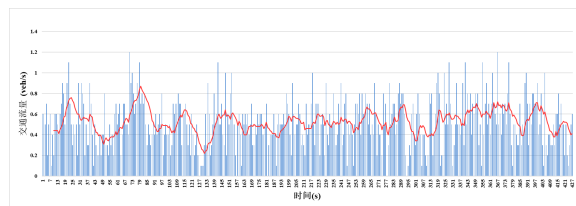
观测点 2 视频 3



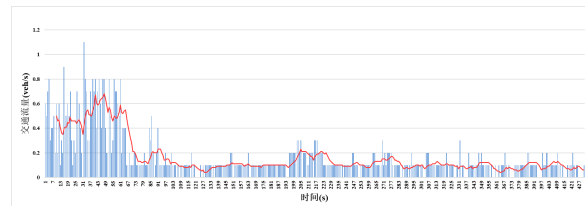
观测点 3 视频 1



观测点 3 视频 2



观测点 4 视频 1



观测点 4 视频 2

图 8 四个观测点交通流量-时间变化图

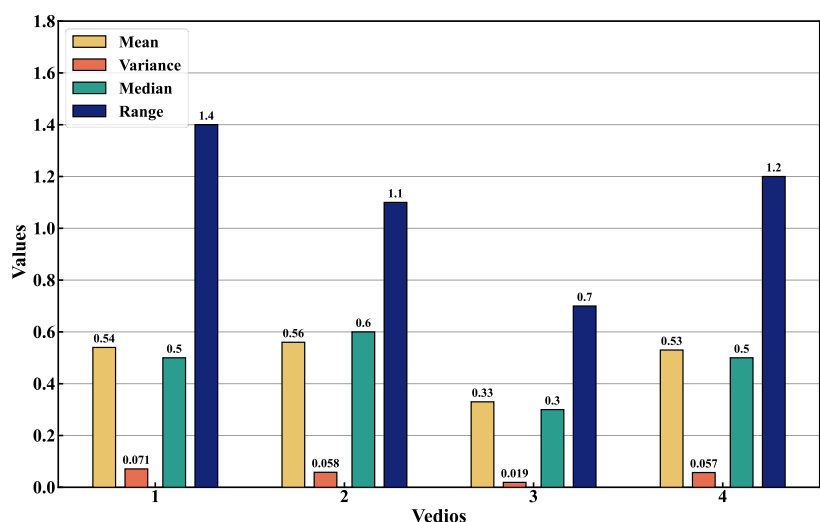


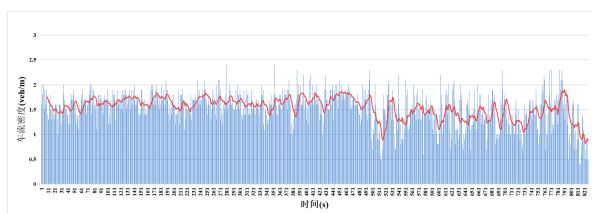
图 9 四个观测点的交通流量平均值-方差-中位数-极差图

4.2.2 车流密度随时间变化分析

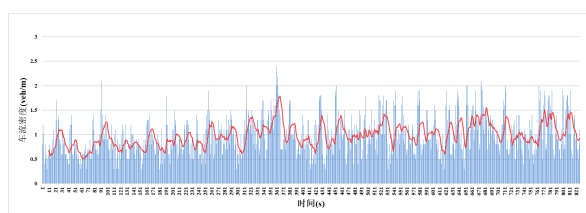
交通流密度是指单位长度道路上在给定的观测时间内通过的车辆密度，本文通过对每一帧的车辆进行统计，计算出每秒内的车流密度。这一过程涉及到对 25 帧中的车辆进行去重，得到在某一帧内出现的车辆总数。

在观测点 1 的视频 1 中，大约在 285 秒时，单位时间内的车流密度达到了最大值，此时的交通流量也随之达到顶峰，而时间平均速度则逐渐降低，这表明此时可以考虑开启应急车道。

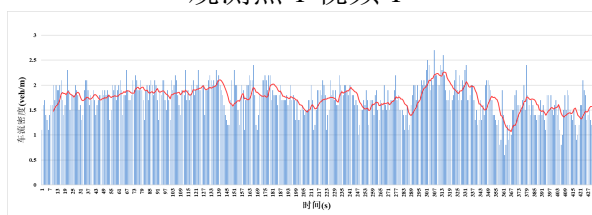
在观测点 4 的视频 2 中，车流密度逐渐降低。可能的原因是观测点 4 位于服务区入口，而从观测点 1 到观测点 4 之间的路段处于封闭状态。在观测点 4 前段的视频 1 中，大部分车辆已经离开服务区，因此在视频 2 中观察到的车流密度逐渐减少。



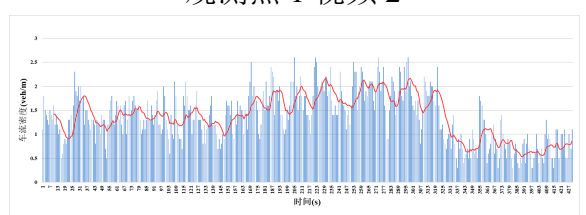
观测点 1 视频 1



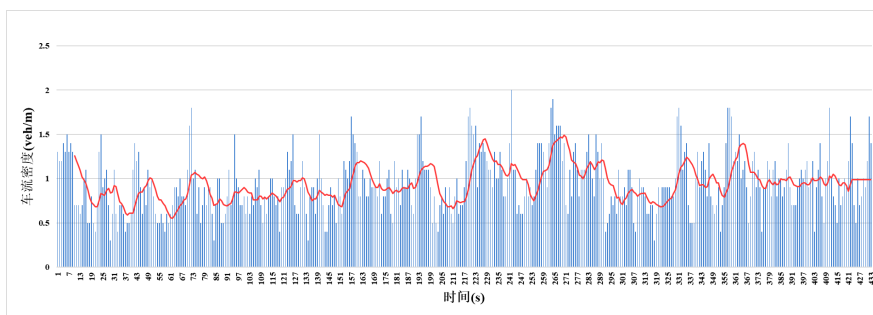
观测点 1 视频 2



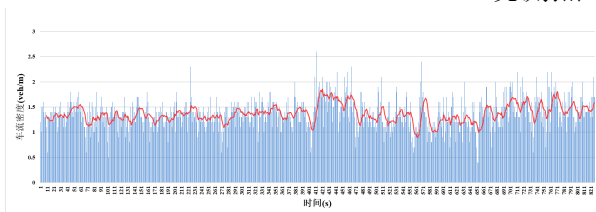
观测点 2 视频 1



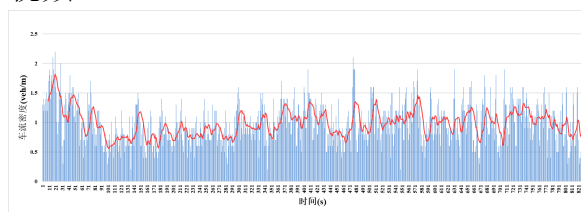
观测点 2 视频 2



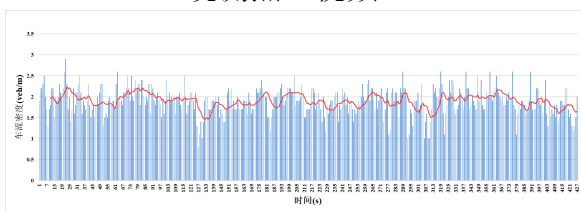
观测点 2 视频 3



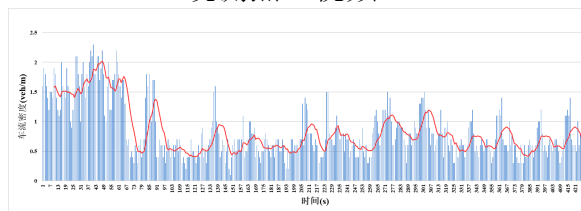
观测点 3 视频 1



观测点 3 视频 2



观测点 4 视频 1



观测点 4 视频 2

图 10 四个观测点车流密度-时间变化图

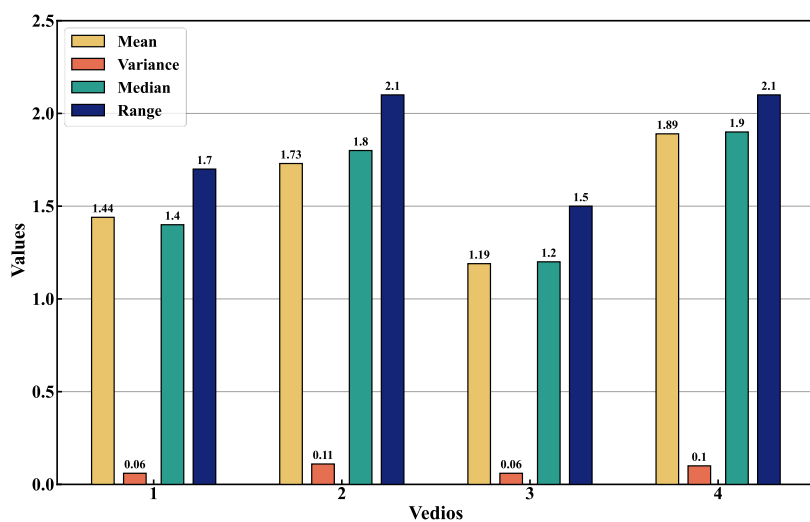


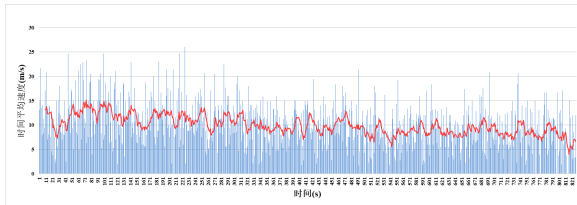
图 11 四个观测点的车流密度平均值-方差-中位数-极差图

4.2.3 时间平均速度随时间变化分析

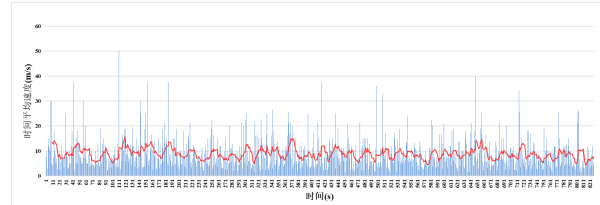
时间平均速度则反映了单位时间内车辆的行驶效率。本文计算每一帧后的 1 秒内，通过 25 帧去除重复车辆后，所呈现的车辆总数，并根据 4.2 中的道路交通流参数的定义计算得出每一秒的时间平均速度。

在观测点 1 的视频 1 中，约在 285 秒时，单位时间内的车流量达到最大，此时的车流密度也相应最高，时间平均速度逐渐下降，表明可以开启应急车道。而在观测点 1 的视频 2 中，后半段的时间平均速度逐渐降低，对应于交通流量和车流密度的减少。

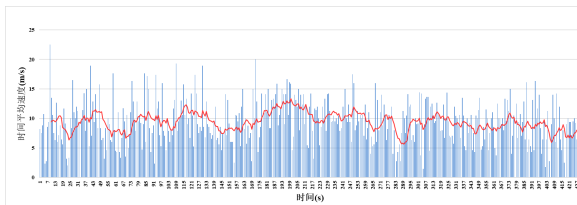
在观测点 4 的视频 2 中，时间平均速度的提升可能与交通流量的降低有关。这是因为观测点 4 位于服务区入口，而从观测点 1 到观测点 4 之间的路段处于封闭状态。在观测点 4 前段的视频 1 中，大部分车辆已经离开服务区，因此在视频 2 中观察到的交通流量逐渐趋于零，导致车辆的时间平均速度逐渐增大。



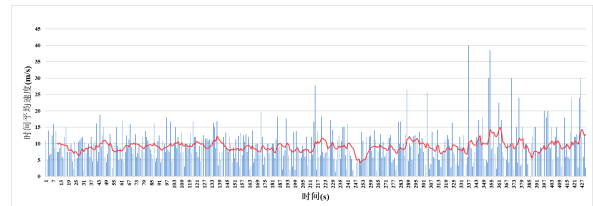
观测点 1 视频 1



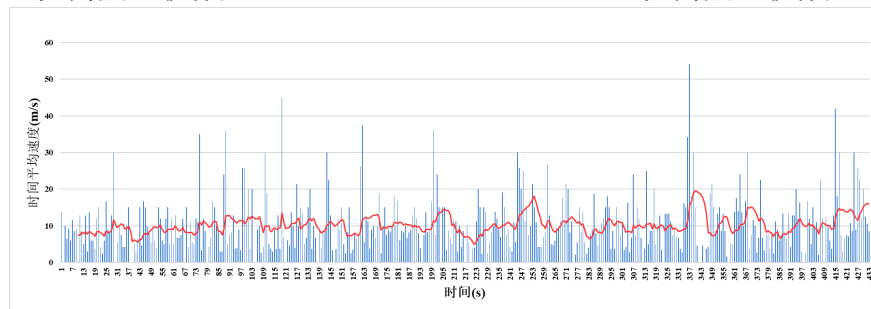
观测点 1 视频 2



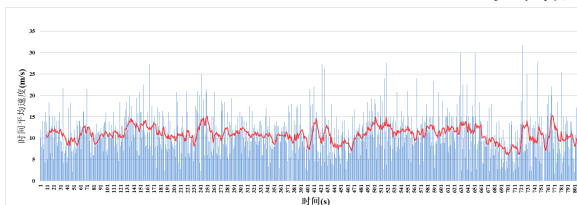
观测点 2 视频 1



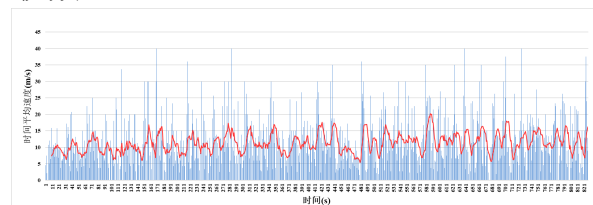
观测点 2 视频 2



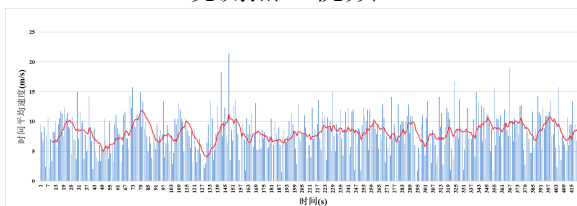
观测点 2 视频 3



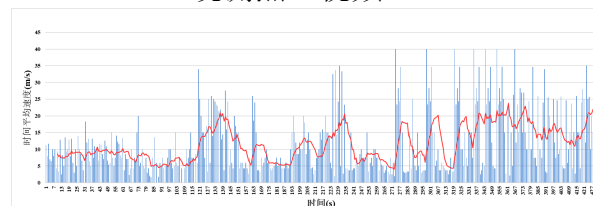
观测点 3 视频 1



观测点 3 视频 2



观测点 4 视频 1



观测点 4 视频 2

图 12 四个观测点时间平均速度-时间变化图

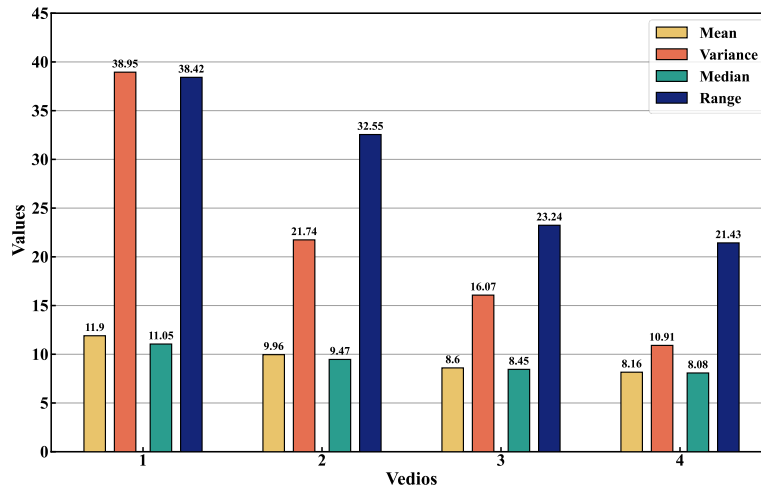


图 13 四个观测点的时间平均速度平均值-方差-中位数-极差图

4.2.4 速度-流量分析

本节将四个观测点所有数据进行汇总，由于每一帧的速度是 0.4 秒，而 25 帧为 1 秒，所以选取每 25 帧进行统计分析，时间平均速度-交通流量变化如图 14 所示。从图中不难看出，随着交通车辆流量的增加，时间平均速度也随之降低，车道越拥堵。其中红色圆圈圈出的异常点，是因为随着交通车流量的增加，观测点 1 处的视频，在中途启用了应急车道，此时，双车道变成了三车道，故即便车流量大，车速并不低。

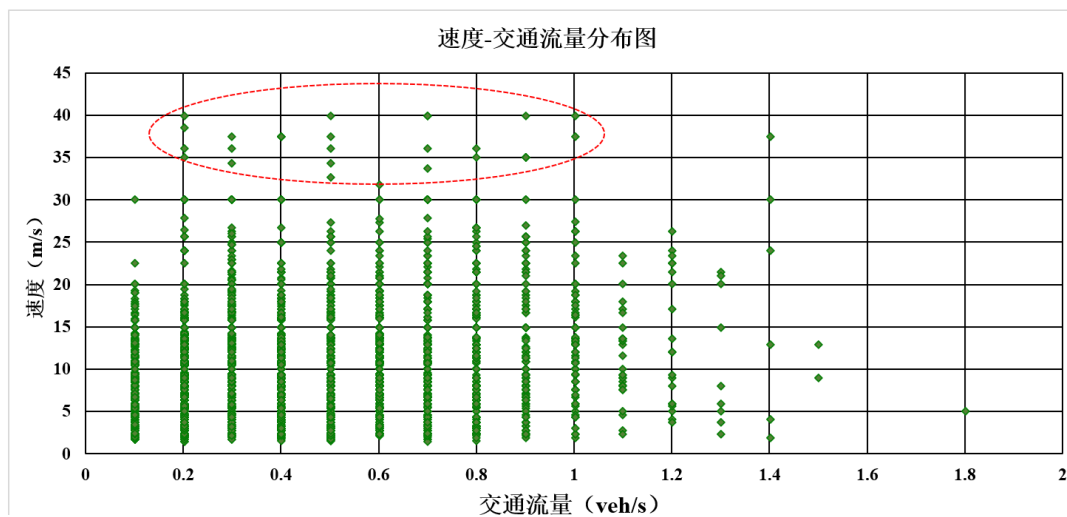


图 14 四个观测点时间平均速度-交通流量变化图

4.3 交通流拥堵模型的建立与求解

4.3.1 交通拥堵指数

交通状态评价标准作为应急车道开放决策模型中动态应急车道开放的先决条件，主要依据交通拥挤程度决定。高速公路的封闭性使得交通流量、车流密度和时间平均速度具有连续流的特性。为此，本文以交通拥堵指数衡量交通拥挤程度，交通拥堵指数^{[3],[4]} (Traffic Performance Index, 即“TPI”)是根据道路通行情况设置的综合反映道路畅通或

拥堵的概念性指数，相当于对拥堵情况进行了量化处理。交通指数取值范围为 1-10，分为 5 个级别(即“畅通”、“基本畅通”、“轻度拥堵”、“中度拥堵”、“严重拥堵”)，数值越高表明交通拥堵状况越严重。具体数据规定如表 3 所示。

表 3 交通拥堵指数分级

拥堵指数	状态	行程时间	拥堵描述
1-2	畅通	T	基本无拥堵
3-4	基本畅通	$1.2\ T - 1.5\ T$	少数路段拥堵
5-6	轻度拥堵	$1.5\ T - 1.8\ T$	部分路段拥堵
7-8	中度拥堵	$1.8\ T - 2.1\ T$	大量路段拥堵
9-10	严重拥堵	$>2.1\ T$	大部分道路拥堵

4.3.2 交通流拥堵模型建立

基于长深高速公路上的视频数据，统计分析后得出的交通流量，车流密度以及占有率，构建交通拥堵模型，如下式所示：

$$\gamma^p(t)=\frac{occ^p(t)^2\times1000}{Q^p(t)\times v^p(t)+0.00001} \tag{7}$$

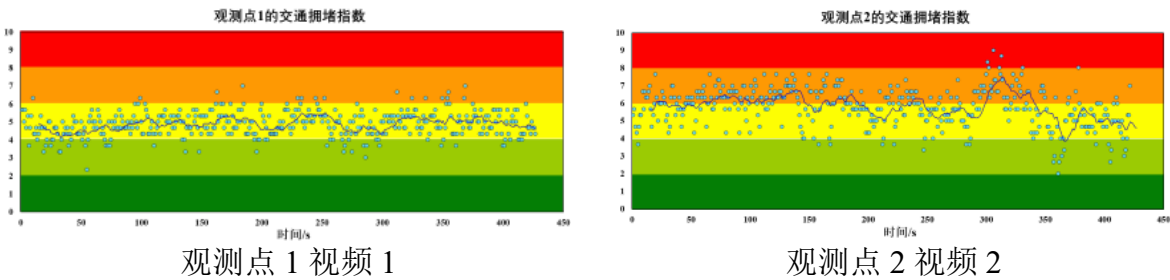
式中， $\gamma^p(t)$ 为在观测点 p 处 t 时刻的对应时段的交通拥堵指数，在本文中， p 取值为 1-4， t 时刻指的是每一帧对应的时刻。 $occ^p(t)$ 为在观测点 p 处 t 时刻的时间占有率，对于每一辆车，记录它开始通过和完全离开观测点的时间； $Q^p(t)$ 为在观测点 p 处 t 时刻的交通流量数； $v^p(t)$ 为在观测点 p 处 t 时刻的时间平均速度。

4.3.3 交通流拥堵模型结果分析

根据公式(7)，本节通过代码得出四个观测点视频对应的交通拥堵指标，在这里因为篇幅原因，观测点 1 和观测点 2 选取了其中某个具有代表性视频。此外，在论文的后续问题中需要具体讨论观测点 3 和观测点 4，因此观测点 3 和观测点 4 所有的视频都列举如下所示。

观测点 3 中的视频 1 和视频 2，中途并没有出入口，虽然观察到该观测点处的两个视频都使用了应急车道，但该段路程还是处于轻度拥堵的状态。

由于观测点 4 全程关闭了应急车道，所以，视频 1 的交通拥堵指数都大于 6。随着观测点 4 中的视频 2 中途打开了闸道出口，交通拥堵指数极具降低。



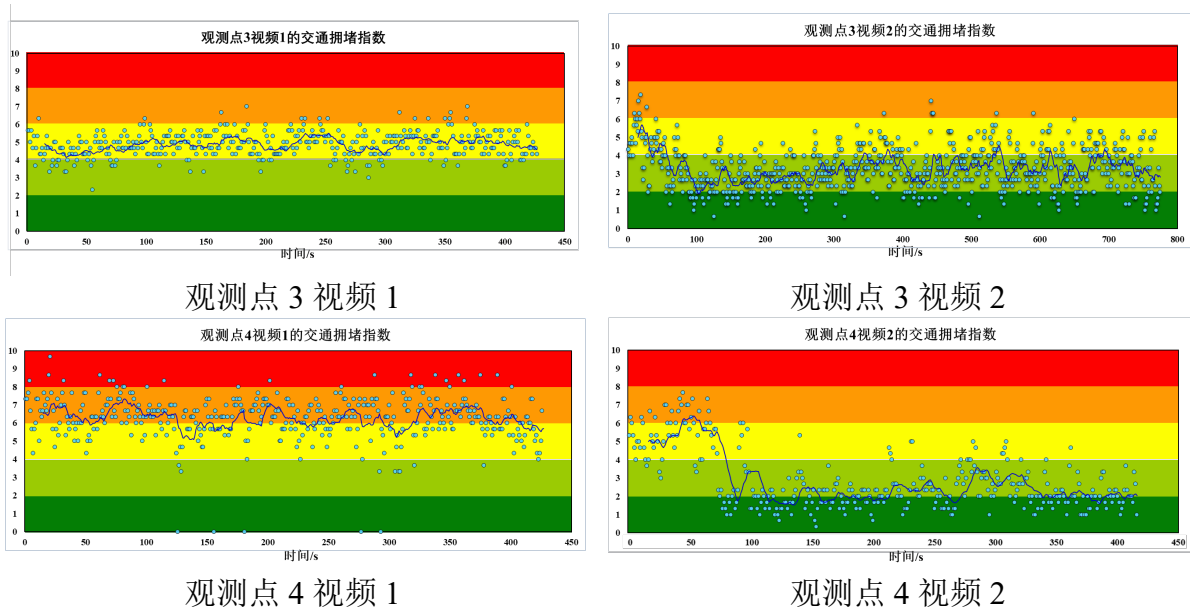


图 15 四个观测点的交通拥堵指数示意图

4.4 交通流拥堵预警模型的建立

4.4.1 基于时空集成学习的时间序列预测方法

支持向量回归作为一种具有坚实理论基础的高泛化性学习方法，适用于各种类型的序列学习场景。但传统支持向量回归模型不擅长于高噪声或特征数超过数据样本数场景下的预测任务。此外，该方法面对时序预测场景下的时空特征学习能力较弱，且对模型参数及核函数的敏感性较高，无法保证模型性能的稳定性。针对上述不足，本文基于支持向量回归模型提出了基于时空集成学习的时间序列预测方法，结构如图 16 所示。

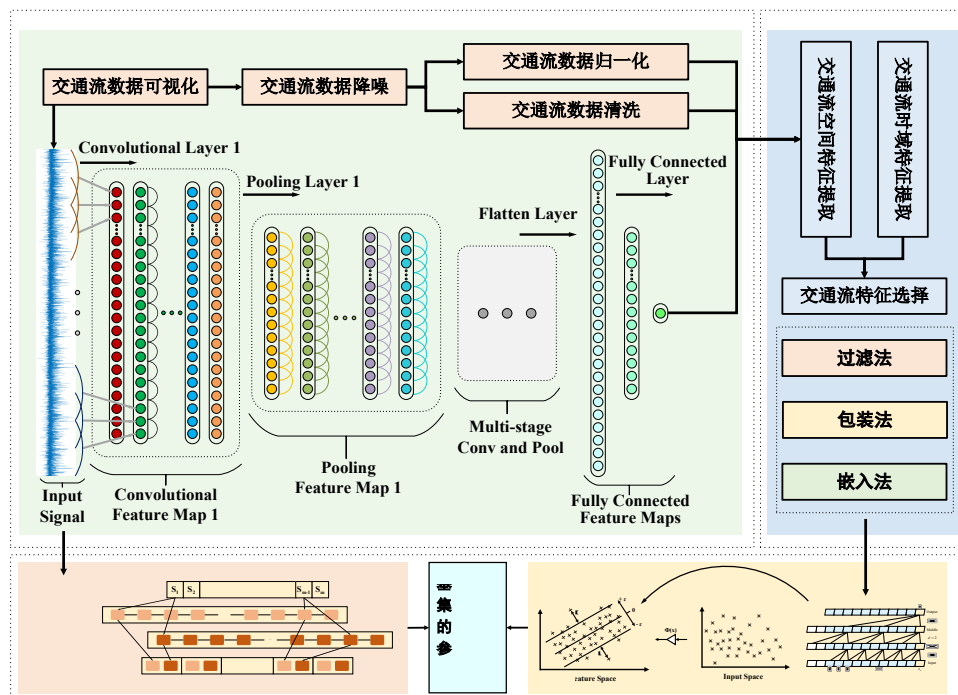


图 16 基于 SVR 的时空自适应集成预测模型结构

该模型主要由时空特征提取模块(a)、特征选择模块(b)和自适应集成预测模块(c)三部分构成。时空特征提取模块能够基于 1D-CNN 构建时空特征提取器充分挖掘数据时空特征。特征选择模块通过结合过滤法和嵌入法筛选出高相关性时空特征。自适应集成预测模块基于时空自适应支持向量回归与双向门控循环单元进行集成学习以捕获时序数据中的长期依赖和局部特征。

(1) 深度特征提取

卷积神经网络 (CNN) 是一种带有卷积结构的深度神经网络, 通常由输入层、卷积层、池化层、全连接层和输出层组成。卷积层中的卷积核通常负责对上一层的输出特征执行卷积操作, 并通过非线性激活函数构造输出特征。卷积层中的每个输出特征都是卷积核与多个输入特征卷积后计算得到的, 卷积操作的数学模型如下式所示。

$$y_i^{l+1}(j) = K_i^l * x^l(j) + b_i^l \quad (8)$$

公式(8)中 K_i^l 和 b_i^l 分别表示第 l 层的第 i 个卷积核的权重和偏置, $x^l(j)$ 表示第 l 层中的第 j 个局部区域, $y_i^{l+1}(j)$ 表示第 $l+1$ 层的第 i 个序列中第 j 个神经元的输入, 符号 $*$ 表示卷积核和局部区域执行点积运算。完成上述卷积运算后, 激活函数将对所有卷积的输出执行非线性变换, 以将线性不可分的原始特征转换到线性可分的新空间中。本文采用 ReLU 函数作为激活函数, 在输入值大于 0 时的导数值恒等于 1, 有效缓解了梯度消失问题。ReLU 函数的数学表达如公式(9)所示。

$$a_i^{l+1}(j) = f(y_i^{l+1}(j)) = \max\{0, y_i^{l+1}(j)\} \quad (9)$$

公式(9)中 $y_i^{l+1}(j)$ 表示卷积运算的输出值, $a_i^{l+1}(j)$ 表示 $y_i^{l+1}(j)$ 的激活值。完成上述激活运算后, 引入池化层来缩小模型的参数规模。在池化层中, 大矩阵通过数据下采样操作转化为小矩阵, 提升了运算效率的同时避免了过拟合现象。本文采用最大池化策略输出感知域中的最大特征值, 最大池化的过程如公式(10)所示。

$$P_i^{l+1}(j) = \max_{(j-1)W+1 \leq t \leq jW} \{q_i^l(t)\} \quad (10)$$

公式(10)中 $q_i^l(t)$ 表示第 l 层第 i 个特征中第 t 个神经元的值 ($t \in [(j-1)W+1, jW]$), W 表示池化感知域的范围, $P_i^{l+1}(j)$ 表示 $l+1$ 层神经元的值。完成多对卷积池化组合操作后, 通过展平操作将最后一个池化层的输出依次拼接为一维向量, 并将其作为全连接层的输入。最终, 全连接层能够将卷积池化组合中的局部信息整合为模型最终的输出, 全连接层的计算过程如公式(11)所示。

$$z^{l+1}(j) = f\left(\sum_{i=1}^m \sum_{t=1}^n W_{ij}^l a_i^l(t) + b_j^l\right) \quad (11)$$

公式(11)中 W_{ij}^l 表示第 l 层第 i 个特征中第 t 个神经元与第 $l+1$ 层中第 j 个神经元之间的权重, $z^{l+1}(j)$ 表示第 $l+1$ 层中第 j 个神经元的值, b_j^l 表示网络偏置, $a_i^l(t)$ 表示第 l 层中第 i 个特征的第 t 个神经元输出值, $f(\cdot)$ 表示激活函数 ReLU。

一维 CNN (1D-CNN) 专门面向一维输入向量场景，一维结构的卷积核使得卷积层和池化层的输出均为一维向量。为了从时间序列中提取除了全局统计特征外所蕴含的深层局部特征，本文提出了一种全新的 1D-CNN 特征提取结构。1D-CNN 能够将预处理后的交通流时间序列数据直接输入模型，接着通过多组卷积池化组合挖掘出时序数据中所隐藏的深层非线性特征，最后依托全连接层强大的特征整合能力完成自适应特征学习。图 17 中的虚线区域展示了本文所提出 1D-CNN 深度特征提取模型的具体结构，该模型由六个卷积层、六个池化层和三个全连接层组成。交通流数据在卷积池化组合中连续六次被转换为特征图，并通过最大池化进行下采样操作得到最终的特征图。最后将该特征图输入全连接层模块完成展平及局部特征信息整合。卷积核的大小在第 1 层为 64×1 ，第 2 层为 32×1 ，第 3 层和第 4 层为 16×1 ，第 5 层和第 6 层为 8×1 。池化核的大小在第 1 层为 16×1 ，第 2 层为 4×1 ，第 3 - 6 层为 2×1 。全连接层的节点数分别为 100、50 和 1。

在上述交通流特征提取结构的基础上，本文提出了一种基于 1D-CNN 的交通流时间序列特征融合提取模型，该模型将不同属性的交通流时间序列数据作为输入，对这些数据进行智能特征融合，用于 TPI 的预测。基于 1D-CNN 的交通流特征融合模型的整体结构及相关参数如图 17 及表 4 所示。

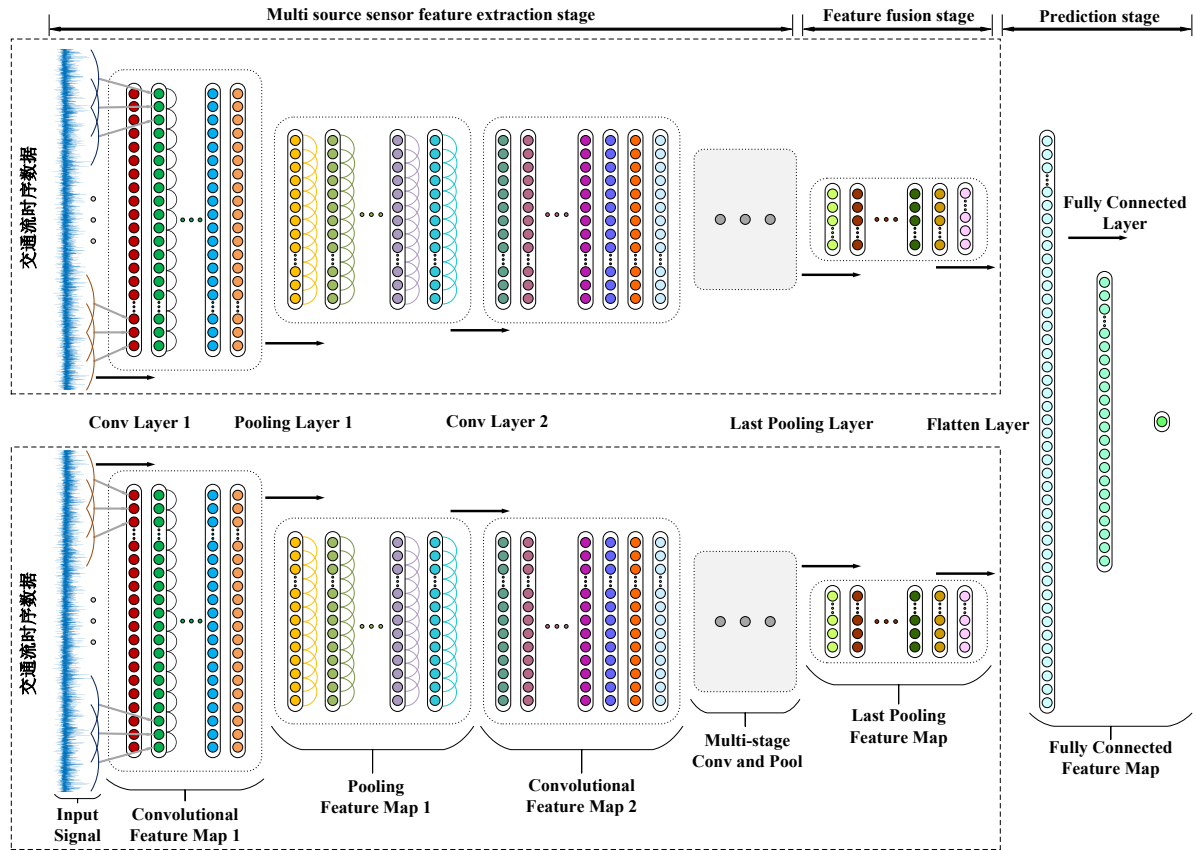


图 17 基于 1D-CNN 的交通流时序数据特征融合模型

具体来说，基于 1D-CNN 的交通流时序数据特征融合模型主要分为三个阶段：交通流特征提取阶段、特征融合阶段和局部特征整合阶段。该模块使用 6 层卷积池化组合结构进行交通流特征提取，连续六次对交通流时间序列数据执行独立的卷积和池化操作。特征融合阶段通过对所提取的全部交通流特征执行展平操作以实现特征融合过程。最后在局部特征整合阶段，融合特征被导入到全连接层，依托其强大的整合能力完成自适应局部特征学习。

表 4 特征融合模型结构参数

序号	层类型	核尺寸/步长	核通道尺寸
1	Convolution 1	$64 \times 1 / 1 \times 1$	16
2	Pooling 1	$16 \times 1 / 16 \times 1$	16
3	Convolution 2	$32 \times 1 / 1 \times 1$	64
4	Pooling 2	$4 \times 1 / 4 \times 1$	64
5	Convolution 3	$16 \times 1 / 1 \times 1$	128
6	Pooling 3	$2 \times 1 / 2 \times 1$	128
7	Convolution 4	$16 \times 1 / 1 \times 1$	128
8	Pooling 4	$2 \times 1 / 2 \times 1$	128
9	Convolution 5	$8 \times 1 / 1 \times 1$	128
10	Pooling 5	$2 \times 1 / 2 \times 1$	128
11	Convolution 6	$8 \times 1 / 1 \times 1$	128
12	Pooling 6	$2 \times 1 / 2 \times 1$	128
13	Fully-connected 1	100	1
14	Fully-connected 2	50	1
15	Fully-connected 3	1	1

(2) 混合特征选择

特征选择通过删除不相关、冗余或嘈杂的特征，从所提取的与输出变量相关的原始特征中选择最相关特征的过程。原始高维数据会致使模型计算时间增加、维数灾难和过拟合等问题，最终导致模型精度的丢失。有效的特征选择是机器学习模型进行预测训练的关键部分，因此使用有效的特征选择方法从提取的原始特征中选取相关性最高的特征至关重要。特征选择方法通常分为过滤法、包装法和嵌入法。

过滤法通常不依赖于模型，仅站在数据的角度进行特征筛选，该方法可细分为两种类型。第一种类型根据特征所包含的信息量进行评判，例如方差选择法根据特征数据的方差来衡量特征的有用性。第二种类型根据每个特征与目标特征的相关度进行评判，剔除相关度低于所设定阈值的特征。特征相关度的计算方式包括 Pearson 相关系数、卡方验证、互信息和最大信息系数等。过滤法的效率高，过拟合风险小，但是可能会出现特征冗余，未考虑到特征间的相互作用。不同于过滤法，包装法根据模型训练后在测试集上的性能来进行特征选择。该方法将数据划分为训练集和测试集，搜索出使得训练后的

模型在测试集上性能最优的特征子集。最优特征集合的搜索方法包括暴力搜索、贪心搜索和启发式搜索三种。相较于过滤法，包装法所筛选出的特征均能够提升模型的性能，但是特征集合搜索过程需要经历大批次的训练，效率低下，仅适用于特征及样本量较少的场景。嵌入法将特征选择嵌入到模型训练过程中，模型训练结束后即可得到有用特征。嵌入法同时兼顾了嵌入法和包装法的优点，运行效率适中且过拟合风险较低。

本文综合考虑各特征选择方法的优缺点，选用过滤法和嵌入法相结合的混合特征选择方法使用皮尔逊相关系数（PCC，过滤法）及随机森林（RF，嵌入法）进行特征选择。皮尔逊相关系数被用于度量每个特征与交通流之间的相关程度，该方法的具体计算过程如公式(12)所示。

$$PCC(r) = \frac{\sum (p_i - \bar{p})(q_i - \bar{q})}{\sqrt{\sum_{i=1}^n (p_i - \bar{p})^2} \sqrt{\sum_{i=1}^n (q_i - \bar{q})^2}} \quad (12)$$

公式(12)中 p_i 代表交通流真实值， $\bar{p}$ 代表真实交通流的平均值， q_i 代表所度量的特征因子， $\bar{q}$ 代表该特征列的平均值。根据每个特征与交通流之间的相关性，皮尔森相关系数的取值范围为 $[-1, 1]$ ，其绝对值越接近 1 代表当前特征与交通流之间的正相关性或负相关性越强。随机森林回归作为一种嵌入式特征选择方法，综合了过滤法和包装法的优点，在提高算法效率的同时降低了过拟合的风险。随机森林由多棵从数据中随机提取特征构建而成的决策树组成，其排名取决于树的相关算法对节点纯度带来的增益大小。树开头的节点纯度增益最大，而树末尾的节点纯度增益最小。随机森林能够针对特定节点下的树进行修剪通过修剪以构建关键特征集合。随机森林中的决策树均未将所有的观察值或特征纳入考虑范围，显著降低了该方法的过拟合风险程度。

（3）时空自适应支持向量回归

支持向量机（SVM）是一种基于统计学习理论和 VC 维理论的机器学习算法。该算法在特征空间搜寻不同种类样本的最优分类超平面，适用于多因素回归拟合问题。SVM 通过核函数将输入空间中的非线性分类问题转化为高维空间中的线性分类问题，克服了维数灾难和非线性可分问题。支持向量机最终的决策函数仅由少数支持向量所确定，因此算法的计算复杂度取决于支持向量的数目而非样本空间维数，具有较强的鲁棒性。SVM 在分类和预测场景中分别对应于支持向量分类和支持向量回归。本文针对 TPI 预测场景，在传统支持向量回归模型的基础上提出了时空自适应支持向量回归。在时空自适应支持向量回归中，我们引入了时间卷积核的概念，各核函数分量对预测过程的影响程度取决于交通流相关特征。通过时间卷积网络（TCN）组件从交通流相关特征中学习出最优的时间卷积核参数，并通过时间卷积核将输入空间映射到特征空间，并在特征空间中找到最优分类曲面以最小化其中缺失训练样本集的误差。首先定义输入的样本数据如下式所示。

$$T = \{(x_1, y_1), \dots, (x_n, y_n)\}, i = 1, 2, \dots, n \quad (13)$$

公式(13)中, $x_i \in X = R^n$, R^n 代表输入向量; $y_i \in Y = R$, R 代表输出向量。预测回归问题通常聚焦于寻找关于 R^n 的函数 $f(x)$, 使得通过 $f(x)$ 能够预测任意输入值 x_i 所对应的输出值。假设在高维特征空间中建立的线性回归函数 $f(x)$ 形式如下式所示。

$$f(x) = w^* \Phi(x) + b \quad (14)$$

公式(14)中 $\Phi(x)$ 表示非线性映射函数, ε 线性不敏感损失函数的定义如下式所示。

$$L(f(x), y, \varepsilon) = \begin{cases} 0, & |y - f(x)| \leq \varepsilon \\ |y - f(x)| - \varepsilon, & |y - f(x)| > \varepsilon \end{cases} \quad (15)$$

引入松弛变量 ξ_i 和 ξ_i^* , 求解线性回归函数 $f(x)$ 中 w 和 b 的数学表达如下式所示。

$$\begin{cases} \min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \\ \text{s.t.} \begin{cases} y_i - w^* \Phi(x_i) - b \leq \varepsilon + \xi_i \\ -y_i + w^* \Phi(x_i) + b \leq \varepsilon + \xi_i^* \\ \xi_i \geq 0, \quad \xi_i^* \geq 0 \end{cases} \end{cases} \quad i = 1, 2, \dots, n \quad (16)$$

惩罚因子较大表明训练误差较大, 样本惩罚 ε 较大; 较小的 ε 则说明回归函数的误差较小。紧接着通过引入拉格朗日函数将上述数学表达转化为如下式所示的对偶形式数学表达。

$$\begin{aligned} \max & \left[-\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) K(x_i, x_j) \right. \\ & \left. - \sum_{i=1}^n (\alpha_i + \alpha_i^*) \varepsilon + \sum_{i=1}^n (\alpha_i - \alpha_i^*) y_i \right] \\ \text{s.t.} & \begin{cases} 0 \leq \alpha_i \leq C \\ \sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0 \\ 0 \leq \alpha_i^* \leq C \end{cases} \end{aligned} \quad (17)$$

公式(17)中 $K(x_i, x_j) = \Phi(x_i) \Phi(x_j)$ 代表核函数。核函数本质上是一种能够将原始非线性输入数据投影到高维特征中以其能够线性表示的投影函数。核函数必须满足 Mercer 定理, 该定理的数学表达如下式所示。

$$K(\vec{x}, \vec{x}^*) \in L_2(\mathbb{R}^n) \otimes L_2(\mathbb{R}^n) \quad (18)$$

$$\iint_{L_2 \otimes L_2} K(\vec{x}, \vec{x}^*) g(\vec{x}) g(\vec{x}^*) d\vec{x} d\vec{x}^* \geq 0, \exists \forall g(x) \in L_2(\mathbb{R}^n) \quad (19)$$

在 Mercer 定理的基础上, 核函数可细分为全局核函数和局部核函数。全局核函数允许间隔较远的数据点影响内核值, 而局部核函数只允许彼此接近的数据点影响内核值。具体来说, 多项式核函数和径向基核函数分别是全局核函数和局部核函数的典型例子。对于多项式核函数, 次数越高, 其插值能力越好; 对于径向基核函数, γ 值越小, 其插值能力越差。综上所述, 单个核参数 γ 或多项式次数无法提供兼具插值和外推特性的模型, 几种不同类型的经典核函数如表 5 所示。

表 5 经典核函数模型及其参数

核函数	数学模型	C	γ	d	ε	m
线性核	$K(x_i, x_j) = \langle x_i, x_j \rangle + c$	$2^0 - 2^7$	$\times$	$\times$	5×10^{-4}	$\times$
多项式核	$K(x_i, x_j) = (\gamma \langle x_i, x_j \rangle + c)^d$	$2^{-4} - 2^4$	$2^{-3} - 2^3$	2	10^{-4}	$\times$
径向基核	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$	$2^{-3} - 2^3$	$2^{-15} - 2^{15}$	$\times$	10^{-3}	$\times$
Sigmoid核	$K(x_i, x_j) = \tanh(\gamma \langle x_i, x_j \rangle + c)$	$2^{-0} - 2^{10}$	$2^{-6} - 2^1$	$\times$	10^{-1}	$\times$
混合核	$K_{\text{mix}} = \rho K_{\text{poly}} + (1 - \rho) K_{\text{rbf}}$	$10^{-2} - 10^5$	$10^{-3} - 10^4$	1-4.5	$10^{-4} - 10^{-1}$	0-1

假设 $K_1(\vec{x}, \vec{x}^*)$ 和 $K_2(\vec{x}, \vec{x}^*)$ 均为符合 Mercer 定理规则的支持向量核函数，且有 $C_1, C_2 \geq 0$ ，则不同核函数的线性组合也是一个合法的核函数。混合核函数 $K(\vec{x}, \vec{x}^*)$ 结合了 $K_1(\vec{x}, \vec{x}^*)$ 和 $K_2(\vec{x}, \vec{x}^*)$ 的优良特性，能够有效提升模型的拟合及泛化能力，混合核函数的数学描述如下式所示。

$$K(\vec{x}, \vec{x}^*) = C_1 K_1(\vec{x}, \vec{x}^*) + C_2 K_2(\vec{x}, \vec{x}^*) \quad (20)$$

基于上述混合核函数的推论，本文将时间卷积网络 (TCN) 引入核函数的构建过程，提出了能够根据所输入交通流相关特征选择最优核函数组合的时间卷积核。在交通流预测过程中，任意输入样本点的核函数仅与当前及历史输入相关，与未来输入不存在较显著的关联。因此 TCN 中的因果卷积使得时间卷积核仅受到当前及历史的交通流相关特征影响，而与未来的交通流特征无关。TCN 的因果性使其需要设计非常深的网络结构才能够捕获悠久的历史信息，严重降低了模型的效率。因此，在 TCN 中引入膨胀卷积以使其在深层网络结构中仍能够高效提取悠久的历史信息。膨胀卷积能够将 TCN 的感受野实现指数级的放大，其网络结构如图 18 所示。

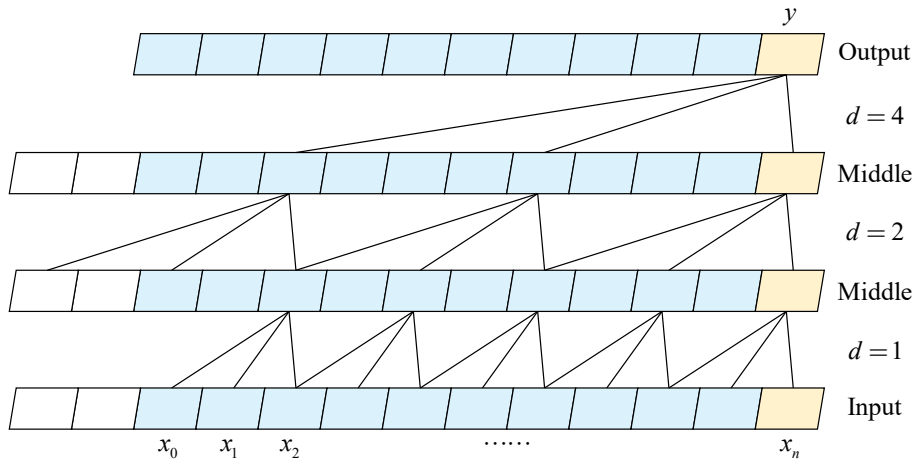


图 18 膨胀因果卷积网络结构

膨胀卷积在卷积核中引入了膨胀因子，使网络能够在相同的计算复杂度下观察到更长的历史时间序列。对于一维时间序列 x 和滤波器 f ，膨胀卷积在时间序列元素上的处理过程如公式(21)所示。

$$F(s) = (x * f)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i} \quad (21)$$

式中， $*$ 代表卷积运算， k 代表滤波器的尺寸， d 代表膨胀因子， $s-d \cdot i$ 代表核函数仅与当前及历史输入相关。由此可见，膨胀本质上就是在两个相邻滤波器抽头间设置固定的步长。当 $d=1$ 时，膨胀卷积变为常规卷积。随着膨胀因子 d 的增加，网络能够考虑更广泛的历史信息，有效增加了 TCN 的感受野。

在本文所提出的时间卷积核中，膨胀因子 d 随着网络的层数呈指数型增长，确保核函数能够较好的描述总体数据特征。残差网络通常被用于解决卷积神经网络层数过多所导致的性能衰减问题。在时间卷积核中，因果卷积和膨胀卷积的叠加使得训练过程中出现梯度衰减及梯度消失的可能性大幅上升。因此在 TCN 的输出层引入残差连接，将一维时间序列 x 与卷积网络输出融合的数学表达如下式所示。

$$o = \text{Activation}(x + F(x)) \quad (22)$$

公式(22)中 $F(x)$ 代表卷积网络的输出， $\text{Activation}(\cdot)$ 代表激活函数。在 TCN 的残差块中，TCN 包含两个膨胀的因果卷积层。考虑到去线性化问题，选择修正线性单元(ReLU)作为激活函数。权重归一化作用于卷积滤波器以对深度学习网络的权重进行归一化。此外，在每个膨胀卷积后添加一个 dropout 层以减小过拟合的可能性。最后，通过一个全连接层得到最终的时间卷积核参数，TCN 残差块的具体结构如图 19 所示。

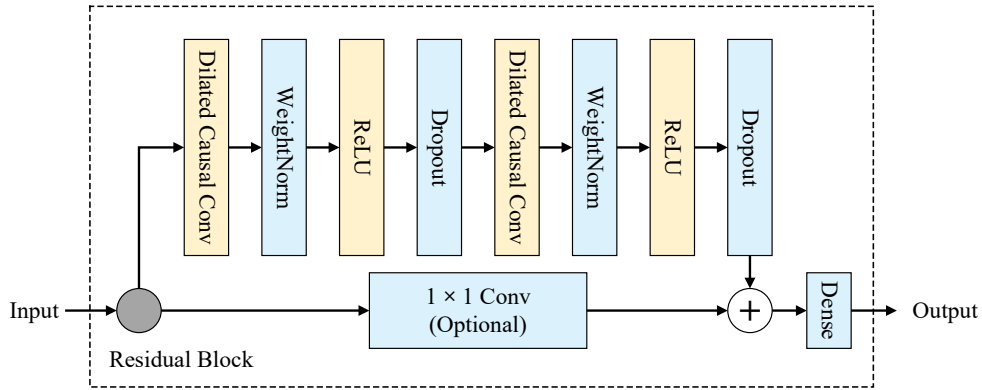


图 19 TCN 残差块结构

假设公式(17)的最优解为 $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_n]$, $[\alpha_1^*, \alpha_2^*, \dots, \alpha_n^*]$ ，则 w^* 和 b^* 的数学表达如公式(23)-(24)所示。

$$w^* = \sum_{i=1}^n (\alpha_i - \alpha_i^*) * \Phi(x) \quad (23)$$

$$b^* = \frac{1}{N} \left\{ \sum_{0 < \alpha_i < C} [y_i - \sum (\alpha_i - \alpha_i^*) K(x_i, x_j) - \varepsilon] + [y_i - \sum (\alpha_i - \alpha_i^*) K(x_i, x_j) + \varepsilon] \right\} \quad (24)$$

综上所述，回归函数的数学表达如公式(25)所示。支持向量回归算法的关键问题是如何求解参数 α_i 和 α_i^* 。上述参数通常由顺序最小优化方法确定，而支持向量回归模型的性能受到惩罚因子 C 、精度 ε 和核函数方差 σ 等超参数的极大影响。因此在超参数搜索过程中引入智能优化算法以科学有效地设定上述超参数。

$$f(x) = w^* \Phi(x) + b^* = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \Phi(x_i) + b \quad (25)$$

为了获得 SVR 的最优超参数组合，本文采用粒子群优化算法对 STA-SVR 的参数选择过程进行优化。粒子群优化算法 (PSO) 是一种可用于解决优化问题的元启发式技术。粒子根据其自身的历史最优位置及所有粒子的历史全局最优位置进行位置和速度的更新。假定 Φ 是粒子的搜索空间，即向量 (γ, C, ε) 的取值范围， n 是群大小。 (γ, C, ε) 的范围分别设置为 $[0, 1000]$ 、 $[1, 10000]$ 和 $[0, 50]$ 。每个粒子都有自己的位置 $\vec{x}_i$ 和速度 $\vec{v}_i$ ，其中， $\vec{x}_i$ 对应于 (γ, C, ε) 。粒子是否优秀取决于其适应度函数，即使用该粒子参数组合训练 STA-SVR 所得到预测结果的 RMSE 值。粒子每次迭代都会根据自身的历史最优位置 $\vec{b}_i$ 与历史全局最优位置 $\vec{g}$ 更新其位置和速度，其位置和速度的更新过程如下式所示。迭代在达到最大迭代次数或 RMSE 小于预设值时终止，位置 $\vec{g}$ 表示所期望参数组合的输出。

$$\vec{v}_i^{t+1} = w\vec{v}_i^t + \varphi_1 \vec{U}_1^t (\vec{b}_i - \vec{x}_i^t) + \varphi_2 \vec{U}_2^t (\vec{g} - \vec{x}_i^t) \quad (26)$$

$$\vec{x}_i^{t+1} = \vec{x}_i^t + \vec{v}_i^{t+1} \quad (27)$$

公式(26)-(27)中 $\vec{v}_i^t$ 和 $\vec{x}_i^t$ 分别表示第 t 次迭代时第 i 个粒子的速度和位置； w 代表惯性因子，表示粒子对于先前飞行方向的记忆，可防止飞行方向的剧烈变化； φ_1 表示模拟粒子返回到历史最佳位置趋势的认知因素； φ_2 表示量化后粒子相对于全局最优粒子性能的社会因素； $\vec{U}_1$ 和 $\vec{U}_2$ 均为 $n \times n$ 对角矩阵，其中主对角线是均匀分布在区间 $[0, 1]$ 中的随机数。STA-SVR 的输入输出、初始化及迭代等详细流程如算法 1 所示。

算法 1: STA-SVR 的参数优化过程

Input: 训练集，群体大小 n ，惯性因子 w ，认知因素 φ_1 ，社会因素 φ_2 ，最大迭代次数 T ，粒子的搜索空间 Φ 。

Output: (γ, C, ε) 的最佳组合。

- 1: //初始化:
 - 2: $t = 0$
 - 3: 将粒子位置 $\vec{x}_i^0$ 初始化为搜索空间 Φ 范围内的值
 - 4: 将粒子速度 $\vec{v}_i^0$ 初始化为 0 或者非常小的随机浮点数
 - 5: 通过适应度函数计算各粒子的适应度值 $\text{frmse}(\vec{x}_i^0)$
 - 6: 使用当前粒子位置 $\vec{x}_i^0$ 初始化粒子历史最优位置 $\vec{b}_i$
 - 7: 使用当前粒子适应度值 $\text{frmse}(\vec{x}_i^0)$ 初始化粒子历史最优适应度值 fitness
-

```

8:   For  $t=1$  to  $T$  step Do:
9:       通过粒子历史最优位置  $\vec{b}_i$  更新历史全局最优位置  $\vec{g}$ 
10:      随机生成对角矩阵  $\vec{U}_1'$  和  $\vec{U}_2'$ 
11:      更新  $n$  个粒子的位置  $\vec{x}_i' = \vec{x}_i^{t-1} + \vec{v}_i'$ 
12:      更新  $n$  个粒子的速度  $\vec{v}_i' = w\vec{v}_i^{t-1} + \varphi_1\vec{U}_1'(\vec{b}_i - \vec{x}_i^{t-1}) + \varphi_2\vec{U}_2'(\vec{g} - \vec{x}_i^{t-1})$ 
13:      通过适应度函数计算各粒子的适应度值  $\text{frmse}(\vec{x}_i')$ 
14:      若  $\text{frmse}(\vec{x}_i') \leq \text{fitness}$ ,  $\vec{b}_i = \vec{x}_i'$ ,  $\text{fitness} = \text{frmse}(\vec{x}_i')$ 
15:   End for

```

Return γ , C and ε

在 STA-SVR 的参数优化算法中, 首先分别对迭代计数器 t 、粒子位置 $\vec{x}_i'$ 、粒子速度 $\vec{v}_i'$ 和粒子历史最优位置 $\vec{b}_i$ 进行初始化。将迭代计数器设置为 0, 粒子位置初始化为搜索空间内的随机值, 粒子速度初始化为 0 或者非常小的随机浮点数, 粒子历史最优位置 $\vec{b}_i$ 设置为粒子的初始化位置。完成初始化操作后, 进行最优解迭代寻优过程, 首先将迭代计数器更新为 1 并用当前粒子的历史最优位置 $\vec{b}_i$ 更新历史全局最优位置 $\vec{g}$ 。接着, 若迭代计数器 t 小于最大迭代次数 T 则更新 n 个粒子的位置、速度及历史最优位置, 反之则终止迭代并输出历史全局最优位置 $\vec{g}$ 。最后将位置更新后粒子的适应度与该粒子的历史最优适应度进行比较, 若当前适应度值小于历史最优适应度则将历史最优位置更新为粒子当前位置。完成上述单轮迭代操作后, 将迭代计数器增加一次并继续更新全局历史最优位置直至算法结束并输出全局最优参数组合。

(4) 双向门控循环单元

循环神经网络 (RNN) 能够记忆历史数据特征, 适用于各种序列数据问题。门控循环单元 (GRU) 神经网络使用更新门和重置门选择性地过滤和记忆历史信息, 解决了传统 RNN 的梯度消失、梯度爆炸及长期记忆力差等问题。由于数据的时序特征, 本文使用 GRU 提取交通流特征。GRU 模型的内部详细结构如图 20 所示, 其中 h_{t-1} 和 h_t 分别代表前一个单元和当前单元的隐藏状态; $\tilde{h}_t$ 代表当前单元的候选状态; x_t 代表当前单元的输入张量。具体来说, h_t 和 $\tilde{h}_t$ 是通过 h_{t-1} 和 x_t 求解得到的, 其计算过程如公式(28)-(31)所示。

$$r_t = \sigma(W_r \cdot [h_{t-1}; x_t]) \quad (28)$$

$$z_t = \sigma(W_z \cdot [h_{t-1}; x_t]) \quad (29)$$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}; x_t]) \quad (30)$$

$$h_t = h_{t-1} \odot (1 - z_t) + \tilde{h}_t \quad (31)$$

公式(28)-(31)中 r_t 和 z_t 分别代表重置门和更新门; W_r 、 W_z 和 W_h 代表权重矩阵; h_t 由历史及当前的状态信息组成; $h_{t-1} \odot (1 - z_t)$ 代表对选择性遗忘历史信息; $\tilde{h}_t \odot \mathbf{Z}_t$ 代表选择性记忆当前信息。当使用 GRU 处理数据时, 从先前单元所继承历史信息的清晰度与序

列长度呈负相关。为了增强时序数据各维度间的相关性，引入了双向 GRU 及多头自注意力机制。时序数据分别从正向和反向输入双向 GRU，其网络及单元的具体结构如图 21 所示。双向 GRU 将所获取的正向及反向输出特征合并为输出特征图，增强了时序数据各维度间的相关性。

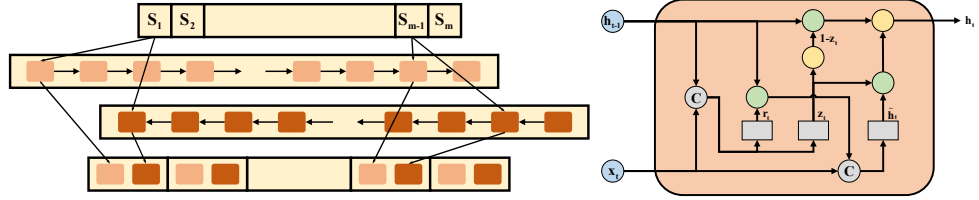


图 20 双向 GRU 网络结构及 GRU 单元详细结构

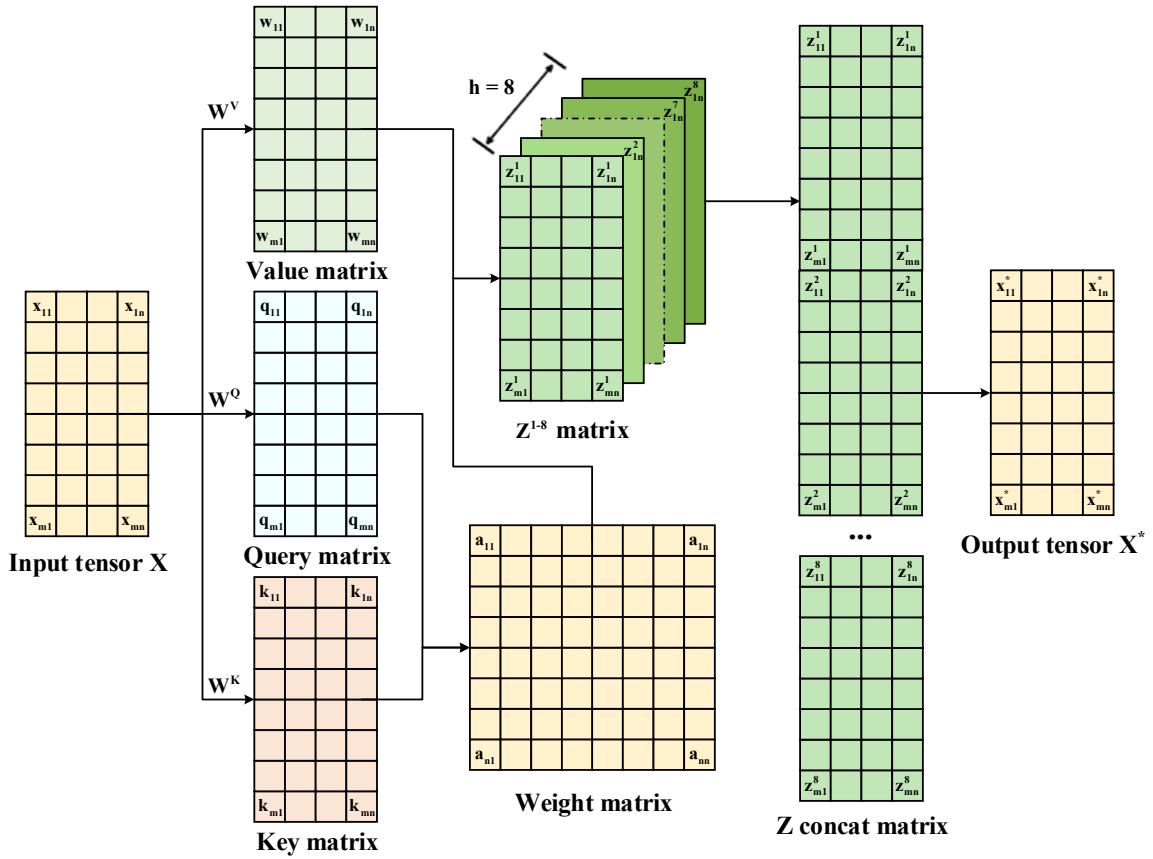


图 21 多头自注意力机制结构

将双向 GRU 的输出特征映射作为其输入张量，分别用 $\mathbf{W}^k$ 、 $\mathbf{W}^Q$ 和 $\mathbf{W}^V$ 计算出 Query、Key、Value 三个特征矩阵。通过 Query 矩阵和 Key 矩阵的点积运算来表征两者间的相关性，该计算过程如公式(32)所示。接着将上述运算结果与 Query 矩阵的列维度进行除法运算，以防止运算结果过大。通过 softmax 运算将上述结果转换为权重矩阵，并将权重矩阵与 Value 矩阵相乘得到 Z 矩阵。

$$Z = \text{softmax} \left(\frac{Q^T K}{\sqrt{d_k}} \right) V \quad (32)$$

将上述所计算出的多个 Z 矩阵执行并联操作，并将其变换为目标输出张量 X^* 。输入张量和输出张量具有相同的数据结构，输出张量的所有维度均将其余维度的相关性纳入考虑范围。该机制能够充分提取数据维度间的相关性，增强数据各维的关联性。双向门控循环单元主要由三个 GRU 层和一个多头自注意力机制构成。考虑到模型参数规模及模型训练时间的合理性，首先将第一个 GRU 层设置为双向 GRU 层，用于提取数据沿正向和反向维度的位置特征。接着将多头自注意力机制设置在双向 GRU 层之后以进行权重设定。该机制能够对双向 GRU 层中所提取输出特征的维度权重进行汇总以增强数据间的关联性。最后将其余 GRU 层设置为正常的单向 GRU 层，用于提取多头自注意力机制输出矩阵的特征。

4.4.2 拥堵状态实时预警模型的建立

由于 5.3.2 节所建立的交通流拥堵模型可以得出对应的交通拥堵指数，以此判断道路的拥堵状况，即与阈值相比，从而判断是畅通或拥堵。但该计算结果仅为交通流判定的初步状态，可能存在一定程度的误判。

为保证分析结论的稳定性和准确性，通常需要对检测维持其持续性，并以此来决策是否该状态是否开启预警，本节根据下式可以进行更为准确地评判。

$$\bar{I}^p(t) = \begin{cases} 0, & \text{if } I^p(t)I^p(t-g) < 2^{g+1} \\ 1, & \text{else} \end{cases} \quad (33)$$

式中， $\bar{I}^p(t)$ 为在观测点 p 的 t 时刻的交通拥挤程度评判值，0 代表不预警状态，1 代表拥堵状态； $I^p(t)$ 为在观测点 p 的 t 时刻单次交通状态程度评判值，1 代表畅通状态，2 代表拥堵状态； g 为设定的检测次数。

因此，本节设置大小为 10，步长为 1 的滑动窗口，即 10 秒提取一次数据，重新构造一个以 10 秒为单位长度的时间序列预测模型，接着我们在长度为 10 的数据样本中对其进行标签分类的划分，比较 1 至 5 和 6 至 10 样本数据的 TPI 值，同时计算 TPI 曲线的梯度，TPI 曲线的梯度大小反映了 TPI 曲线的变化剧烈程度，TPI 曲线的梯度越大，说明 TPI 曲线上升的速率越快，如果后者的 TPI 值的平均值大于前者的 TPI 值的平均值的 1 倍，同时 TPI 曲线的梯度大于 1.3，那么我们对其进行赋予预警的标签，否则赋予非预警的标签，接着对定类变量进行 01 分类标签的处理：0 是不预警，1 是预警。

本节选取观测点 3 和观测点 4 的数据进行实时预警，预警的结果如图 23、表 6 和图 24、表 7 所示，图 23 和图 24 给出了 TPI 曲线、TPI 曲线梯度的变化趋势以及实时预警的预警点，在拥堵状态发生前 5 分钟给出预警信号。

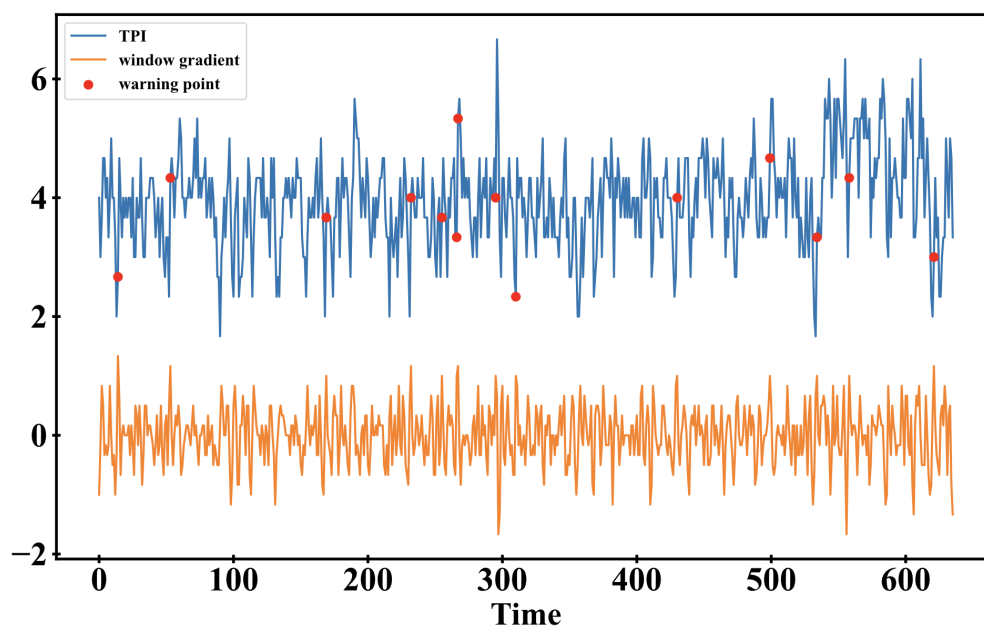


图 22 观测点 3 拥堵状态的预警时间

表 6 观测点 3 拥堵状态的预警时间

序号	图中时间/s	实际时间
1	14	11:38:03
2	53	11:44:33
3	169	12:03:53
4	232	12:14:23
5	255	12:18:13
6	266	12:20:03
7	295	12:24:53
8	310	12:27:23
9	430	12:47:23
10	499	12:58:53
11	534	13:04:43
12	558	13:08:43
13	621	13:19:13

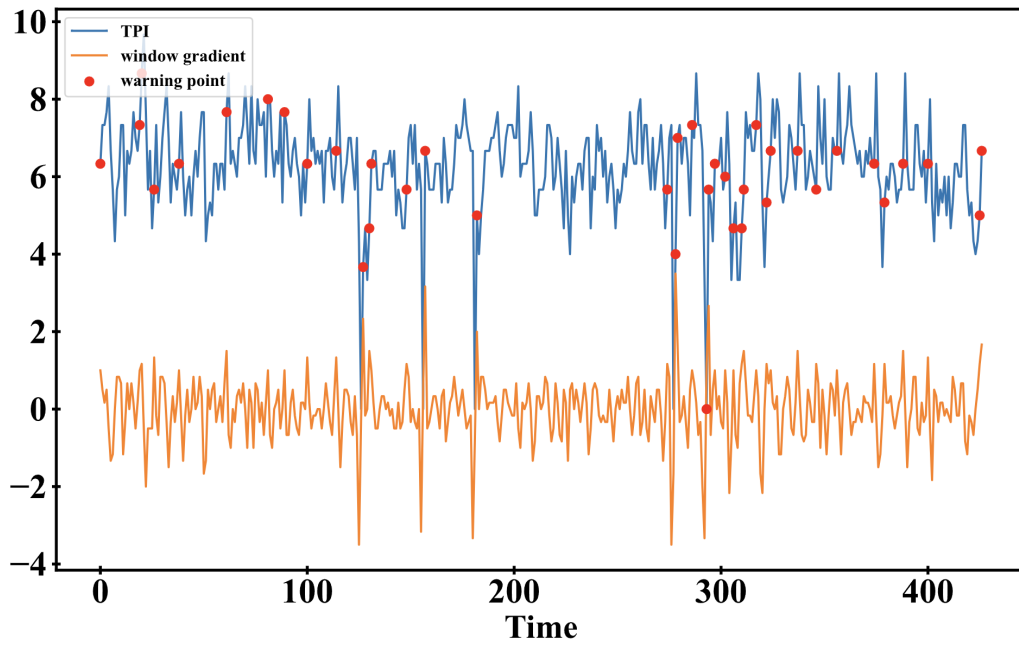


图 23 观测点 4 拥堵状态的预警时间

表 7 观测点 4 拥堵状态的预警时间

序号	图中时间/s	实际时间
1	0	12:56:47
2	19	12:59:57
3	20	13:00:07
4	26	13:01:07
5	38	13:03:07
6	61	13:06:57
7	81	13:10:17
8	89	13:11:37
9	100	13:13:27
10	114	13:15:47
11	127	13:17:57
12	130	13:18:27
13	131	13:18:37
14	148	13:21:27
15	157	13:22:57
16	182	13:27:07
17	274	13:42:27
18	278	13:43:07
19	279	13:43:17
20	286	13:44:27
21	293	13:45:37
22	294	13:45:47
23	297	13:46:17
24	302	13:47:07

25	306	13:47:47
26	310	13:48:27
27	311	13:48:37
28	317	13:49:37
29	322	13:50:27
30	324	13:50:47
31	337	13:52:57
32	346	13:54:27
33	356	13:56:07
34	374	13:59:07
35	379	13:59:57
36	388	14:01:27
37	400	14:03:27
38	425	14:07:37
39	426	14:07:47

4.4.3 交通流参数预测结果

本节基于上节建立的交通流参数预测模型对观测点 3 和和测点 4 的交通流参数进行预测，对每个数据流采用 7:3 的比例进行训练和测试，训练完成后，利用训练好的模型对观测点 3 和观测点 4 的交通流量、车流密度以及平均速度进行预测，并与真实值进行对比，结果如下：

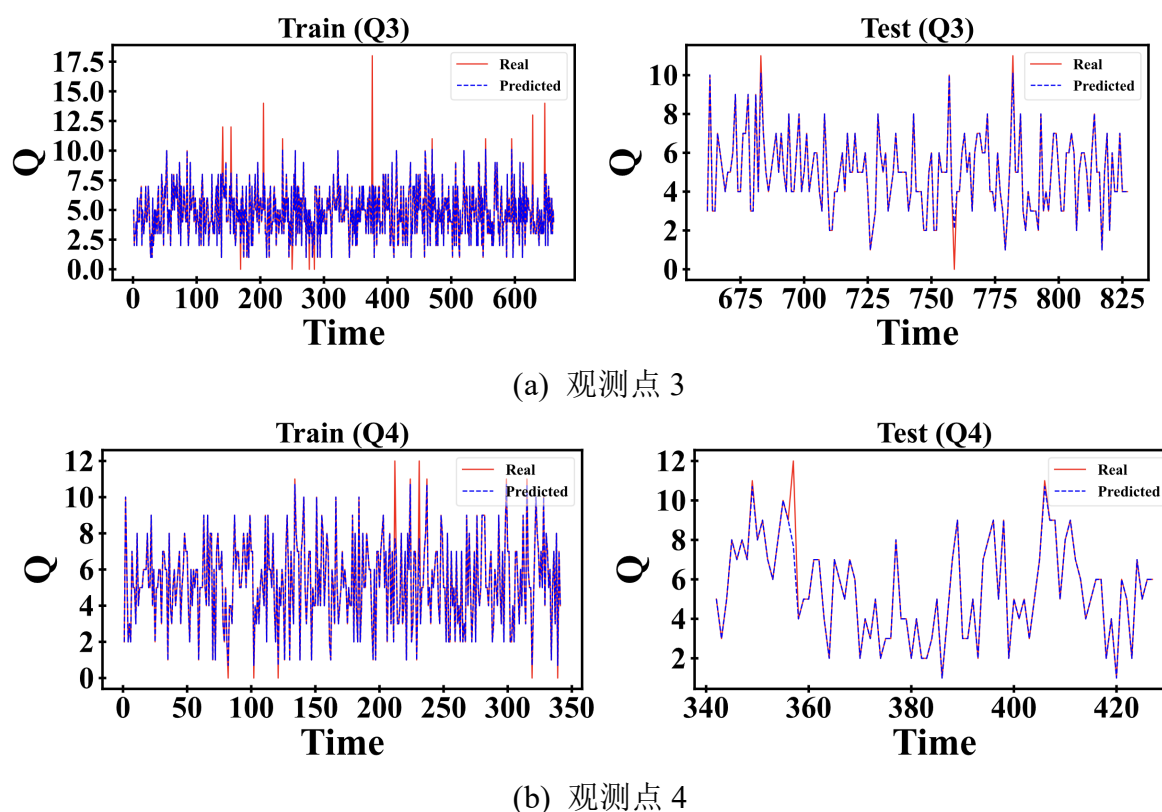
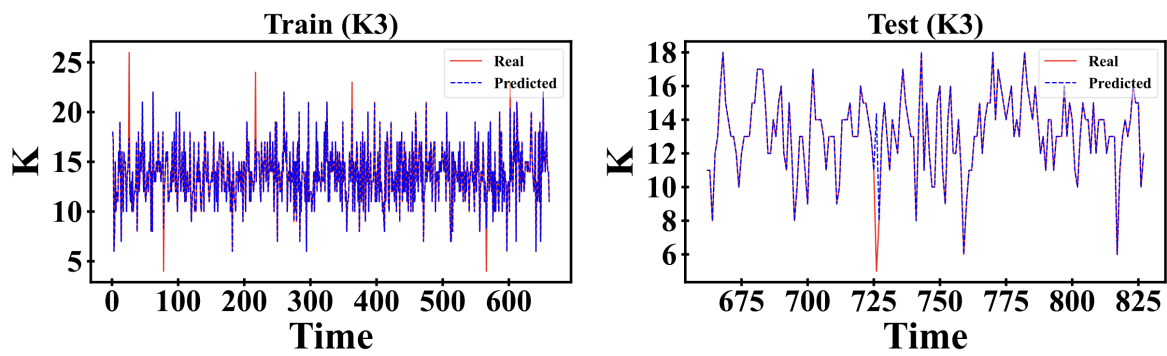
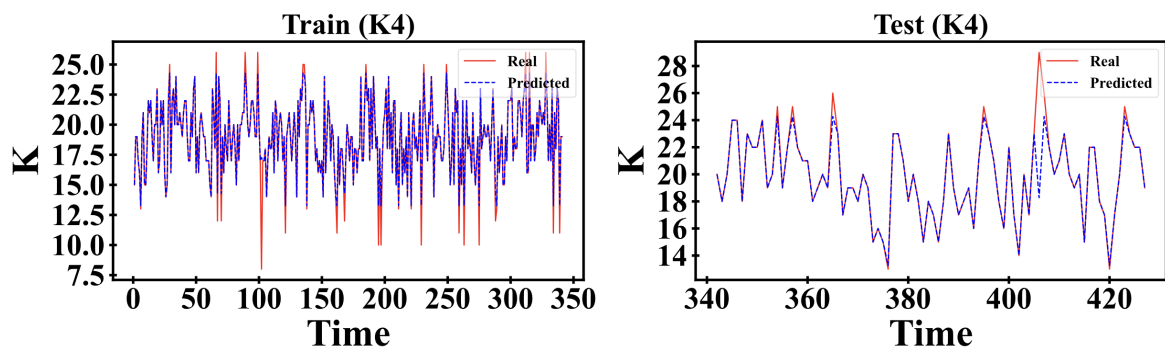


图 24 观测点 3 和 4 的交通流量训练和预测数

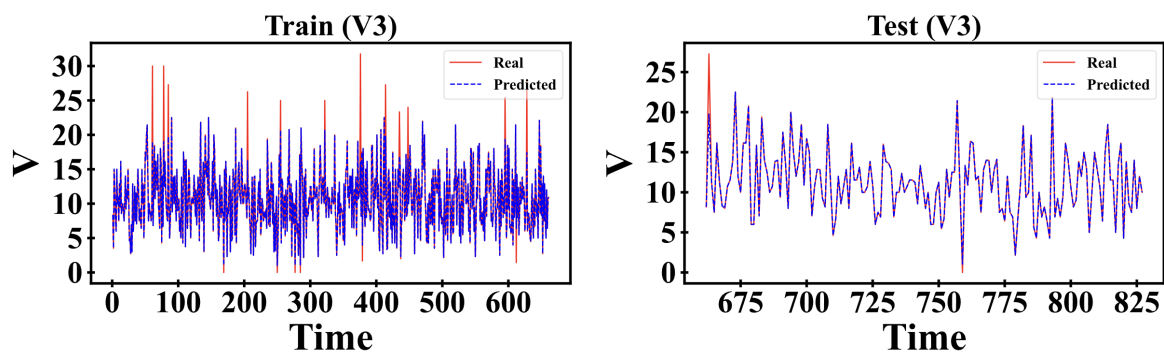


(a) 观测点 3

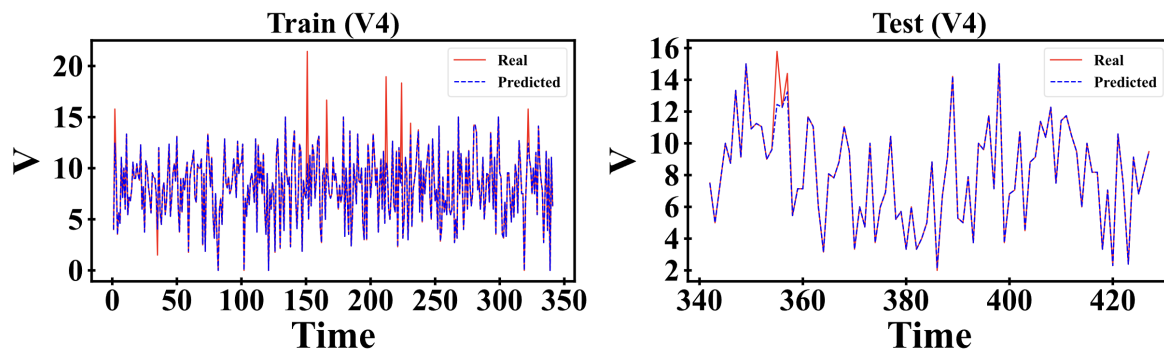


(b) 观测点 4

图 25 观测点 3 和 4 的车流密度训练和预测数



(a) 观测点 3

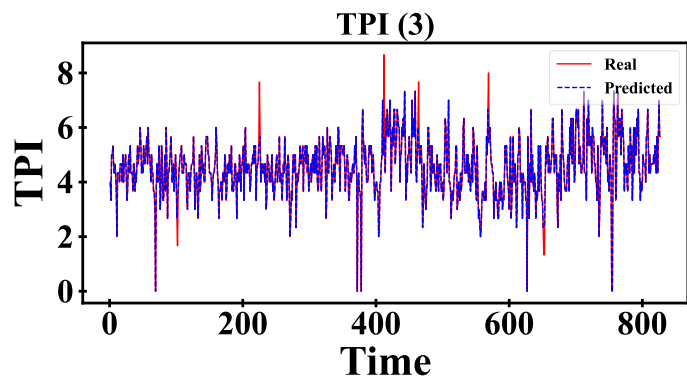


(b) 观测点 4

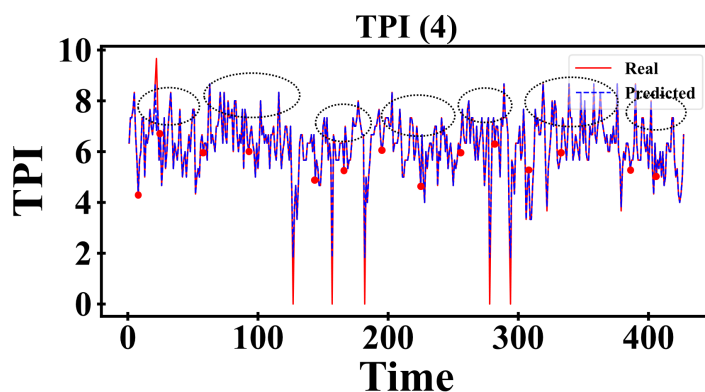
图 26 观测点 3 和 4 的时间平均速度训练和预测数

4.4.4 交通拥堵指标的预测结果

根据训练模型预测出观测点 3 和观测点 4 的三个交通流参数之后，计算相应的交通拥堵指标并进行实时预警，并与真实指标进行对比，结果如图 28 所示，在模型给出实时预警之后的一段时间，TPI 指数达到一段时间内的极值点（图 28 中虚线框），对应车流出现拥堵情况，可以看出模型对于交通流参数与交通指标的预测结果较好。



(a) 观测点 3



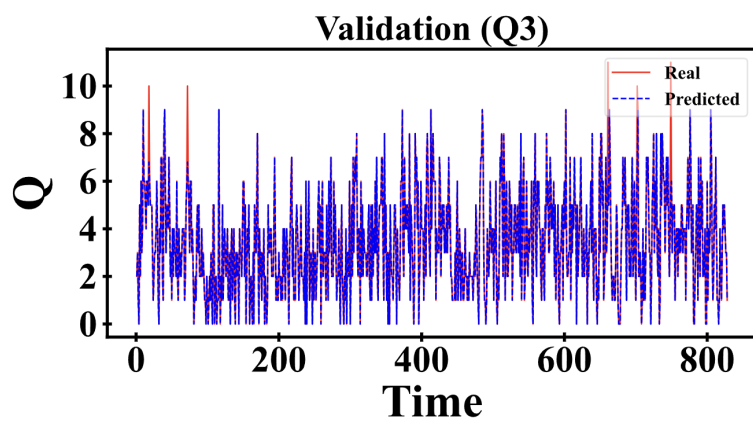
(b) 观测点 4

图 27 观测点 3 和 4 的交通流量验证结果

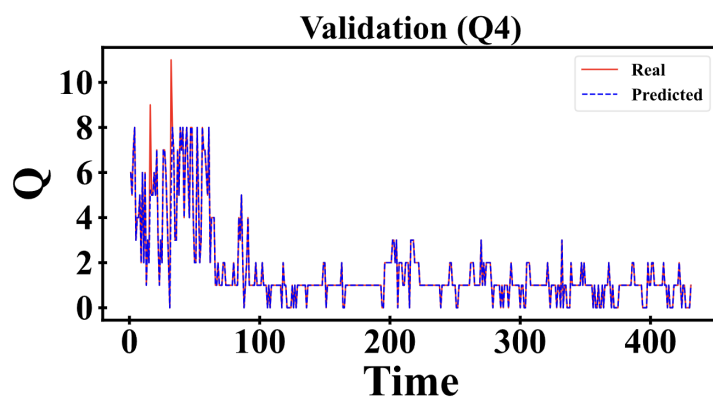
4.5 交通流参数预测模型的验证与分析

4.5.1 交通流参数及其拥堵模型的验证

利用模型对交通流参数进行预测之后，需要对模型的预测结果进行正确的评估，以验证模型的有效性，在上节建立预测模型时，本文选取了观测点 3 和观测点 4 的第一段视频的数据进行训练，在本节中，为了验证所建模型对不同视频段的预测结果的正确性，本节选取了观测点 3 和观测点 4 的第二段视频进行测试，首先利用 YOLOv8 对这两个视频段进行预处理，得到这两个视频段的相应预测参数，然后利用这些参数，模型对交通流三个参数以及 TPI 指数进行预测，并给出实时预警，验证结果如下：

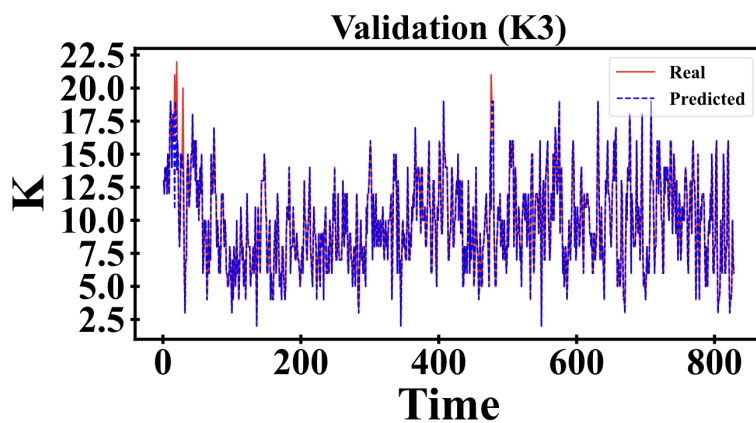


(a) 观测点 3

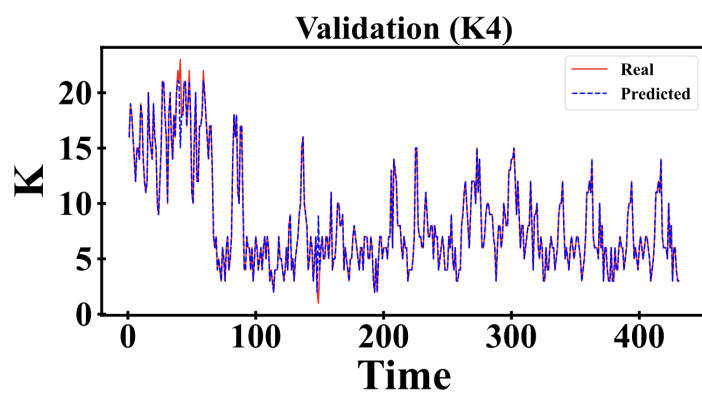


(b) 观测点 4

图 28 观测点 3 和 4 的交通流量验证结果

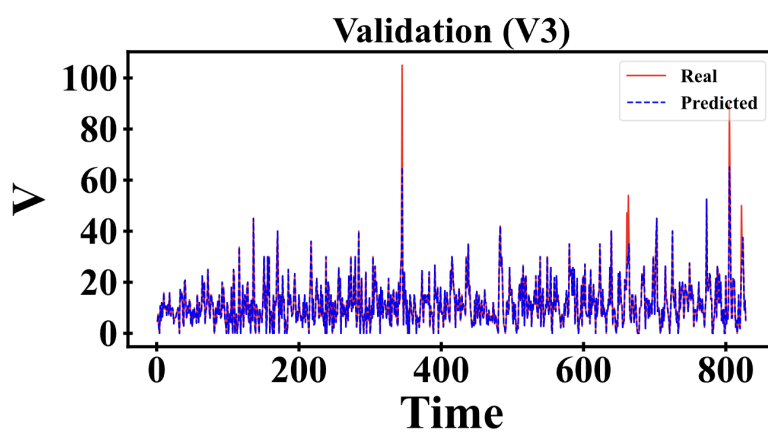


(a) 观测点 3

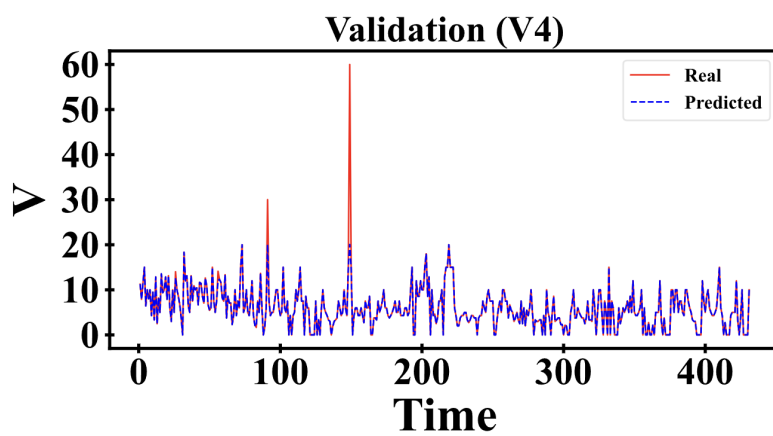


(b) 观测点 4

图 29 观测点 3 和 4 的车流密度验证结果

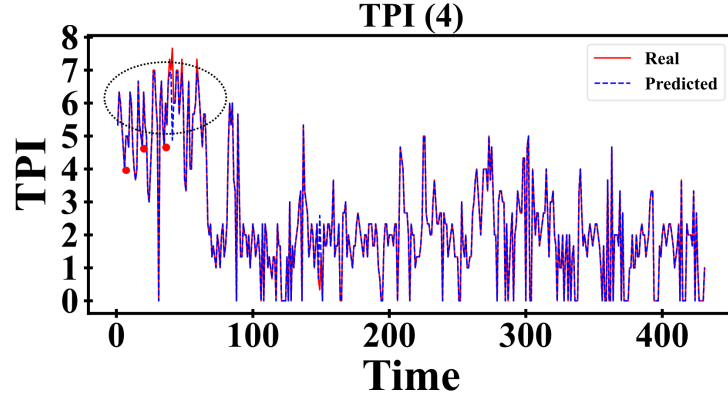


(a) 观测点 3

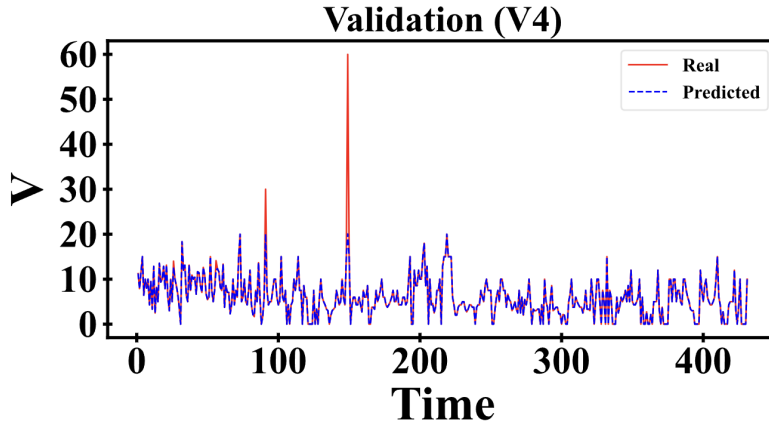


(b) 观测点 4

图 30 观测点 3 和 4 的时间平均速度验证结果



(a) 观测点 3



(b) 观测点 4

图 31 观测点 3 和 4 的交通拥堵指标验证结果

4.5.2 评价指标

在本文中，使用五个性能评估指标，即决定系数 (R^2)、均方根误差 (RMSE)、平均绝对百分比误差 (MAPE)，均方误差 (MSE) 和平均绝对误差 (MAE) 来评估所提出模型的性能。其中,均方根误差是回归模型的典型指标,用于指示模型在预测中会产生多大的误差,代表预测值和真实值之差的样本标准差。平均绝对百分比误差是相对误差度量值,使用绝对值来避免正负误差相互抵消,常用于衡量各种时间序列模型预测的准确性。决定系数反映了模型拟合数据的准确程度,其取值范围通常为[0,1]。其值越接近1,表明当前方程变量对目标量的解释能力越强,即该模型的数据拟合性越好。 R^2 、RMSE 和 MAPE 的计算过程如公式(34)-(36)所示,其中 n 代表样本总量, y_i 代表第 i 个样本的真实值, $\hat{y}_i$ 代表第 i 个样本的预测值, $\bar{y}$ 代表真实值的平均值。

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (34)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (35)$$

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right| \quad (36)$$

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \quad (37)$$

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m \left| (y_i - \hat{y}_i) \right| \quad (38)$$

表 8 四个观测点预测数值的评价指标结果

	观测点 3	观测点 4
R ²	0.9638	0.8048
RMSE	0.1304	0.1652
MAPE	0.0084	0.0161
MSE	0.0170	0.2164
MAE	0.0157	0.1190

五、问题二模型的建立与求解

5.1 问题分析

问题二主要目的是：合理高速公路启用应急车道模型。主要要点是：

(1) 基于问题一中获取的交通流量，车流密度和时间平均速度的结果，得出的交通拥堵指数进行分析；

(2) 构建并求解交通流中断概率模型，从而确定高速公路启用应急车道速度与交通流量的阈值，达到合理启用高速公路应急车道模型。

基于上述分析，得到问题二的求解思路如图 32 所示。

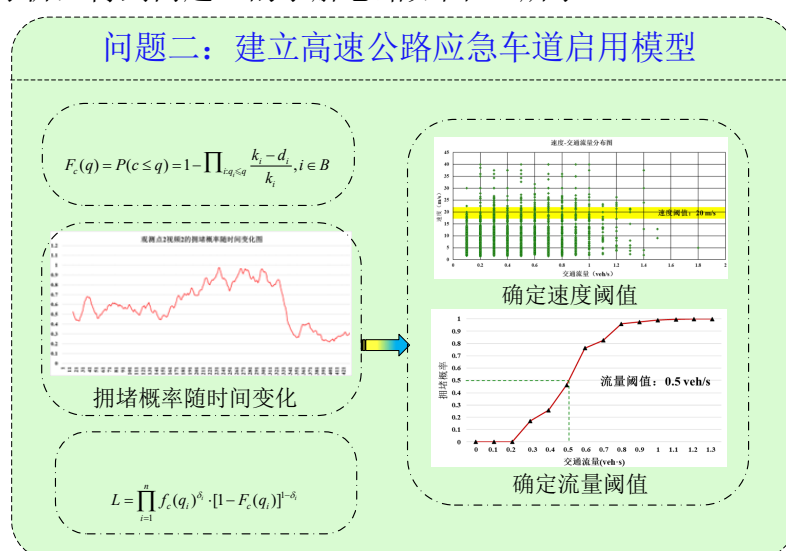


图 32 问题二思路流程图

5.2 高速公路应急车道启用模型的建立

为了使用高速公路通行能力随机性的概念，有必要了解更多关于通行能力分布函数的知识。然而，它是一项具有挑战性的任务。显然，任何分析方法都必须得到广泛的实证调查的支持。

高速公路上的交通流量观测在预定的观测间隔（指数 i ）期间提供交通流量和时间平均速度的数值。根据通行能力的定义，如果平均速度超过一定的阈值（例如，德国高速公路的情况约为 70 公里/小时），观察到的容量将低于通行能力。当平均速度低于阈值时，交通流会变得拥挤。因此，在两次这样的观测之间的时间内，流量一定已经超过了容量。然而，容量本身无法直接测量。此外，在实地看到需求量增加的可能性较小，因为在需求量增加之前，在前几个间隔期间以较低的需求量发生细分的可能性也较大。这使得很难估计容量分布函数，其定义如下：

$$F_c(q) = P(c \leq q) \quad (39)$$

式中， $F_c(q)$ 是指容量分布函数， c 是指容量， q 是指交通流量。

为了评估交通拥堵的概率，需要利用信息采集设备收集速度、流量和密度等数据，这些设备应布置在现有和潜在的所有瓶颈点的上游位置。本节基于当前时间间隔内路段的车速和交通量状况，确定下一时间间隔内交通流发生中断（即交通拥堵）的概率。Brilon 等人^{[5][6]}基于详细的交通流量数据，构建了交通流中断概率模型，模型如下所示：

$$F_c(q) = P(c \leq q) = 1 - \prod_{i: q_i \leq q} \frac{k_i - d_i}{k_i}, i \in B \quad (40)$$

式中， q 为每小时每车道的车辆通过量，veh/(h·ln)； c 是容量，veh/(h·ln)，即通行能力； $P(c \leq q)$ 是容量小于观察到流量的概率； q_i 是在间隔 i 处观察到的交通流量，它表示速度下降之前的交通流量； k_i 是指 $q \geq q_i$ 的时间间隔数； d_i 是指交通流量达到 q_i 时的时间间隔数，当每一个拥堵被单独考虑时， $d_i = 1$ ； B 是拥堵间隔 $\{B_1, B_2, \dots\}$ 的集合。

根据公式（40），交通流量 q ：

B： 流量在时间间隔 i 中是流畅的，但是观察到的流量引起故障，即在下一个时间间隔 $i + 1$ 中平均速度降低到阈值速度以下

F： 在区间 i 和随后的区间 $i + 1$ 中流量是流畅的。这个间隔 i 包含一个删除值。它的信息是区间 i 的实际容量大于观察到的容量 q_i 。

C1： 在间隔 i 中流量拥塞，即平均速度低于阈值。由于这个时间间隔 i 没有提供有关容量的信息，因此它被忽略。

C2： 在间隔 i 中流量是流畅的，但是观察到的流量会导致故障。然而，与分类 B 相反，在间隔 i 或 $i - 1$ 期间，交通在下游横截面上是拥挤的。在这种情况下，在观察点的故障应该是由于下游的尾流。由于这个时间间隔 i 不包含任何信息的能力评估在观察点，它是忽略的。

5.3 高速公路应急车道启用模型的求解

根据公式(40)可得最后的概率随时间变化的图，由于篇幅限制，这里观测点 2 的 3 个视频中，本节只选取了其中 2 个视频。图 33(a)是观测点 1 的视频 1，具体地，从 11 点 41 分行驶了 3640 秒后，即时间为 12 点 41 分 44 秒，开启了应急车道。而问题一的 5.2.1 节的交通流量分析中，可以看到在 2880 秒的时候，交通流量已经达到了最大值。所以，即便开启了应急车道，也需要一定的时间来缓解交通流量大于车道容量的状态，图 33(a)中 364 秒后有一段时间交通拥堵概率达到最大值，接着拥堵概率逐步降低，直至到图 33(b)中交通拥堵概率基本稳定在 0.1-0.4 范围内时，关闭应急车道，即图 33(b)中的 230 秒左右。同样的，在观测点 1 的视频 2 的 230 秒处也关闭了应急车道。

观测点 3 的视频 1 整体拥堵概率较高，整体维持在 0.6 到 0.9 之间，说明视频 1 的车流量较大，会导致整体车流速度降低。在观测点 3 视频中刚开始的一段时间，车流量较大，因此拥堵概率较高，但随着时间的流逝，车流量下降导致拥堵概率逐渐降低，最后维持在 0.45 左右。

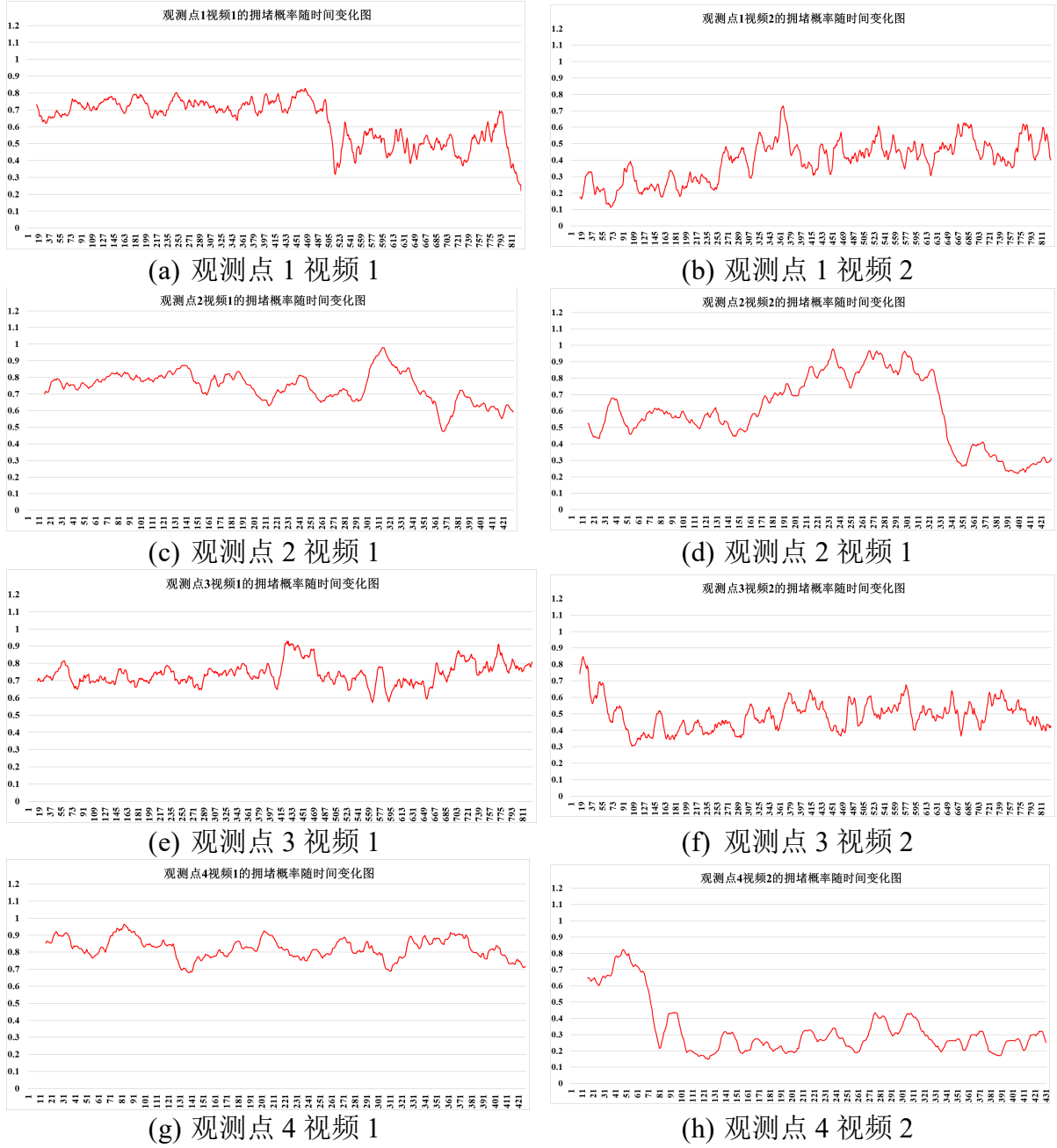


图 33 求解的交通流中断概率密度

5.4 最大似然法估计分布参数

为了克服交通流量和交通速度的阈值确定困难的问题，采用了最大似然估计的方法。最大似然估计法首先要得到似然函数，似然函数由以下公式给出：

$$L = \prod_{i=1}^n f_c(q_i)^{\delta_i} \cdot [1 - F_c(q_i)]^{1-\delta_i} \quad (41)$$

式中， $f_c(q_i)$ 代表车道容量的统计密度函数； $F_c(q_i)$ 代表车道容量的累积分布函数； n 表示时间间隔数； $\delta_i = 1$ ，时间间隔 i 内包括未审查对象； $\delta_i = 0$ ，时间间隔 i 内不包括未审查对象。

必须最大化似然函数或其自然对数以确定分布函数的参数。根据似然函数的值比较不同类型的函数，Weibull 分布被证明是最符合所有高速公路路段的观测值的函数。Weibull 分布函数为：

$$F(x) = 1 - e^{-\left(\frac{x}{\beta}\right)^\alpha}, x \geq 0 \quad (42)$$

式中， α 代表形状参数， β 代表尺度参数。

利用最大似然估计法可以初步确定交通流量和交通速度的阈值，当交通流量大于该阈值时，会增加车辆发生拥堵的概率，当交通速度低于阈值时，说明车流的整体速度较低，则有可能发生了拥堵，因此，必须合理有效地确定出路段的车流量阈值和车速阈值，以供决策者做出更合理的决策，预防交通拥堵的发生。最大似然估计的方法也可以用于流量密度的估计，但是，基于密度的分析还需要对车道畅通流量和拥堵流量之间的速度阈值进行更为复杂的定义，这对于路段堵塞的预测是不利的，因此，下面开始确定速度和流量阈值。

5.5 速度和流量阈值判定

本节将四个观测点所有数据进行汇总，由于每一帧的速度是 0.4 秒，而 25 帧为 1 秒，所以选取每 25 帧进行统计分析，时间平均速度-交通流量变化如图 34 所示。从图中可以看出，当车辆的时间平均速度超过 20m/s 时，表示为不拥堵状态；车辆的时间平均速度低于 15m/s 时，表示为拥堵状态。

当车流量在 0-0.2 辆/秒时，车流时间平均速度集中在速度上游，平均速度基本上都集中大于 20m/s；当车流量在 0.2-1 辆/秒时，当车流量在 0.2-1 辆/秒时，随着车流量的增加，车流时间平均速度逐渐集中在下游，基本上都几种小于 20m/s。

图中的红色虚线的曲线可以看出，随着交通车辆流量的增加，时间平均速度也随之降低，车道越拥堵。其中红色圆圈圈出的异常点，是因为随着交通车流量的增加，有人借用了应急车道，此时，双车道变成了三车道，故此时车流量大，但车速并不低。

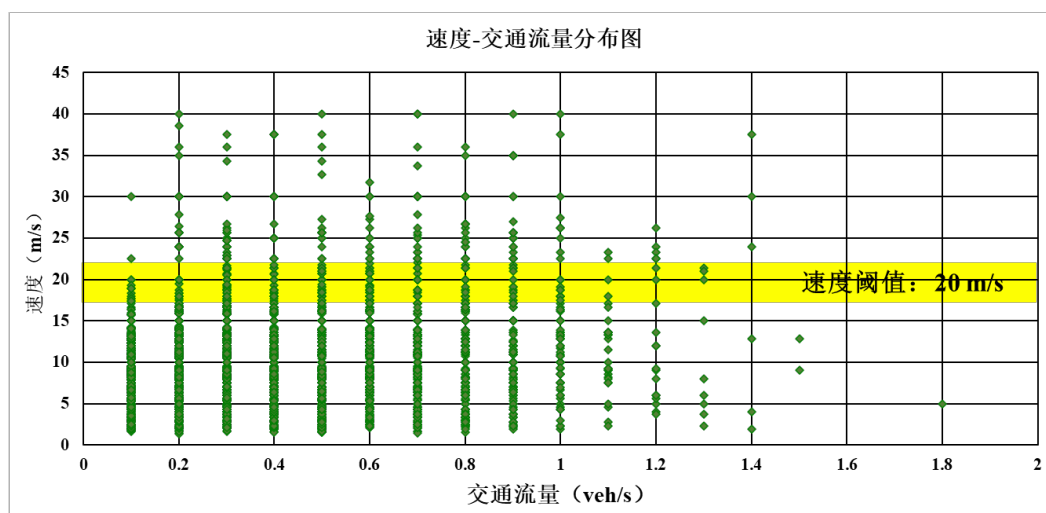


图 34 四个观测点时间平均速度-交通流量变化图

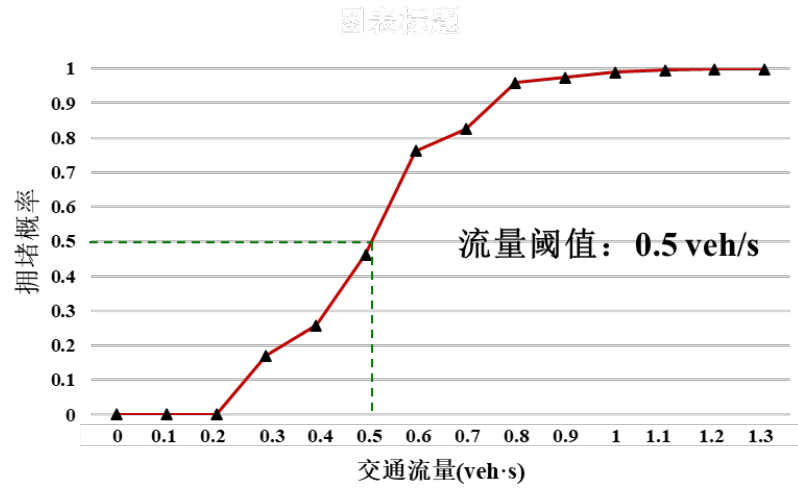


图 35 四个观测点交通流量-拥堵概率变化图

六、问题三模型的建立与求解

6.1 问题分析

问题三具体包含以下两个要点：

(1) 决策机制：在确保车辆在行驶过程中的道路安全的前提条件下，根据建立的拥堵模型，制定应急车道临时启用规则。当模型预警第三点到第四点之间路段可能出现持续拥堵时，决策系统会根据预警时间、拥堵程度等因素，决定是否启用应急车道。

(2) 效果评估：选择如总行程时间、总碰撞暴露时间、平均通行时间、总延误时间和车辆平均速度等指标，量化评估应急车道启用模型的效果，包括减少拥堵时间、提升通行效率、降低交通事故风险等方面的效益。

基于上述分析，得到问题三的求解思路如图 36 所示。

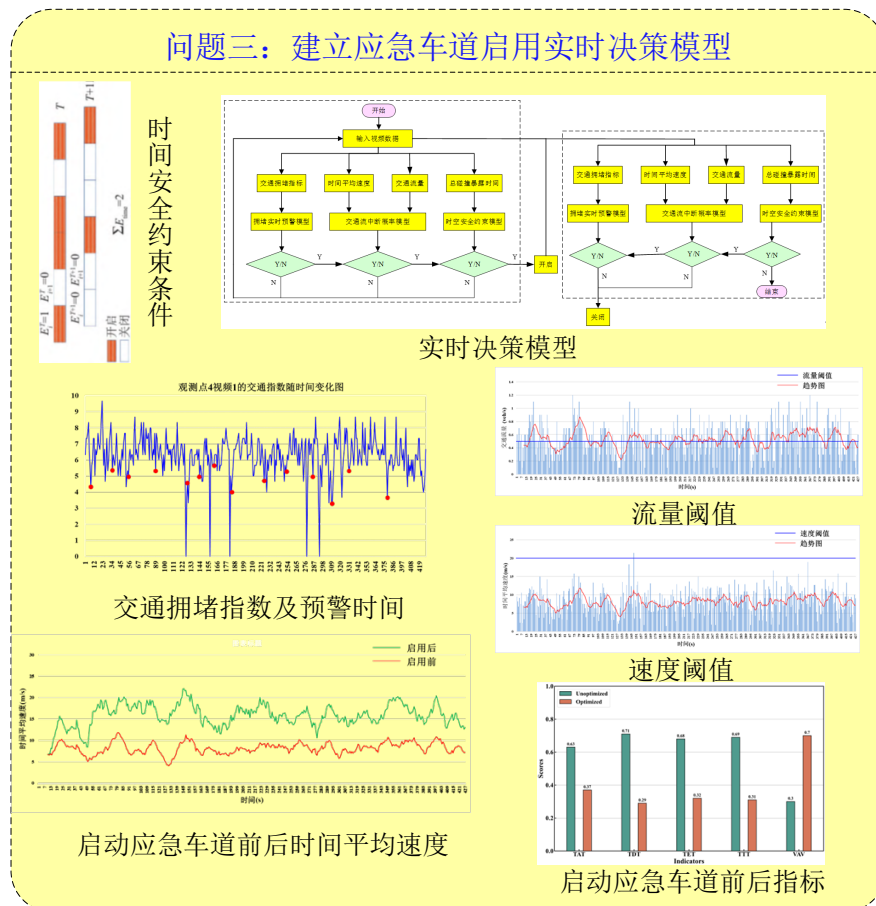


图 36 问题三思路流程图

6.2 应急车道启用实时决策模型的建立

在实际车道场景中，由于应急车道开启与关闭的频繁变化会对管理者的管理成本和驾驶员的行车安全提出一定的要求，为实时决策模型的决策效果，本文加入了时空安全约束条件。通过设定两次时间间隔为 2 分钟，以此建立时间约束，从而分析道路交通状态的变化。每 1 分钟进行一次应急车道的开关决策，并将时间窗视为观察仿真结果的依

据。相邻两次决策中，应急车道的空间片段开关状态变化的总次数被称为时间窗的跳变次数。

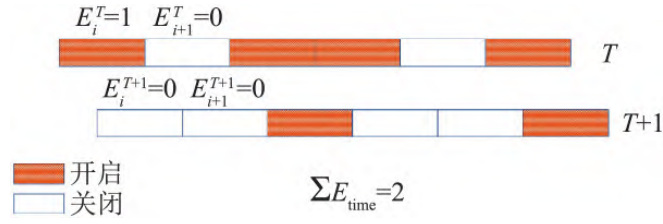


图 37 时间安全约束图

如图 37 所示， T 代表着应急车道在开启与关闭状态下的 1 分钟时间间隔，这是设定的最小决策时间。 $E_i(t)$ 为第 t 个时间段第 i 个空间片段的开关状态； $E_i(t+1)$ 为第 $t+1$ 个时间段第 i 个空间片段的开关状态。一次时间窗跳变次数规定为控制变量 E_i 由 0 变为 1，由 1 变为 0。时间窗跳变次数反映着应急车道开启与关闭的变化频率，为了减少变化频率，应该将时间窗跳变次数控制在合理的范围内，如下式所示：

$$\sum_{i=1}^L (E_i(t+1) - E_i(t))^2 \leq N \quad (43)$$

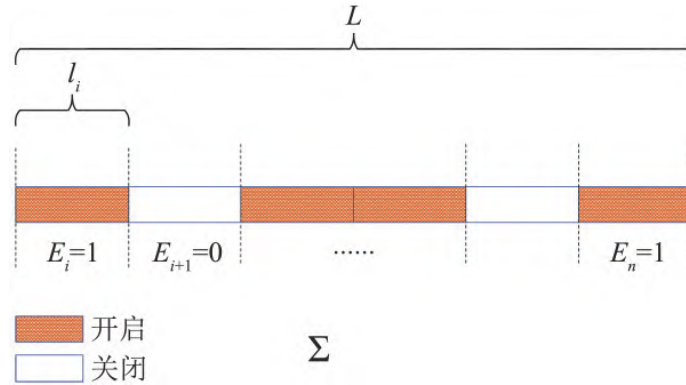


图 38 空间安全约束图

为减少应急车道空间片段的开关状态过于分散，从而降低驾驶员在主道与应急车道之间换道的次数和安全风险，本文增加了空间安全约束条件。如图 38 所示，其中 L 表示车道总长， l_i 为空间片段， E_i 为空间片段的开关控制变量， E^{cc} 则表示车道分量。引入图论中的连通分量概念，定义车道分量数来描述应急车道开关状态的离散程度。在 1 分钟的时间间隔内，空间上相连且开放的应急车道片段视为一个车道分量。用 0 和 1 来表示 E^{cc} 当前空间片段是否能组成一个车道分量。为了避免增加应急车道与主干道之间的换道次数以及管理者过大的成本，应该将车道分量控制在一个合理的范围内：

$$\sum E^{cc} \leq N \quad (44)$$

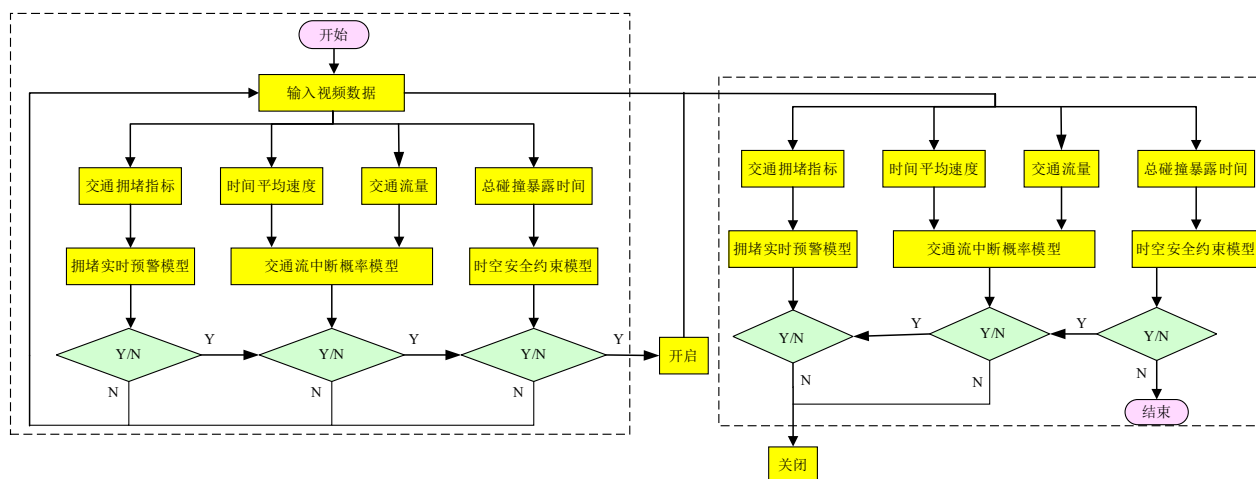


图 39 开启应急车道实时决策策略

实时决策模型的决策过程如图 39 所示，应急车道共有开启和关闭两个状态，利用实时决策模型进行决策时，首先可以利用监控摄像头获取监控视频，然后载入到决策模型中，模型载入视频数据后，对视频数据进行预处理，从而获取到时间平均速度、交通流量、总碰撞暴露时间以及交通拥堵指标等交通流参数。获取到交通流参数之后，在每个决策时间内，拥堵实时预警模型首先利用交通拥堵指标进行预测，并判断未来一段时间之内是否会发生持续拥堵，如果发生则发出预警信号，否则进入下一个决策时间。发出预警信号后，交通流中断概率模型判决时间平均速度和交通流量是否超出相应的阈值，如果时间平均速度低于阈值并且交通流量高于阈值，则判决未来一段时间内会发生拥堵，否则不发出判决信息。判决发生拥堵后，时空安全约束模型根据总碰撞暴露时间来决策当前是否处于安全开启状态，如果处于安全开启状态，则控制相应的交通信号灯，开放应急车道，以预防交通拥堵的发生。

开放应急车道后，处于管理者成本和驾驶员安全的角度考虑，在不会发生交通拥堵和确保安全的前提下，需要关闭应急车道，应急车道的关闭判决与开启的判决条件基本一致，在未来两段决策时间内，如果没有预警信号的发出，并且时间平均速度高于阈值、交通流量低于阈值，同时满足安全性的要求，实时决策模型会发出关闭应急车道信号，从而控制交通信号灯关闭应急车道。

6.3 应急车道启用实时决策模型的求解

基于所构建的应急车道启用实时决策模型对视频数据中的应急车道是否启用进行实时决策。通过对各观测点视频进行分析发现，观测点 2 中应急车道指示灯在拥堵发生后改变为允许通行状态，因此利用该观测点应急车道开启段训练应急车道开启状态下的交通流参数预测模型，以便于应急车道决策后的交通流参数预测及模型作用量化。观测点 1 和观测点 3 中应急车道有较长时间处于占用状态且应急车道指示灯状态不明确。观测点 4 的应急车道指示灯全程处于禁止通行状态，且仅有少量车辆非法占用应急车道，因此选取视频 4 实时决策是否启用应急车道。

图 40 是观测点 4 视频 1 基于所建立的拥堵实时预警模型进行拥堵预警的结果。从图中可以看出，该观测点的交通拥堵指数持续处于较高水平，表明该观察点长期处于拥堵状态。因此，拥堵实时预警模型的预警范围涵盖了整个观测过程。图 41 和图 42 分别为观测点 4 视频 1 在交通中断概率模型下流量及速度的评估结果。当前时刻处于拥堵预警状态，速度或流量处于交通中断概率模型所设定的阈值范围内，且满足时刻安全约束条件即可将应急车道变为允许通行状态。通过该观测点视频进行应急车道启用实时决策，决定从第四个时刻开启应急车道，并用所构建的应急车道开启状态下的交通流参数预测模型对开启后的交通情况进行描述，为后续应急车道实时决策提供依据。

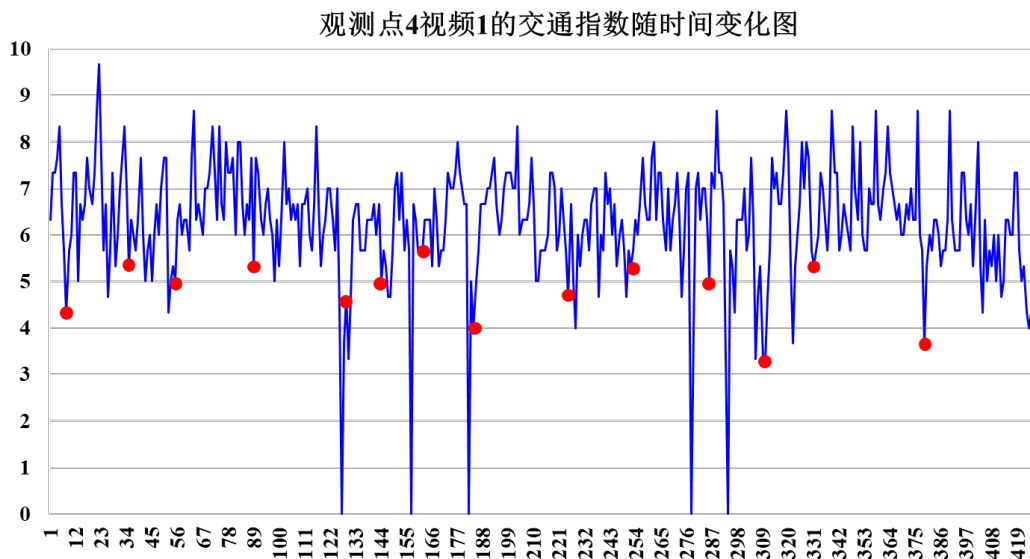


图 40 观测点 4 视频 1 的交通拥堵指数随时间变化图

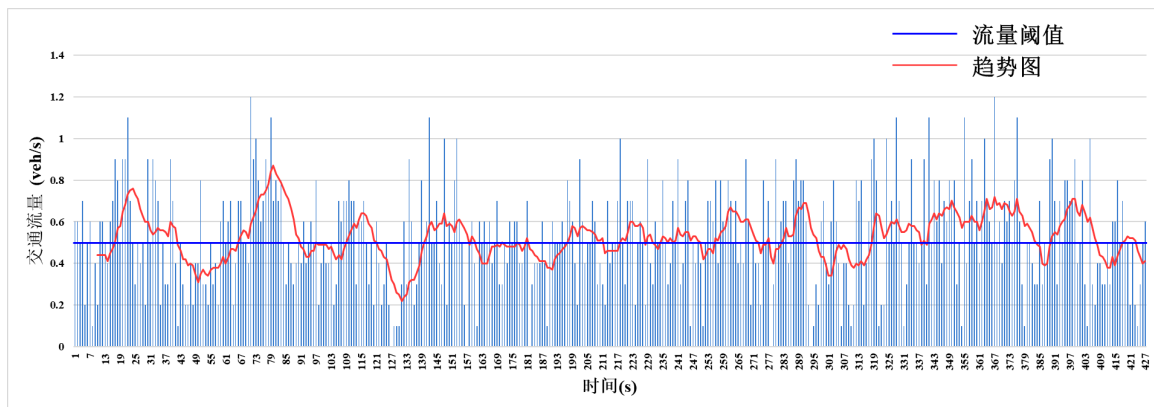


图 41 观测点 4 视频 1 的交通流量随时间变化图

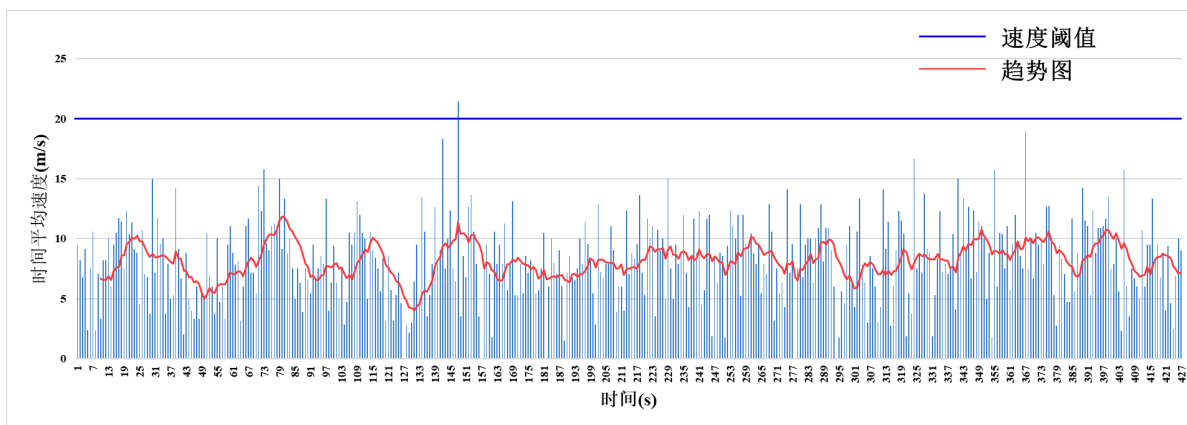


图 42 观测点 4 视频 1 的时间平均速度随时间变化图

通过对观测点 4 视频 2 的交通拥堵指数进行分析发现，该观测点从开始到第六十个时刻处于拥堵状态，此后道路较为畅通。因此，拥堵实时预警模型仅在拥堵时段发生了预警。接着，通过交通中断概率模型对各时刻速度和流量进行阈值判定，并结合预警状态进行应急车道启用实时决策。通过该观测点视频进行应急车道启用实时决策，决定从 2 个时刻开启应急车道，并用所构建的应急车道开启状态下的交通流参数预测模型为后续应急车道实时决策提供依据。

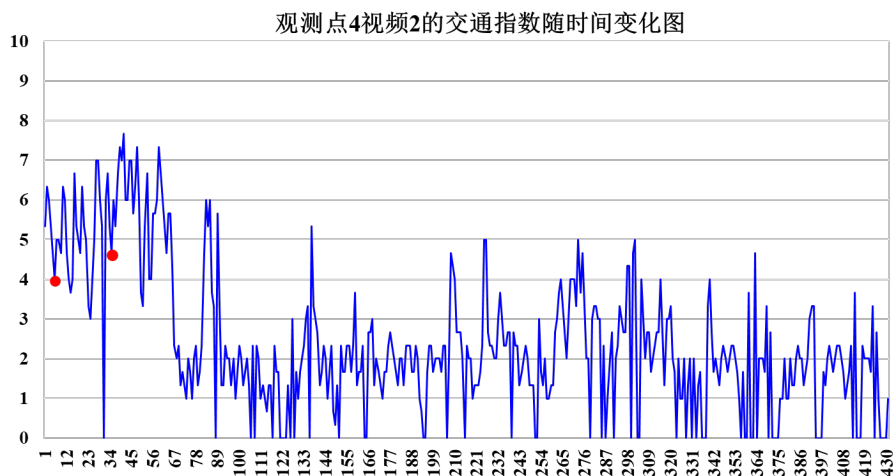


图 43 观测点 4 视频 2 的交通拥堵指数随时间变化图

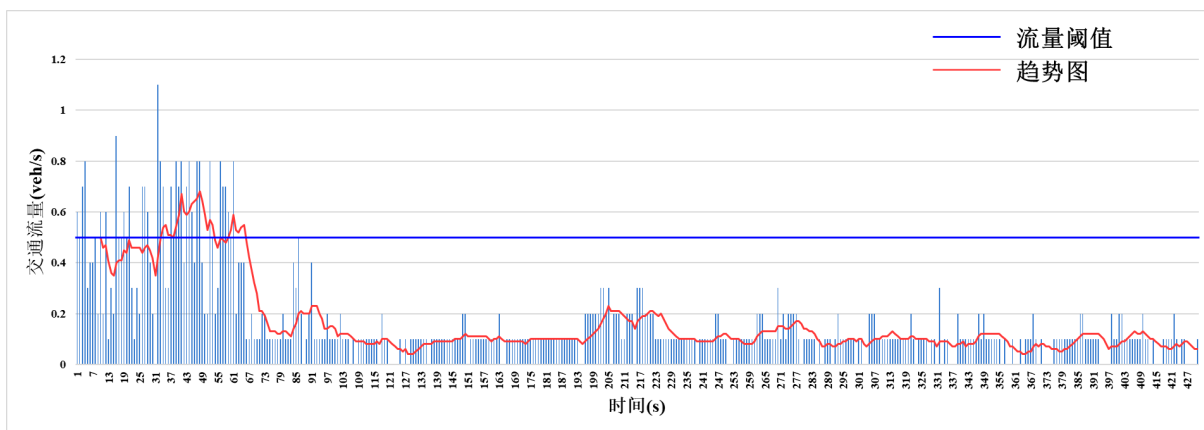


图 44 观测点 4 视频 2 的交通流量随时间变化图

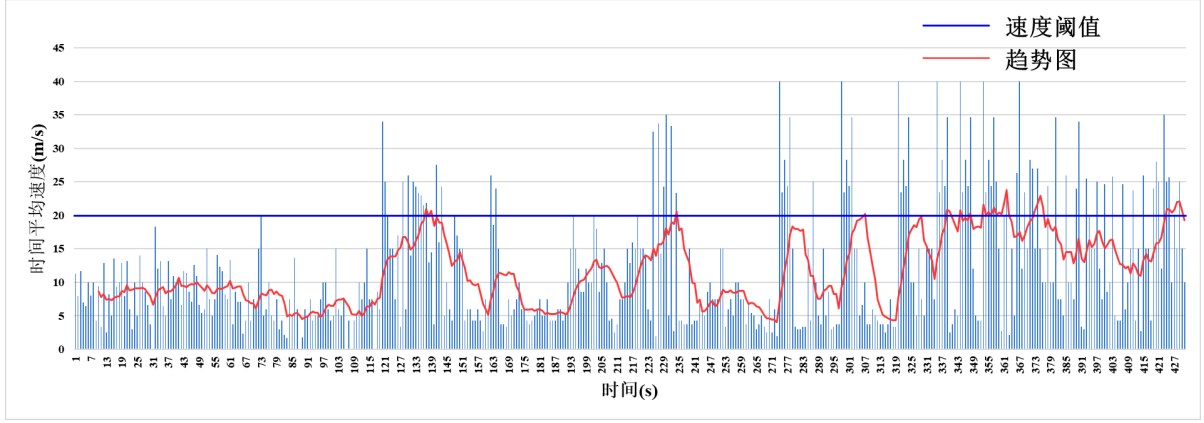


图 45 观测点 4 视频 2 的时间平均速度随时间变化图

6.4 应急车道实时决策模型的量化分析

6.4.1 量化指标体系构建

为了避免主观判断带来的偏差，需要对实时决策模型的决策效果进行量化，因此，本文构建了一系列量化指标，下面对这些指标进行介绍。

(1) 总行程时间：总行程时间是指某路段中所有车辆的行程时间总消耗量。总行程时间的长短反映着道路利用率的高低，总行程时间长，说明道路的利用率低，单位时间内的输出流量最小，其计算方式如下：

$$T_{TT,i}(t,k) = \delta_i \cdot \Delta k, \delta_i = \begin{cases} 1, & \text{车辆在该路段中} \\ 0, & \text{否则} \end{cases} \quad (45)$$

$$T_{TT} = \sum_{k=1}^K \sum_{t=1}^T \sum_{i=1}^N T_{TT,i}(t,k)$$

式中， $T_{TT,i}(t,k)$ 表示第 t 个周期内第 k 个时间段第 i 辆车的行程时间； Δk 为最小时间间隔； N 为该路段上的车辆数。

(2) 总碰撞暴露时间：总碰撞暴露时间是指该路段上所有的车辆在行驶的过程中处于危险状态的总时间。总碰撞暴露时间反映着公路上所有车辆的安全情况，总碰撞暴露时间越长，交通状况越危险。其计算方式如下：

$$T_{ET}(t,k) = \sum_{i=1}^N \delta_k \cdot \Delta k, \delta_k = \begin{cases} 1, & \forall 0 < T_{TC,i}(k) \leq 1.5 \\ 0, & \text{否则} \end{cases} \quad (46)$$

$$T_{ET} = \sum_{k=1}^K \sum_{t=1}^T \sum_{i=1}^N T_{ET}(t,k)$$

式中， $T_{ET}(t,k)$ 为第 t 个周期第 k 个时间段车辆的总碰撞暴露时间，参考相关文献，1.5 为碰撞时间阈值。

(3) 平均通行时间：平均通行时间是指该路段上所有车辆通过该路段的平均时间。通过分析平均通行时间，可以评估道路的使用效率，识别高峰时段和瓶颈路段，从而调整应急车道开放策略。其计算方式如下：

$$T_{AT} = \frac{T_{ET}}{N_{TC}} \quad (47)$$

式中， N_{TC} 代表着车辆总数。

(4) 总延误时间：总延误时间通常指的是在特定时间段内，所有车辆因交通信号、交通拥堵或其他因素导致的延误时间的总和。总延误时间可以反映该时间段内道路的交通状况的好坏。其计算方式如下：

$$T_{DT} = \sum_{i=1}^{N_{TC}} (T_{re} - T_{id}) \quad (48)$$

式中， T_{re} 代表实际通过该路段的时间， T_{id} 表示通过该路段的理想时间。

(5) 车辆平均速度：车辆平均速度是指在特定时间段内，处于该路段上的所有车辆的速度的平均值。它反映了车辆在该段时间内的行驶效率。其计算方式如下：

$$V_{AV} = \frac{\sum_{i=1}^{N_{TC}} V_i}{N_{TC}} \quad (49)$$

式中， V_i 代表该路段上第 i 辆车的车速， N_{TC} 代表该路段上的车辆总数。

6.4.2 实时决策模型结果分析

通过所构建的应急车道启用实时决策模型对观测点的两个视频实施决策，并基于时间平均速度及量化指标体系对应急车道实时决策后的作用进行量化分析。图 46 和图 47 分别展示了应急车道实时决策启用前后的时间平均速度。将决策后速度曲线与决策前速度曲线对比可以看出，开启应急车道实时决策使得时间平均速度明显提升，有效缓解了拥堵。此外，通过所构建的应急车道开启状态下的交通流预测模型对决策开启后的交通流参数进行预测，发现应急车道实时决策的开启使得模型拥堵预警可能性有效降低，使得原先的拥堵预警解除，提升了交通通行效率。

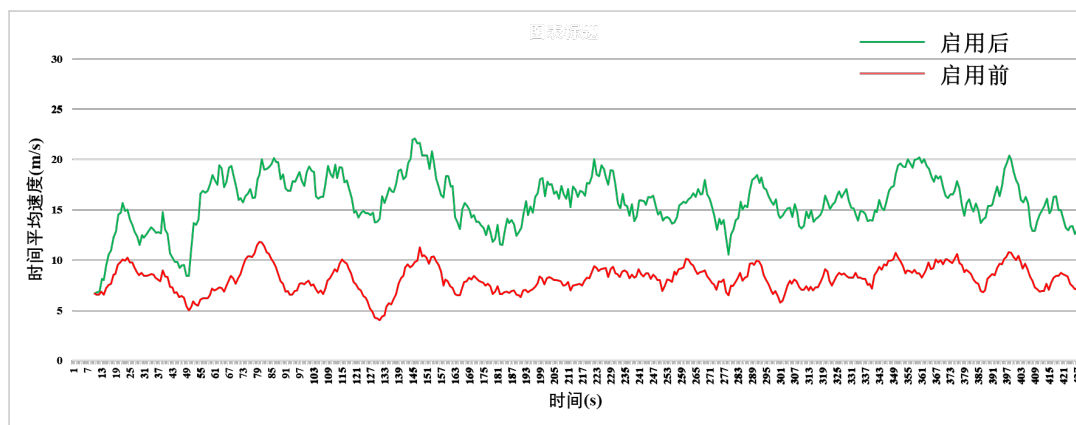


图 46 观测点 4 视频 1 启用前后时间平均速度图

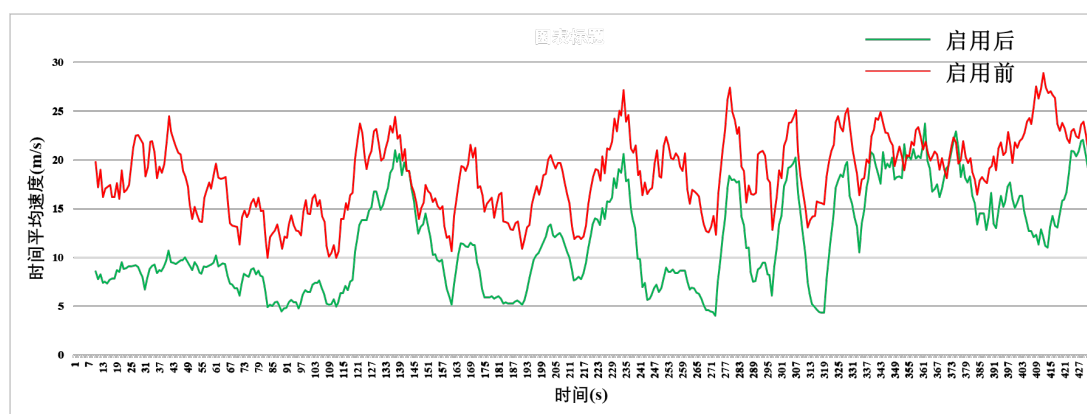


图 47 观测点 4 视频 2 启用前后时间平均速度图

表 9 观测点 4 视频 1 和视频 2 决策开启应急车道前后的指标变化

序号	指标	视频 1		视频 2	
		决策前	决策后	决策前	决策后
1	平均通行时间 (s) T_{AT}	13.1	7.6	6.2	3.4
2	总延误时间 (s) T_{DT}	289	117	122	76
3	总碰撞暴露时 间(s) T_{ET}	144	68	95	50
4	总行程时间(s) T_{TT}	7679.1	3433.9	3502.5	1994.5
5	车辆平均速度 (m/s) V_{AV}	8.16	18.73	11.76	18.61

除了从时间平均速度层面对应急车道启用实时决策模型作用进行量化外，我们还建立了应急车道实时决策量化指标体系进行决策效果的量化评估。实时决策量化指标体系的最终量化结果如表 9 和图 48、图 49 所示。从实时决策量化指标体系的量化结果可以

看出，平均通行时间、总延误时间、总碰撞暴露时间、总行程时间有效降低，车辆平均速度有效提升。其中，平均通行时间分别下降 41.98%及 45.16%，总延误时间分别下降 59.52%及 37.70%，总碰撞暴露时间分别下降 52.78%及 47.37%，总行程时间分别下降 55.28%及 43.05%，车辆平均速度分别升高 56.43%及 36.81%。通过对量化结果的分析可知，所建立的应急车道启用实时决策模型有效缓解了发生的拥堵，并对未来可能预警的拥堵产生影响。

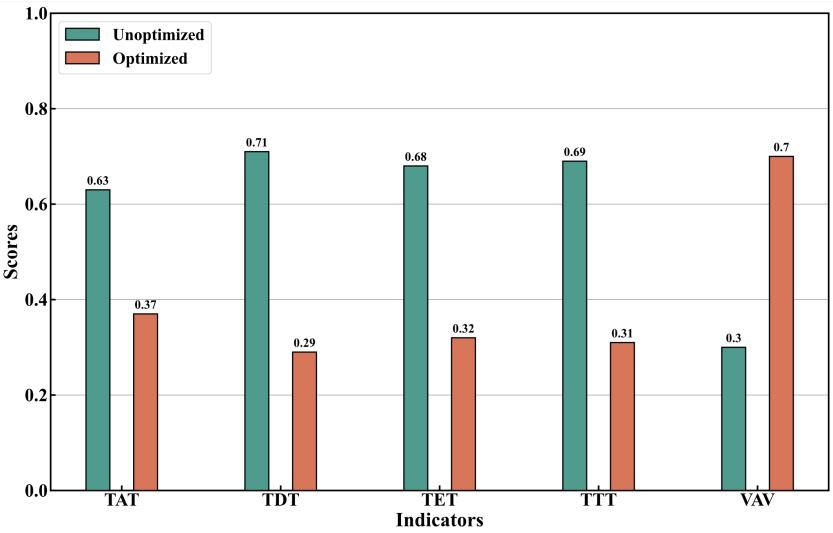


图 48 观测点 4 视频 1 启用前后量化指标柱状图

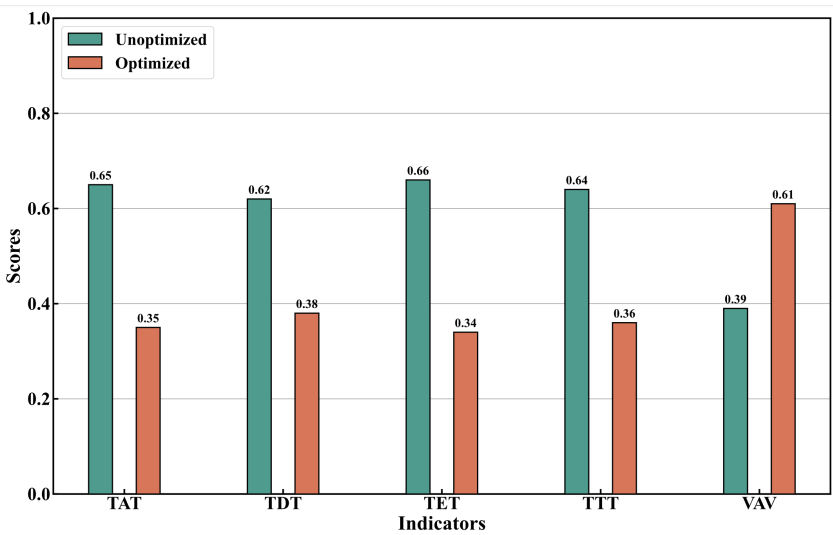


图 49 观测点 4 视频 2 启用前后量化指标柱状图

七、问题四的求解

7.1 问题分析

- 在确保车辆在行驶过程中的道路安全的前提条件下，需考虑以下三点：
- (1) 关键路段:重点关注容易出现拥堵的路段，例如匝道出入口、桥梁入口等。在这些路段增加监控点，可以更准确地监测交通流量变化，提高预警的准确性和及时性。
 - (2) 数据覆盖:确保视频监控覆盖整个路段，以便全面掌握交通流信息，为模型提供更准确的数据支持。
 - (3) 成本控制:合理规划监控点位置，避免过度密集，以控制成本，确保资金效益最大化。

基于上述分析，得到问题四的求解思路如图 52 所示。

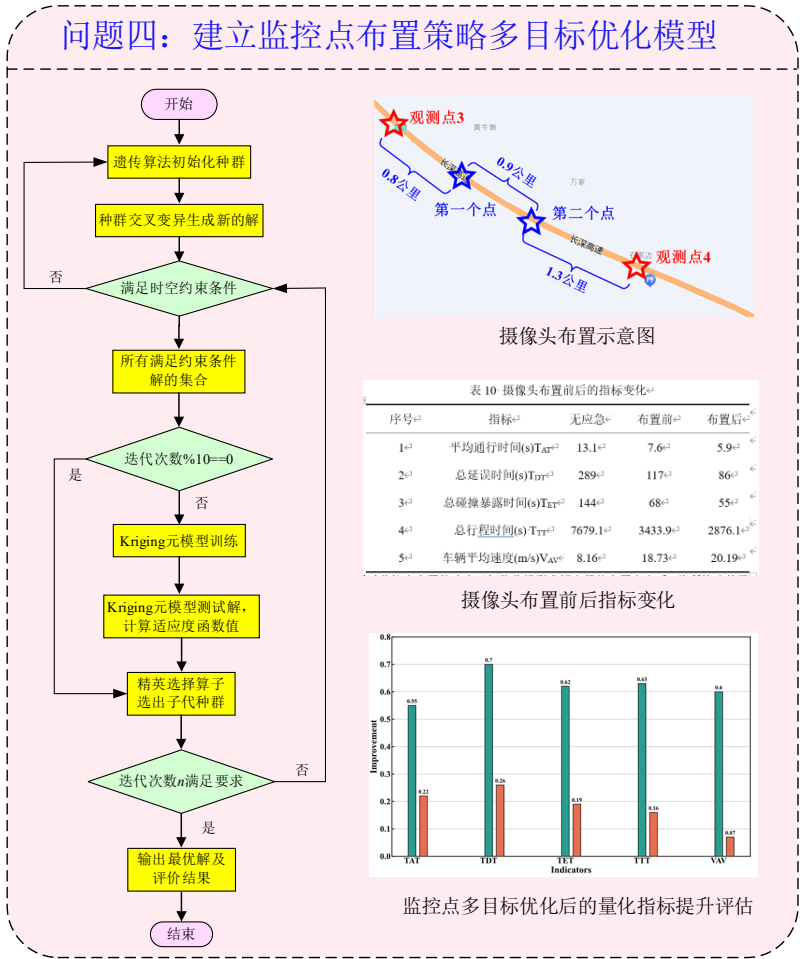


图 50 问题四思路流程图

7.2 监控点布置策略多目标优化模型的建立与分析

视频监控点的布置不但要考虑行车效率的问题，同时还要兼顾安全和成本的问题，基于此，本节考虑效率、安全和成本三个方面作为优化目标。

总行程时间表示路网中所有车辆所花费的行驶时间。当这个总时间最小化时，表明道路的利用效率达到最高，单位时间内的流量也最大。因此，选择所有车辆的总行程时

间作为首要目标指标是合理的。从安全角度来看，总碰撞暴露时间反映了所有车辆在高速公路上处于危险状态的总时长，能够体现高速公路的交通安全状况，因此可以作为第二个目标指标。从成本来看，过多的视频监控点会导致较高的成本，造成资源的浪费，因此将成本作为第三个目标指标。

综合考虑安全性、效率以及成本指标，可以采用加权的方法将出行效率、行车安全以及花费成本这三个目标结合进行优化。通过对这三类指标进行归一化处理，并进行加权求和，可以得到总目标函数：

$$Y = \sum_{k=1}^K \sum_{t=1}^T \sum_{i=1}^L (\alpha_{TTT} T_{TS,i}(t,k) + \beta_{TET} T_{ET,i}(t,k) + \gamma_{COST} N_{COST,i}) \quad (50)$$

式中， $T_{TS,i}(t,k)$ 为路段 i 在第 k 个决策时段第 t 时刻车辆的总行程时间； $T_{ET,i}(t,k)$ 为路段 i 在第 k 个决策时段第 t 时刻车辆的总碰撞暴露时间； $N_{COST,i}$ 为路段 i 中布置监控摄像头的总花费； α_{TTT} 、 β_{TET} 、 γ_{COST} 分别为效率、安全以及成本指标的权重。

为了使得优化模型的求解速度足够快，本节在优化模型中加入了 Kriging 元模型辅助模型求解。该模型可以对全局进行近似，通过寻优算法可以得到高精度的全局近似最优解。通过使用 Kriging 元模型对仿真模型的输入输出隐式关系进行表征，同时结合搜索算法，从而达到加快算法寻优的速度。

Kriging 元模型的基本假设为：

$$G(x) = f(x)^T \beta + Z(x) \quad (51)$$

式中， $G(x)$ 为观察值，由全局估计以及局部估计来确定； $f(x)$ 为回归函数向量； β 为回归系数向量； $Z(x)$ 为均值为 0、方差为 σ^2 的随机过程函数。

Kriging 的基本原理是给出实验数据的某个未知点 x 处的无偏估计值：

$$\hat{g}(x) = f(x)^T \hat{\beta} + Z(x) \quad (52)$$

式中： $\hat{g}(x)$ 为 x 点的估计值； $\hat{\beta}$ 为回归系数向量，可以通过广义最小的平方估计值对其进行估计，从而得到它的值。

本文构建了 Kriging 元模型，以近似仿真模型中的适应值函数，从而在优化过程中提升速度、拓展搜索空间，寻找最优解。为了寻找到最优解，本文选择了遗传算法去进行最优解的求解。

遗传算法是一种基于种群的随机优化方法。其主要操作包括交叉、变异和选择三种。交叉操作模拟了现实世界中父母遗传物质混合的过程，通过这种方式可以产生新的解，而这些解源于两个优秀的父代解，因此通常优于两个父代。变异操作是遗传算法中一个关键的算子，尤其在采用位字符串表示解时，常用位随机翻转突变方法。该方法通过设定突变率，以一定概率翻转 0 或 1。选择操作是遗传算法中的另一重要算子，通过保留适应度较高的个体作为新一代的父代，加速了解向最优解的收敛。其优势在于基于适应

度比例进行采样，确保高适应度解被选择的概率较大，同时也能保留低适应度解，从而降低收敛到局部最优解的风险。

将应急车道控制变量 E_i 设为遗传算法的优化变量，构建每 15 分钟进行优化的应急车道开启与关闭控制模型，以总目标函数作为优化控制的适应度函数。当总目标函数值最小时，表示总行程时间、碰撞暴露时间以及成本之和达到最低，系统在确保安全和成本的前提下效率最高，单位时间内高速公路流量最大，从而实现应急车道控制的最佳效果。图 51 展示了算法流程图，描述了结合 Kriging 模型的遗传算法计算过程。本文中遗传算法的种群规模设为 50，种群代数为 1000，交叉率为 0.9，变异概率为 0.1。

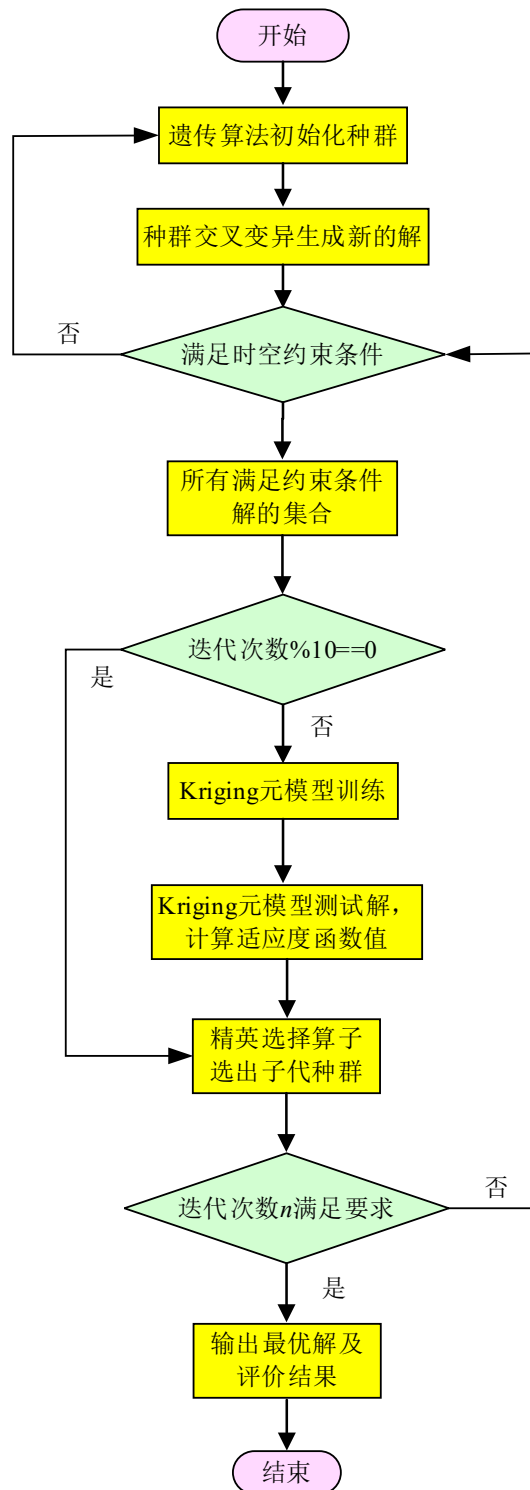


图 51 应急车道实时优化决策算法流程图

7.3 视频监控点最佳布置方案

基于所构建的监控点布置策略多目标优化模型, 综合考虑效率、安全和成本三个优化子目标, 推测出监控点的最佳布置方案。通过所建立的交通流参数预测模型求解交通流描述特征, 并带入多目标优化模型使用结合 Kriging 模型的遗传算法求解出监控点布

置的最佳方案，如图 52 所示。该监控点布置方案在观测点 3 及观测点 4 之间共布置两个监控探头，分别位于距离观测点 3 约 0.8 公里处及距离观测点 4 约 1.3 公里处。

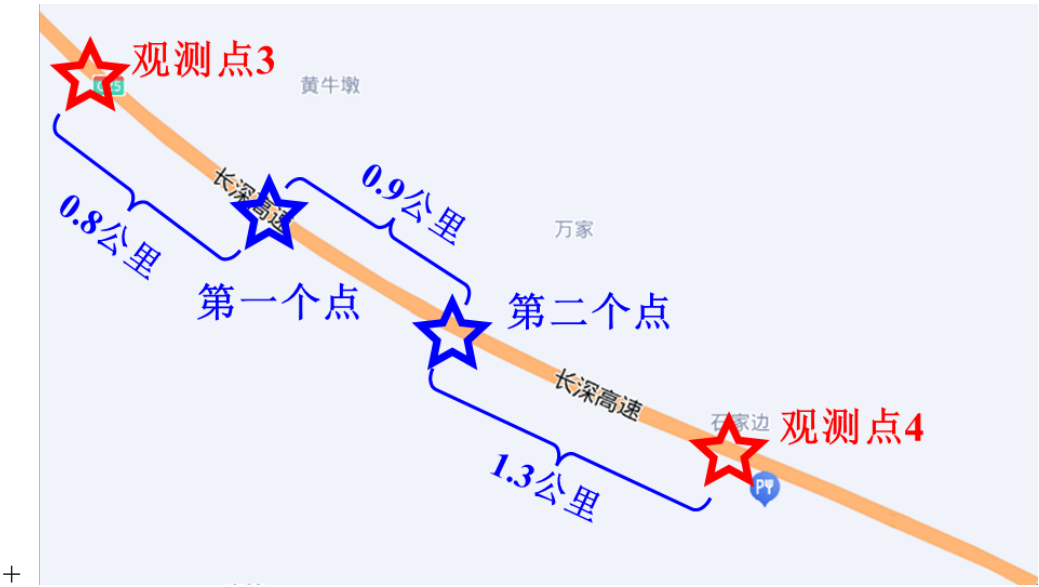


图 52 摄像头布置示意图

表 10 摄像头布置前后的指标变化

序号	指标	无应急	布置前	布置后
1	平均通行时间(s) T_{AT}	13.1	7.6	5.9
2	总延误时间(s) T_{DT}	289	117	86
3	总碰撞暴露时间(s) T_{ET}	144	68	55
4	总行程时间(s) T_{TT}	7679.1	3433.9	2876.1
5	车辆平均速度(m/s) V_{AV}	8.16	18.73	20.19

通过监控点布置策略多目标优化模型求解出最佳布置方案后，将所构建的量化指标体系用于监控点布置新策略的效果评估，评估结果如表 10 及图 53 所示。从量化指标体系可以看出，优化后的监控点布置策略各项指标均取得了提升。其中，平均通行时间降低了 22.37%，总延误时间降低了 26.49%，总碰撞暴露时间降低了 19.12%，总行程时间降低了 16.24%，车辆平均速度提升了 7.23%。

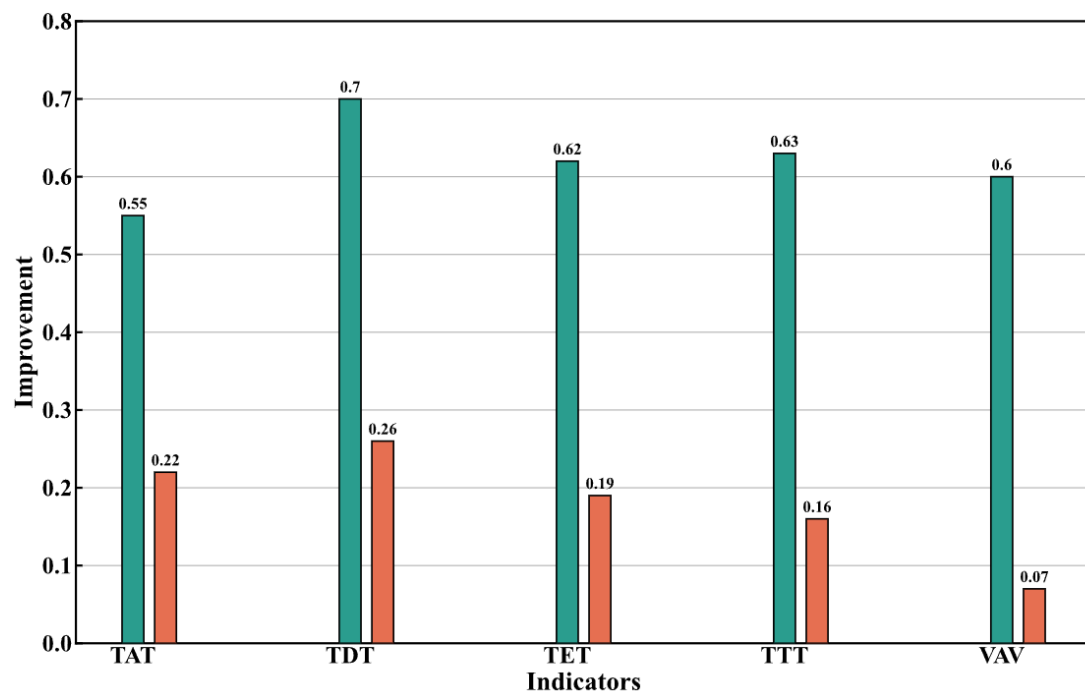


图 53 监控点多目标优化后的量化指标提升评估

八、模型评价

8.1 模型的优点

(1) 针对交通流参数的预测和交通预警信息而建立的交通流参数预测模型和交通预警模型能够同时考虑时间和空间深层特征并进行特征筛选与提取,能够更加准确地预测出交通流参数以及交通拥堵指数。

(2) 针对交通流拥堵概率而建立的交通流中断概率模型采用最大似然估计法估计分布参数,能够准确地给出交通流在不同时刻的拥堵概率,从而确定速度和流量阈值。

(3) 针对应急车道是否启用的实时决策问题而建立的应急车道启用实时决策模型同时利用了交通拥堵预警模型、交通流中断概率模型以及时空安全约束模型,在确保安全的前提下,可以在拥堵发生前一定时间自主决策是否需要开启应急车道。

(4) 针对监控摄像头的合理布置问题,建立了多目标优化函数,同时考虑了效率、安全、成本的指标,并通过 Kriging 元模型辅助的遗传算法模型进行最优值的求解,平衡了效率、安全以及成本的权重,从而更加合理地布置监控摄像头。

8.2 模型的改进和展望

(1) 在从视频数据中提取交通流参数时,可以考虑提取更多的参数,例如车道容量、货车与小轿车的比例等问题,以完善建立的模型。

(2) 对于车流参数的计算可以考虑地更为精细一点,例如可以区分每个车道的车流参数,对每一个车道进行模型的建立分析。

(3) 可以考虑加入更为智能的自动化决策模型,例如利用深度学习技术建立从视频交通流参数提取到控制应急车道开启的全自动化决策模型。

九、 参考文献

- [1] 滕星.图像处理技术在车辆检测系统中的应用[D]. 浙江工业大学, 2012.
- [2] 杨阳, 刘强, 石英杰. 高速公路饱和路段动态应急车道开放决策模型研究[J]. 公路工程, 2022, 47(3) :172-176.
- [3] 郑淑鉴, 杨敬锋. 国内外交通拥堵评价指标计算方法研究[J]. 公路与汽运, 2014, (1): 57-61.
- [4] 王璐媛, 于雷, 孙建平, 等. 交通运行指数的研究与应用综述[J]. 交通信息与安全, 2016, 34(3):1-9,26.
- [5] Brilon W, Lohoff J. Speed -flow models for freeways [J]. Procedia-Social and Behavioral Sciences, 2011, 16: 26-36.
- [6] Brilon W, Geistefeld T J, Regler M. Reliability of free-way traffic flow: a stochastic concept of capacity [C]//Transportation and Traffic Theory: Flow, Dynamics and Human Interaction. 2005:125-144.
- [7] 郭浩宇. 基于多源数据融合的交通拥堵预测方法研究[D]. 北京: 中国人民公安大学, 2022.
- [8] 高榕,万以亮,邵雄凯,等.面向改进的时空 Transformer 的交通流量预测模型[J].计算机工程与应用, 2023, 59(07): 250-260.
- [9] 唐进君, 胡立鹏, 李明洋, 等. 基于 Kriging 遗传算法的高速公路应急车道管控优化[J]. 系统仿真学报, 2024, 36(5): 1165-1178.
- [10] Kononov J, Hersey S, Reeves D, et al. Relationship Between Freeway Flow Parameters and Safety and Its Implications for Hard Shoulder Running[J]. Transportation Research Record, 2012, 2280(1): 10-17.
- [11] Ward J R, Gabriel Agamennoni, Worrall S, et al. Extending Time to Collision for Probabilistic Reasoning in General Traffic Scenarios[J]. Transportation Research Part C: Emerging Technologies, 2015, 51: 66-82.

附录

附录 1

1.问题 1.1 交通流参数求解

```
import cv2
import time
from ultralytics import YOLO, solutions
import openpyxl

cart = 0
road_cars = []
road_car_num = 0
workbook = openpyxl.Workbook()
sheet = workbook.active
sheet['A1'] = '25 帧总车数'
sheet['B1'] = '25 帧车辆列表'

model = YOLO("yolov8s.pt")
names = model.model.names

cap = cv2.VideoCapture("D:\\")
# 获取视频总帧数
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
# 获取视频帧率
fps = cap.get(cv2.CAP_PROP_FPS)
# 获取视频宽度
frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
# 获取视频高度
frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
# 获取视频总帧数
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
# 获取视频时长 (秒)
duration = total_frames / fps

assert cap.isOpened(), "Error reading video file"
w, h, fps = (int(cap.get(x)) for x in (cv2.CAP_PROP_FRAME_WIDTH,
cv2.CAP_PROP_FRAME_HEIGHT, cv2.CAP_PROP_FPS))

line_pts = [(0, 140), (1280, 140)]
speed_obj = solutions.SpeedEstimator(
    reg_pts=line_pts,
    names=names,
    view_img=True,
    spd1_dist_thresh = 50,
)

while True:
    success, im0 = cap.read()
    if not success:
        print("Video frame is empty or video processing has been successfully completed.")
        break

    tracks = model.track(im0, persist=True, conf=0.1)

    if tracks[0].boxes.id is not None:
        for i in tracks[0].boxes.id.int().cpu().tolist():
            if i not in road_cars:
                road_cars.append(i)
                road_car_num += 1
    else:
        continue

    if cart == 33:
        str1 = ' '.join([str(item) for item in road_cars])
        sheet.append([road_car_num, str1])
        road_cars = []
        road_car_num = 0
```

```

        cart = 0

    cart = cart + 1
    workbook.save('108 密度1.xlsx')
    cap.release()
    cv2.destroyAllWindows()

```

2.问题 1.1 交通流速度抓取

```

import time
import cv2
import pandas as pd
from kalman import *
from models.experimental import attempt_load
from utils.general import set_logging, check_img_size
from utils.torch_utils import select_device
from yolov5_detect import detect

counter = 0
counter_up = 0
counter_down = 0
tracker = Sort()
memory = {}
line = [(0, 350), (1800, 350)]
def ccw(A, B, C):
    return (C[1] - A[1]) * (B[0] - A[0]) > (B[1] - A[1]) * (C[0] - A[0])

def intersect(A, B, C, D):
    return ccw(A, C, D) != ccw(B, C, D) and ccw(A, B, C) != ccw(A, B, D)

weights, imgsz = 'yolov5s.pt', 640
set_logging()
device = select_device('')
half = device.type != 'cpu'
model = attempt_load(weights, map_location=device)
stride = int(model.stride.max())
imgsz = check_img_size(imgsz, s=stride) if half:
    model.half()

vs = cv2.VideoCapture('D:\\')
fps = vs.get(cv2.CAP_PROP_FPS)
frame_width = vs.get(cv2.CAP_PROP_FRAME_WIDTH)
frame_height = vs.get(cv2.CAP_PROP_FRAME_HEIGHT)
total_frames = int(vs.get(cv2.CAP_PROP_FRAME_COUNT))
duration = total_frames / fps
frame_time = 0

car_count = []
while True:
    (success, frame) = vs.read()
    if not success:
        break
    (H, W) = frame.shape[:2]
    start = time.time()
    Outputs = detect(source=frame, half=half, model=model, device=device, imgsz=imgsz,
                    stride=stride)
    end = time.time()
    dets = Outputs[:, :4].cpu().numpy()
    np.set_printoptions(formatter={'float': lambda x: "{0:0.3f}".format(x)})
    if np.size(dets) == 0:
        continue
    else:
        tracks = tracker.update(dets)
        boxes = []
        indexIDs = []
        previous = memory.copy()
        memory = {}
        for track in tracks:
            boxes.append([track[0], track[1], track[2], track[3]])
            indexIDs.append(int(track[4]))
            memory[indexIDs[-1]] = boxes[-1]
        np.random.seed(42)
        COLORS = np.random.randint(0, 255, size=(200, 3), dtype='uint8')

```



```

if len(boxes) > 0:
    i = int(0)
    for box in boxes:
        (x, y) = (int(box[0]), int(box[1]))
        (w, h) = (int(box[2]), int(box[3]))
        color = [int(c) for c in COLORS[indexIDs[i] % len(COLORS)]]
        cv2.rectangle(frame, (x, y), (w, h), color, 2)

        if indexIDs[i] in previous:
            previous_box = previous[indexIDs[i]]
            (x2, y2) = (int(previous_box[0]), int(previous_box[1]))
            (w2, h2) = (int(previous_box[2]), int(previous_box[3]))
            p1 = (int(x2 + (w2 - x2) / 2), int(y2 + (h2 - y2) / 2))
            p0 = (int(x + (w - x) / 2), int(y + (h - y) / 2))

            if intersect(p0, p1, line[0], line[1]):
                counter += 1
                if y2 > y:
                    counter_down += 1
                else:
                    counter_up += 1

            i += 1
        cv2.line(frame, line[0], line[1], (0, 255, 0), 3)
        cv2.putText(frame, 'counter:'+str(counter), (30, 40), cv2.FONT_HERSHEY_DUPLEX, 1.0,
(255, 0, 0), 3)
        cv2.putText(frame, 'counter_up:'+str(counter_up), (30, 70), cv2.FONT_HERSHEY_DUPLEX,
1.0, (0, 255, 0), 3)
        cv2.putText(frame, 'counter_down:'+str(counter_down), (30, 100), cv2.FONT_HERSHEY_DU-
PLEX, 1.0, (0, 0, 255), 3)

        if frame_time == 33:
            car_count.append(counter)
            frame_time = 0
            counter = 0
            frame_time += 1

        writer = None
        if writer is None:
            fourcc = cv2.VideoWriter_fourcc(*"mp4v")
            writer = cv2.VideoWriter("./output/output.mp4", fourcc, 25, (frame.shape[1],
frame.shape[0]), True)
            writer.write(frame)
            cv2.imshow("", frame)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

df = pd.DataFrame(car_count, columns=['Numbers'])
df.to_excel('交通量参数.xlsx', index=False)

writer.release()
vs.release()
cv2.destroyAllWindows()

```

3.问题 1.2 交通拥堵实时预警

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

font1 = {'family': 'Times New Roman',
        'weight': 'bold',
        'size': 23,
        } font2 = {'family': 'Times New Roman',
        'weight': 'bold',
        'size': 12,
        }

data = pd.read_excel('D:\\')
array = data['交通指数'].values
plt.figure(figsize=(10, 5))
plt.plot(array, label='TPI')
plt.xlabel('Time', fontdict=font1)

```

```

derivatives = np.gradient(array)
x_label = 0
x_list = []
y_list = []
for num in derivatives:
    if num > 0.85:
        x_list.append(x_label)
        y_list.append(array[x_label])
        x_label += 1

df = pd.DataFrame(x_list, columns=['时间'])
df.to_excel("time.xlsx", index=False)

plt.plot(derivatives, label='window gradient')
plt.scatter(x_list, y_list, color='red', zorder=2, label='warning point')
plt.legend(loc=2, prop=font2)

plt.rcParams['xtick.direction'] = 'in'
plt.rcParams['ytick.direction'] = 'in'
fig = plt.gcf()
ax = plt.gca()
fig.set_size_inches(12, 7)
ax.spines['top'].set_linewidth(2)
ax.spines['bottom'].set_linewidth(2)
ax.spines['left'].set_linewidth(2)
ax.spines['right'].set_linewidth(2)
plt.xticks(fontproperties = 'Times New Roman', size = 24, weight='bold')
plt.yticks(fontproperties = 'Times New Roman', size = 24, weight='bold')
plt.tick_params(direction='inout', width=2, length=6)
plt.show()

```

4.问题 1.3 交通流预测

```

import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

def weight_variables(shape):
    w = tf.Variable(tf.random_normal(shape=shape, mean=0.0, stddev=1.0))
    return w

def bias_variables(shape):
    b = tf.Variable(tf.constant(0.001, shape=shape))
    return b

def attention(inputs, attention_size, time_major=False):
    if isinstance(inputs, tuple):
        inputs = tf.concat(inputs, 2)

    if time_major:
        inputs = tf.transpose(inputs, [1, 0, 2])

    inputs_shape = inputs.shape
    sequence_length = inputs_shape[1].value
    hidden_size = inputs_shape[2].value

    W_omega = tf.Variable(tf.random_normal([hidden_size, attention_size], stddev=0.1))
    b_omega = tf.Variable(tf.random_normal([attention_size], stddev=0.1))
    u_omega = tf.Variable(tf.random_normal([attention_size], stddev=0.1))

    v = tf.tanh(tf.matmul(tf.reshape(inputs, [-1, hidden_size]), W) + tf.reshape(b_omega,
[1, -1]))
    vu = tf.matmul(v, tf.reshape(u_omega, [-1, 1]))
    exps = tf.reshape(tf.exp(vu), [-1, sequence_length])
    alphas = exps / tf.reshape(tf.reduce_sum(exps, 1), [-1, 1])
    output = tf.reduce_sum(inputs * tf.reshape(alphas, [-1, sequence_length, 1]), 1)

    return output

def CNN_BIGRU_Attention(x, n_future, n_class, is_training=False):
    x_reshape = tf.reshape(x, [-1, 1, n_future, 1])
    with tf.variable_scope("conv1"):
        w_conv1 = weight_variables([1, 3, 1, 6])
        b_conv1 = bias_variables([6])

```

```

        x_conv1 = tf.nn.conv2d(x_reshape, w_conv1, strides=[1, 1, 1, 1], padding="SAME") +
b_conv1
        x_relu1 = tf.nn.relu(x_conv1)
        x_pool1 = tf.nn.max_pool(x_relu1, ksize=[1, 1, 2, 1], strides=[1, 1, 2, 1], padding="SAME")
        with tf.variable_scope("cov2"):
            w_conv2 = weight_variables([1, 3, 6, 16])
            b_conv2 = bias_variables([16])
            x_conv2 = tf.nn.conv2d(x_pool1, w_conv2, strides=[1, 1, 1, 1], padding="SAME") +
b_conv2
            x_relu2 = tf.nn.relu(x_conv2)
            x_pool2 = tf.nn.max_pool(x_relu2, ksize=[1, 1, 2, 1], strides=[1, 1, 2, 1], padding="SAME")
            pool_shape = x_pool2.get_shape().as_list()
            nodes = pool_shape[1] * pool_shape[2] * pool_shape[3]
            x_fc_reshape = tf.reshape(x_pool2, [-1, nodes])

        with tf.variable_scope("conv_fc"):
            fc1 = 128
            w_fc1 = weight_variables([nodes, fc1])
            b_fc1 = bias_variables([fc1])
            y_fc1 = tf.matmul(x_fc_reshape, w_fc1) + b_fc1
            x_reshape = tf.reshape(y_fc1, [-1, 1, fc1])
        with tf.variable_scope("BIGRU"):
            rnn_cellforward = tf.nn.rnn_cell.MultiRNNCell([tf.nn.rnn_cell.GRUCell(20),
tf.nn.rnn_cell.GRUCell(20)])
            rnn_cellbackward = tf.nn.rnn_cell.MultiRNNCell([tf.nn.rnn_cell.GRUCell(20),
tf.nn.rnn_cell.GRUCell(20)])

            outputs, _ = tf.nn.bidirectional_dynamic_rnn(rnn_cellforward, rnn_cellbackward, x_reshape, dtype=tf.float32)
            rnn_out = outputs
        with tf.variable_scope("attention"):
            attention_size = 64
            attention_out = attention(rnn_out, attention_size, False)
            pool_shape = attention_out.get_shape().as_list()
            nodes = pool_shape[1]
            x_at_reshape = tf.reshape(attention_out, [-1, nodes])
        with tf.variable_scope("output"):
            w_fc2 = weight_variables([nodes, n_class])
            b_fc2 = bias_variables([n_class])
            y_predict = tf.matmul(x_at_reshape, w_fc2) + b_fc2
            y = tf.reshape(y_predict, [-1, n_class])
        return y

```

5.问题 2 应急车道决策

```

def identity_block(input_tensor, kernel_size, filters, stage, block,
                    use_bias=True, train_bn=True):
    nb_filter1, nb_filter2, nb_filter3 = filters
    conv_name_base = 'res' + str(stage) + block + '_branch'
    bn_name_base = 'bn' + str(stage) + block + '_branch'

    x = KL.Conv2D(nb_filter1, (1, 1), name=conv_name_base + '2a',
                    use_bias=use_bias)(input_tensor)
    x = BatchNorm(name=bn_name_base + '2a')(x, training=train_bn)
    x = KL.Activation('relu')(x)

    x = KL.Conv2D(nb_filter2, (kernel_size, kernel_size), padding='same',
                    name=conv_name_base + '2b', use_bias=use_bias)(x)
    x = BatchNorm(name=bn_name_base + '2b')(x, training=train_bn)
    x = KL.Activation('relu')(x)

    x = KL.Conv2D(nb_filter3, (1, 1), name=conv_name_base + '2c',
                    use_bias=use_bias)(x)
    x = BatchNorm(name=bn_name_base + '2c')(x, training=train_bn)

    x = KL.Add()([x, input_tensor])
    x = KL.Activation('relu', name='res' + str(stage) + block + '_out')(x)
    return x

def conv_block(input_tensor, kernel_size, filters, stage, block,
                strides=(2, 2), use_bias=True, train_bn=True):
    nb_filter1, nb_filter2, nb_filter3 = filters

```

```

conv_name_base = 'res' + str(stage) + block + '_branch'
bn_name_base = 'bn' + str(stage) + block + '_branch'

x = KL.Conv2D(nb_filter1, (1, 1), strides=strides,
              name=conv_name_base + '2a', use_bias=use_bias)(input_tensor)
x = BatchNorm(name=bn_name_base + '2a')(x, training=train_bn)
x = KL.Activation('relu')(x)

x = KL.Conv2D(nb_filter2, (kernel_size, kernel_size), padding='same',
              name=conv_name_base + '2b', use_bias=use_bias)(x)
x = BatchNorm(name=bn_name_base + '2b')(x, training=train_bn)
x = KL.Activation('relu')(x)

x = KL.Conv2D(nb_filter3, (1, 1), name=conv_name_base +
              '2c', use_bias=use_bias)(x)
x = BatchNorm(name=bn_name_base + '2c')(x, training=train_bn)

shortcut = KL.Conv2D(nb_filter3, (1, 1), strides=strides,
                    name=conv_name_base + '1', use_bias=use_bias)(input_tensor)
shortcut = BatchNorm(name=bn_name_base + '1')(shortcut, training=train_bn)

x = KL.Add()([x, shortcut])
x = KL.Activation('relu', name='res' + str(stage) + block + '_out')(x)
return x

def resnet_graph(input_image, architecture, stage5=False, train_bn=True):
    assert architecture in ["resnet50", "resnet101"]
    x = KL.ZeroPadding2D((3, 3))(input_image)
    x = KL.Conv2D(64, (7, 7), strides=(2, 2), name='conv1', use_bias=True)(x)
    x = BatchNorm(name='bn_conv1')(x, training=train_bn)
    x = KL.Activation('relu')(x)
    C1 = x = KL.MaxPooling2D((3, 3), strides=(2, 2), padding="same")(x)
    x = conv_block(x, 3, [64, 64, 256], stage=2, block='a', strides=(1, 1),
train_bn=train_bn)
    x = identity_block(x, 3, [64, 64, 256], stage=2, block='b', train_bn=train_bn)
    C2 = x = identity_block(x, 3, [64, 64, 256], stage=2, block='c', train_bn=train_bn)
    x = conv_block(x, 3, [128, 128, 512], stage=3, block='a', train_bn=train_bn)
    x = identity_block(x, 3, [128, 128, 512], stage=3, block='b', train_bn=train_bn)
    x = identity_block(x, 3, [128, 128, 512], stage=3, block='c', train_bn=train_bn)
    C3 = x = identity_block(x, 3, [128, 128, 512], stage=3, block='d', train_bn=train_bn)
    x = conv_block(x, 3, [256, 256, 1024], stage=4, block='a', train_bn=train_bn)
    block_count = {"resnet50": 5, "resnet101": 22}[architecture]
    for i in range(block_count):
        x = identity_block(x, 3, [256, 256, 1024], stage=4, block=chr(98 + i),
train_bn=train_bn)
    C4 = x
    if stage5:
        x = conv_block(x, 3, [512, 512, 2048], stage=5, block='a', train_bn=train_bn)
        x = identity_block(x, 3, [512, 512, 2048], stage=5, block='b', train_bn=train_bn)
        C5 = x = identity_block(x, 3, [512, 512, 2048], stage=5, block='c',
train_bn=train_bn)
    else:
        C5 = None
    return [C1, C2, C3, C4, C5]

def apply_box_deltas_graph(boxes, deltas):
    height = boxes[:, 2] - boxes[:, 0]
    width = boxes[:, 3] - boxes[:, 1]
    center_y = boxes[:, 0] + 0.5 * height
    center_x = boxes[:, 1] + 0.5 * width
    center_y += deltas[:, 0] * height
    center_x += deltas[:, 1] * width
    height *= tf.exp(deltas[:, 2])
    width *= tf.exp(deltas[:, 3])
    y1 = center_y - 0.5 * height
    x1 = center_x - 0.5 * width
    y2 = y1 + height
    x2 = x1 + width
    result = tf.stack([y1, x1, y2, x2], axis=1, name="apply_box_deltas_out")
    return result

def clip_boxes_graph(boxes, window):
    wy1, wx1, wy2, wx2 = tf.split(window, 4)

```

```

y1, x1, y2, x2 = tf.split(boxes, 4, axis=1)
y1 = tf.maximum(tf.minimum(y1, wy2), wy1)
x1 = tf.maximum(tf.minimum(x1, wx2), wx1)
y2 = tf.maximum(tf.minimum(y2, wy2), wy1)
x2 = tf.maximum(tf.minimum(x2, wx2), wx1)
clipped = tf.concat([y1, x1, y2, x2], axis=1, name="clipped_boxes")
clipped.set_shape((clipped.shape[0], 4))
return clipped

class ProposalLayer(KE.Layer):

    def __init__(self, proposal_count, nms_threshold, config=None, **kwargs):
        super(ProposalLayer, self).__init__(**kwargs)
        self.config = config
        self.proposal_count = proposal_count
        self.nms_threshold = nms_threshold

    def call(self, inputs):
        scores = inputs[0][:, :, 1]
        deltas = inputs[1]
        deltas = deltas * np.reshape(self.config.RPN_BBOX_STD_DEV, [1, 1, 4])
        anchors = inputs[2]
        pre_nms_limit = tf.minimum(self.config.PRE_NMS_LIMIT, tf.shape(anchors)[1])
        ix = tf.nn.top_k(scores, pre_nms_limit, sorted=True,
                        name="top_anchors").indices
        scores = utils.batch_slice([scores, ix], lambda x, y: tf.gather(x, y),
                                   self.config.IMAGES_PER_GPU)
        deltas = utils.batch_slice([deltas, ix], lambda x, y: tf.gather(x, y),
                                   self.config.IMAGES_PER_GPU)
        pre_nms_anchors = utils.batch_slice([anchors, ix], lambda a, x: tf.gather(a, x),
                                             self.config.IMAGES_PER_GPU,
                                             names=["pre_nms_anchors"])
        boxes = utils.batch_slice([pre_nms_anchors, deltas],
                                   lambda x, y: apply_box_deltas_graph(x, y),
                                   self.config.IMAGES_PER_GPU,
                                   names=["refined_anchors"])
        window = np.array([0, 0, 1, 1], dtype=np.float32)
        boxes = utils.batch_slice(boxes,
                                   lambda x: clip_boxes_graph(x, window),
                                   self.config.IMAGES_PER_GPU,
                                   names=["refined_anchors_clipped"])

    def nms(self, boxes, scores):
        indices = tf.image.non_max_suppression(
            boxes, scores, self.proposal_count,
            self.nms_threshold, name="rpn_non_max_suppression")
        proposals = tf.gather(boxes, indices)
        # Pad if needed
        padding = tf.maximum(self.proposal_count - tf.shape(proposals)[0], 0)
        proposals = tf.pad(proposals, [(0, padding), (0, 0)])
        return proposals

    proposals = utils.batch_slice([boxes, scores], nms,
                                   self.config.IMAGES_PER_GPU)

    return proposals

    def compute_output_shape(self, input_shape):
        return (None, self.proposal_count, 4)

def log2_graph(x):
    return tf.log(x) / tf.log(2.0)

class PyramidROIAlign(KE.Layer):

    def __init__(self, pool_shape, **kwargs):
        super(PyramidROIAlign, self).__init__(**kwargs)
        self.pool_shape = tuple(pool_shape)

    def call(self, inputs):
        boxes = inputs[0]
        image_meta = inputs[1]
        feature_maps = inputs[2:]
        y1, x1, y2, x2 = tf.split(boxes, 4, axis=2)

```

```

h = y2 - y1
w = x2 - x1
image_shape = parse_image_meta_graph(image_meta)['image_shape'][0]
image_area = tf.cast(image_shape[0] * image_shape[1], tf.float32)
roi_level = log2_graph(tf.sqrt(h * w) / (224.0 / tf.sqrt(image_area)))
roi_level = tf.minimum(5, tf.maximum(
    2, 4 + tf.cast(tf.round(roi_level), tf.int32)))
roi_level = tf.squeeze(roi_level, 2)
pooled = []
box_to_level = []
for i, level in enumerate(range(2, 6)):
    ix = tf.where(tf.equal(roi_level, level))
    level_boxes = tf.gather_nd(boxes, ix)
    box_indices = tf.cast(ix[:, 0], tf.int32)
    box_to_level.append(ix)
    level_boxes = tf.stop_gradient(level_boxes)
    box_indices = tf.stop_gradient(box_indices)
    pooled.append(tf.image.crop_and_resize(
        feature_maps[i], level_boxes, box_indices, self.pool_shape,
        method="bilinear"))
pooled = tf.concat(pooled, axis=0)
box_to_level = tf.concat(box_to_level, axis=0)
box_range = tf.expand_dims(tf.range(tf.shape(box_to_level)[0]), 1)
box_to_level = tf.concat([tf.cast(box_to_level, tf.int32), box_range],
    axis=1)
sorting_tensor = box_to_level[:, 0] * 100000 + box_to_level[:, 1]
ix = tf.nn.top_k(sorting_tensor, k=tf.shape(
    box_to_level[0]).indices[:-1])
ix = tf.gather(box_to_level[:, 2], ix)
pooled = tf.gather(pooled, ix)
shape = tf.concat([tf.shape(boxes)[:2], tf.shape(pooled)[1:]], axis=0)
pooled = tf.reshape(pooled, shape)
return pooled

def compute_output_shape(self, input_shape):
    return input_shape[0][:2] + self.pool_shape + (input_shape[2]-1,)

def overlaps_graph(boxes1, boxes2):
    b1 = tf.reshape(tf.tile(tf.expand_dims(boxes1, 1),
        [1, 1, tf.shape(boxes2)[0]]), [-1, 4])
    b1_y1, b1_x1, b1_y2, b1_x2 = tf.split(b1, 4, axis=1)
    b2_y1, b2_x1, b2_y2, b2_x2 = tf.split(b2, 4, axis=1)
    y1 = tf.maximum(b1_y1, b2_y1)
    x1 = tf.maximum(b1_x1, b2_x1)
    y2 = tf.minimum(b1_y2, b2_y2)
    x2 = tf.minimum(b1_x2, b2_x2)
    intersection = tf.maximum(x2 - x1, 0) * tf.maximum(y2 - y1, 0)
    b1_area = (b1_y2 - b1_y1) * (b1_x2 - b1_x1)
    b2_area = (b2_y2 - b2_y1) * (b2_x2 - b2_x1)
    union = b1_area + b2_area - intersection
    iou = intersection / union
    overlaps = tf.reshape(iou, [tf.shape(boxes1)[0], tf.shape(boxes2)[0]])
    return overlaps

def detection_targets_graph(proposals, gt_class_ids, gt_boxes, gt_masks, config):
    asserts = [
        tf.Assert(tf.greater(tf.shape(proposals)[0], 0), [proposals],
            name="roi_assertion"),
    ]
    with tf.control_dependencies(asserts):
        proposals = tf.identity(proposals)

    proposals, _ = trim_zeros_graph(proposals, name="trim proposals")
    gt_boxes, non_zeros = trim_zeros_graph(gt_boxes, name="trim gt_boxes")
    gt_class_ids = tf.boolean_mask(gt_class_ids, non_zeros,
        name="trim_gt_class_ids")
    gt_masks = tf.gather(gt_masks, tf.where(non_zeros)[:, 0], axis=2,
        name="trim_gt_masks")
    crowd_ix = tf.where(gt_class_ids < 0)[:, 0]
    non_crowd_ix = tf.where(gt_class_ids > 0)[:, 0]
    crowd_boxes = tf.gather(gt_boxes, crowd_ix)
    gt_class_ids = tf.gather(gt_class_ids, non_crowd_ix)
    gt_boxes = tf.gather(gt_boxes, non_crowd_ix)
    gt_masks = tf.gather(gt_masks, non_crowd_ix, axis=2)
    overlaps = overlaps_graph(proposals, gt_boxes)

```

```

crowd_overlaps = overlaps_graph(proposals, crowd_boxes)
crowd_iou_max = tf.reduce_max(crowd_overlaps, axis=1)
no_crowd_bool = (crowd_iou_max < 0.001)
roi_iou_max = tf.reduce_max(overlaps, axis=1)
positive_roi_bool = (roi_iou_max >= 0.5)
positive_indices = tf.where(positive_roi_bool)[: , 0]
negative_indices = tf.where(tf.logical_and(roi_iou_max < 0.5, no_crowd_bool))[: , 0]
positive_count = int(config.TRAIN_ROIS_PER_IMAGE *
                     config.ROI_POSITIVE_RATIO)
positive_indices = tf.random_shuffle(positive_indices)[:positive_count]
positive_count = tf.shape(positive_indices)[0]
negative_count = tf.cast(r * tf.cast(positive_count, tf.float32), tf.int32) - positive_count
negative_indices = tf.random_shuffle(negative_indices)[:negative_count]
positive_rois = tf.gather(proposals, positive_indices)
negative_rois = tf.gather(proposals, negative_indices)
positive_overlaps = tf.gather(overlaps, positive_indices)
roi_gt_box_assignment = tf.cond(
    tf.greater(tf.shape(positive_overlaps)[1], 0),
    true_fn=lambda: tf.argmax(positive_overlaps, axis=1),
    false_fn=lambda: tf.cast(tf.constant([]), tf.int64)
)
roi_gt_boxes = tf.gather(gt_boxes, roi_gt_box_assignment)
roi_gt_class_ids = tf.gather(gt_class_ids, roi_gt_box_assignment)
deltas = utils.box_refinement_graph(positive_rois, roi_gt_boxes)
deltas /= config.BBOX_STD_DEV
roi_masks = tf.gather(transposed_masks, roi_gt_box_assignment)
boxes = positive_rois
if config.USE_MINI_MASK:
    y1, x1, y2, x2 = tf.split(positive_rois, 4, axis=1)
    gt_y1, gt_x1, gt_y2, gt_x2 = tf.split(roi_gt_boxes, 4, axis=1)
    gt_h = gt_y2 - gt_y1
    gt_w = gt_x2 - gt_x1
    y1 = (y1 - gt_y1) / gt_h
    x1 = (x1 - gt_x1) / gt_w
    y2 = (y2 - gt_y1) / gt_h
    x2 = (x2 - gt_x1) / gt_w
    boxes = tf.concat([y1, x1, y2, x2], 1)
box_ids = tf.range(0, tf.shape(roi_masks)[0])
masks = tf.image.crop_and_resize(tf.cast(roi_masks, tf.float32), boxes,
                                box_ids,
                                config.MASK_SHAPE)
masks = tf.squeeze(masks, axis=3)
masks = tf.round(masks)
rois = tf.concat([positive_rois, negative_rois], axis=0)
N = tf.shape(negative_rois)[0]
P = tf.maximum(config.TRAIN_ROIS_PER_IMAGE - tf.shape(rois)[0], 0)
rois = tf.pad(rois, [(0, P), (0, 0)])
roi_gt_boxes = tf.pad(roi_gt_boxes, [(0, N + P), (0, 0)])
roi_gt_class_ids = tf.pad(roi_gt_class_ids, [(0, N + P)])
deltas = tf.pad(deltas, [(0, N + P), (0, 0)])
masks = tf.pad(masks, [(0, N + P), (0, 0), (0, 0)])

return rois, roi_gt_class_ids, deltas, masks

class DetectionTargetLayer(KE.Layer):
    def __init__(self, config, **kwargs):
        super(DetectionTargetLayer, self).__init__(**kwargs)
        self.config = config

    def call(self, inputs):
        proposals = inputs[0]
        gt_class_ids = inputs[1]
        gt_boxes = inputs[2]
        gt_masks = inputs[3]
        names = ["rois", "target_class_ids", "target_bbox", "target_mask"]
        outputs = utils.batch_slice(
            [proposals, gt_class_ids, gt_boxes, gt_masks],
            lambda w, x, y, z: detection_targets_graph(
                w, x, y, z, self.config),
            self.config.IMAGES_PER_GPU, names=names)
        return outputs

    def compute_output_shape(self, input_shape):
        return [

```

```

        (None, self.config.TRAIN_ROIS_PER_IMAGE, 4), # rois
        (None, self.config.TRAIN_ROIS_PER_IMAGE), # class_ids
        (None, self.config.TRAIN_ROIS_PER_IMAGE, 4), # deltas
        (None, self.config.TRAIN_ROIS_PER_IMAGE, self.config.MASK_SHAPE[0],
         self.config.MASK_SHAPE[1]) # masks
    ]

    def compute_mask(self, inputs, mask=None):
        return [None, None, None, None]

def refine_detections_graph(rois, probs, deltas, window, config):
    class_ids = tf.argmax(probs, axis=1, output_type=tf.int32)
    indices = tf.stack([tf.range(probs.shape[0]), class_ids], axis=1)
    class_scores = tf.gather_nd(probs, indices)
    deltas_specific = tf.gather_nd(deltas, indices)
    refined_rois = apply_box_deltas_graph(
        refined_rois = clip_boxes_graph(refined_rois, window)
    keep = tf.where(class_ids > 0)[: , 0]
    if config.DETECTION_MIN_CONFIDENCE:
        conf_keep = tf.where(class_scores >= config.DETECTION_MIN_CONFIDENCE)[: , 0]
        keep = tf.sets.set_intersection(tf.expand_dims(keep, 0),
                                       tf.expand_dims(conf_keep, 0))

        keep = tf.sparse_tensor_to_dense(keep)[0]
    pre_nms_class_ids = tf.gather(class_ids, keep)
    pre_nms_scores = tf.gather(class_scores, keep)
    pre_nms_rois = tf.gather(refined_rois, keep)
    unique_pre_nms_class_ids = tf.unique(pre_nms_class_ids)[0]

    def nms_keep_map(class_id):
        ixs = tf.where(tf.equal(pre_nms_class_ids, class_id))[: , 0]
        class_keep = tf.image.non_max_suppression(
            tf.gather(pre_nms_rois, ixs),
            tf.gather(pre_nms_scores, ixs),
            max_output_size=config.DETECTION_MAX_INSTANCES,
            iou_threshold=config.DETECTION_NMS_THRESHOLD)
        class_keep = tf.gather(keep, tf.gather(ixs, class_keep))
        gap = config.DETECTION_MAX_INSTANCES - tf.shape(class_keep)[0]
        class_keep = tf.pad(class_keep, [(0, gap)],
                           mode='CONSTANT', constant_values=-1)
        class_keep.set_shape([config.DETECTION_MAX_INSTANCES])
        return class_keep

    nms_keep = tf.map_fn(nms_keep_map, unique_pre_nms_class_ids,
                        dtype=tf.int64)
    nms_keep = tf.reshape(nms_keep, [-1])
    nms_keep = tf.gather(nms_keep, tf.where(nms_keep > -1)[: , 0])
    keep = tf.sets.set_intersection(tf.expand_dims(keep, 0),
                                   tf.expand_dims(nms_keep, 0))

    keep = tf.sparse_tensor_to_dense(keep)[0]
    roi_count = config.DETECTION_MAX_INSTANCES
    class_scores_keep = tf.gather(class_scores, keep)
    num_keep = tf.minimum(tf.shape(class_scores_keep)[0], roi_count)
    top_ids = tf.nn.top_k(class_scores_keep, k=num_keep, sorted=True)[1]
    keep = tf.gather(keep, top_ids)
    detections = tf.concat([
        tf.gather(refined_rois, keep),
        tf.to_float(tf.gather(class_ids, keep))[..., tf.newaxis],
        tf.gather(class_scores, keep)[..., tf.newaxis]
    ], axis=1)
    gap = config.DETECTION_MAX_INSTANCES - tf.shape(detections)[0]
    detections = tf.pad(detections, [(0, gap), (0, 0)], "CONSTANT")
    return detections

class DetectionLayer(KE.Layer):

    def __init__(self, config=None, **kwargs):
        super(DetectionLayer, self).__init__(**kwargs)
        self.config = config

    def call(self, inputs):
        rois = inputs[0]
        mrcnn_class = inputs[1]
        mrcnn_bbox = inputs[2]
        image_meta = inputs[3]

```



```

        m = parse_image_meta_graph(image_meta)
        image_shape = m['image_shape'][0]
        window = norm_boxes_graph(m['window'], image_shape[:2])
        detections_batch = utils.batch_slice(
            [rois, mrcnn_class, mrcnn_bbox, window],
            lambda x, y, w, z: refine_detections_graph(x, y, w, z, self.config),
            self.config.IMAGES_PER_GPU)
        return tf.reshape(
            detections_batch,
            [self.config.BATCH_SIZE, self.config.DETECTION_MAX_INSTANCES, 6])

    def compute_output_shape(self, input_shape):
        return (None, self.config.DETECTION_MAX_INSTANCES, 6)

def rpn_graph(feature_map, anchors_per_location, anchor_stride):
    shared = KL.Conv2D(512, (3, 3), padding='same', activation='relu',
                       strides=anchor_stride,
                       name='rpn_conv_shared')(feature_map)
    x = KL.Conv2D(2 * anchors_per_location, (1, 1), padding='valid',
                 activation='linear', name='rpn_class_raw')(shared)
    rpn_class_logits = KL.Lambda(
        lambda t: tf.reshape(t, [tf.shape(t)[0], -1, 2]))(x)
    rpn_probs = KL.Activation(
        "softmax", name="rpn_class_XXX")(rpn_class_logits)
    x = KL.Conv2D(anchors_per_location * 4, (1, 1), padding="valid",
                 activation='linear', name='rpn_bbox_pred')(shared)
    rpn_bbox = KL.Lambda(lambda t: tf.reshape(t, [tf.shape(t)[0], -1, 4]))(x)

    return [rpn_class_logits, rpn_probs, rpn_bbox]

def build_rpn_model(anchor_stride, anchors_per_location, depth):
    input_feature_map = KL.Input(shape=[None, None, depth],
                                 name="input_rpn_feature_map")
    outputs = rpn_graph(input_feature_map, anchors_per_location, anchor_stride)
    return KM.Model([input_feature_map], outputs, name="rpn_model")

def fpn_classifier_graph(rois, feature_maps, image_meta,
                        pool_size, num_classes, train_bn=True,
                        fc_layers_size=1024):
    x = PyramidROIAlign([pool_size, pool_size],
                       name="roi_align_classifier")([rois, image_meta] + feature_maps)
    x = KL.TimeDistributed(KL.Conv2D(fc_layers_size, (pool_size, pool_size), padding="valid"),
                          name="mrcnn_class_conv1")(x)
    x = KL.TimeDistributed(BatchNorm(), name='mrcnn_class_bn1')(x, training=train_bn)
    x = KL.Activation('relu')(x)
    x = KL.TimeDistributed(KL.Conv2D(fc_layers_size, (1, 1)),
                          name="mrcnn_class_conv2")(x)
    x = KL.TimeDistributed(BatchNorm(), name='mrcnn_class_bn2')(x, training=train_bn)
    x = KL.Activation('relu')(x)

    shared = KL.Lambda(lambda x: K.squeeze(K.squeeze(x, 3), 2),
                       name="pool_squeeze")(x)
    mrcnn_class_logits = KL.TimeDistributed(KL.Dense(num_classes),
                                           name='mrcnn_class_logits')(shared)
    mrcnn_probs = KL.TimeDistributed(KL.Activation("softmax"),
                                    name="mrcnn_class")(mrcnn_class_logits)
    x = KL.TimeDistributed(KL.Dense(num_classes * 4, activation='linear'),
                          name='mrcnn_bbox_fc')(shared)
    s = K.int_shape(x)
    mrcnn_bbox = KL.Reshape((s[1], num_classes, 4), name="mrcnn_bbox")(x)

    return mrcnn_class_logits, mrcnn_probs, mrcnn_bbox

def build_fpn_mask_graph(rois, feature_maps, image_meta,
                        pool_size, num_classes, train_bn=True):
    x = PyramidROIAlign([pool_size, pool_size],
                       name="roi_align_mask")([rois, image_meta] + feature_maps)

    x = KL.TimeDistributed(KL.Conv2D(256, (3, 3), padding="same"),
                          name="mrcnn_mask_conv1")(x)
    x = KL.TimeDistributed(BatchNorm(),
                          name='mrcnn_mask_bn1')(x, training=train_bn)

```

```

x = KL.Activation('relu')(x)

x = KL.TimeDistributed(KL.Conv2D(256, (3, 3), padding="same"),
                        name="mrcnn_mask_conv2")(x)
x = KL.TimeDistributed(BatchNorm(),
                        name='mrcnn_mask_bn2')(x, training=train_bn)
x = KL.Activation('relu')(x)

x = KL.TimeDistributed(KL.Conv2D(256, (3, 3), padding="same"),
                        name="mrcnn_mask_conv3")(x)
x = KL.TimeDistributed(BatchNorm(),
                        name='mrcnn_mask_bn3')(x, training=train_bn)
x = KL.Activation('relu')(x)

x = KL.TimeDistributed(KL.Conv2D(256, (3, 3), padding="same"),
                        name="mrcnn_mask_conv4")(x)
x = KL.TimeDistributed(BatchNorm(),
                        name='mrcnn_mask_bn4')(x, training=train_bn)
x = KL.Activation('relu')(x)

x = KL.TimeDistributed(KL.Conv2DTranspose(256, (2, 2), strides=2, activation="relu"),
                        name="mrcnn_mask_deconv")(x)
x = KL.TimeDistributed(KL.Conv2D(num_classes, (1, 1), strides=1, activation="sigmoid"),
                        name="mrcnn_mask")(x)

return x

def smooth_l1_loss(y_true, y_pred):
    diff = K.abs(y_true - y_pred)
    less_than_one = K.cast(K.less(diff, 1.0), "float32")
    loss = (less_than_one * 0.5 * diff ** 2) + (1 - less_than_one) * (diff - 0.5)
    return loss

def rpn_class_loss_graph(rpn_match, rpn_class_logits):
    rpn_match = tf.squeeze(rpn_match, -1)
    anchor_class = K.cast(K.equal(rpn_match, 1), tf.int32)
    indices = tf.where(K.not_equal(rpn_match, 0))
    rpn_class_logits = tf.gather_nd(rpn_class_logits, indices)
    anchor_class = tf.gather_nd(anchor_class, indices)
    loss = K.sparse_categorical_crossentropy(target=anchor_class,
                                           output=rpn_class_logits,
                                           from_logits=True)
    loss = K.switch(tf.size(loss) > 0, K.mean(loss), tf.constant(0.0))
    return loss

def rpn_bbox_loss_graph(config, target_bbox, rpn_match, rpn_bbox):
    rpn_match = K.squeeze(rpn_match, -1)
    indices = tf.where(K.equal(rpn_match, 1))

    rpn_bbox = tf.gather_nd(rpn_bbox, indices)

    batch_counts = K.sum(K.cast(K.equal(rpn_match, 1), tf.int32), axis=1)
    target_bbox = batch_pack_graph(target_bbox, batch_counts,
                                   config.IMAGES_PER_GPU)

    loss = smooth_l1_loss(target_bbox, rpn_bbox)

    loss = K.switch(tf.size(loss) > 0, K.mean(loss), tf.constant(0.0))
    return loss

def mrcnn_class_loss_graph(target_class_ids, pred_class_logits,
                           active_class_ids):
    target_class_ids = tf.cast(target_class_ids, 'int64')
    pred_class_ids = tf.argmax(pred_class_logits, axis=2)
    loss = tf.nn.sparse_softmax_cross_entropy_with_logits(
        labels=target_class_ids, logits=pred_class_logits)
    loss = loss * pred_active
    loss = tf.reduce_sum(loss) / tf.reduce_sum(pred_active)
    return loss

def mrcnn_bbox_loss_graph(target_bbox, target_class_ids, pred_bbox):
    target_class_ids = K.reshape(target_class_ids, (-1,))

```

```

target_bbox = K.reshape(target_bbox, (-1, 4))
pred_bbox = K.reshape(pred_bbox, (-1, K.int_shape(pred_bbox)[2], 4))
positive_roi_ix = tf.where(target_class_ids > 0)[: , 0]
positive_roi_class_ids = tf.cast(
    tf.gather(target_class_ids, positive_roi_ix), tf.int64)
indices = tf.stack([positive_roi_ix, positive_roi_class_ids], axis=1)
target_bbox = tf.gather(target_bbox, positive_roi_ix)
pred_bbox = tf.gather_nd(pred_bbox, indices)
loss = K.switch(tf.size(target_bbox) > 0,
    smooth_l1_loss(y_true=target_bbox, y_pred=pred_bbox),
    tf.constant(0.0))
loss = K.mean(loss)
return loss

def mrcnn_mask_loss_graph(target_masks, target_class_ids, pred_masks):
    target_class_ids = K.reshape(target_class_ids, (-1,))
    mask_shape = tf.shape(target_masks)
    target_masks = K.reshape(target_masks, (-1, mask_shape[2], mask_shape[3]))
    pred_shape = tf.shape(pred_masks)
    pred_masks = K.reshape(pred_masks,
        (-1, pred_shape[2], pred_shape[3], pred_shape[4]))
    pred_masks = tf.transpose(pred_masks, [0, 3, 1, 2])
    positive_ix = tf.where(target_class_ids > 0)[: , 0]
    positive_class_ids = tf.cast(
        tf.gather(target_class_ids, positive_ix), tf.int64)
    indices = tf.stack([positive_ix, positive_class_ids], axis=1)
    y_true = tf.gather(target_masks, positive_ix)
    y_pred = tf.gather_nd(pred_masks, indices)
    loss = K.switch(tf.size(y_true) > 0,
        K.binary_crossentropy(target=y_true, output=y_pred),
        tf.constant(0.0))
    loss = K.mean(loss)
    return loss

def load_image_gt(dataset, config, image_id, augment=False, augmentation=None,
    use_mini_mask=False):
    image = dataset.load_image(image_id)
    mask, class_ids = dataset.load_mask(image_id)
    original_shape = image.shape
    image, window, scale, padding, crop = utils.resize_image(
        image,
        min_dim=config.IMAGE_MIN_DIM,
        min_scale=config.IMAGE_MIN_SCALE,
        max_dim=config.IMAGE_MAX_DIM,
        mode=config.IMAGE_RESIZE_MODE)
    mask = utils.resize_mask(mask, scale, padding, crop)
    if augment:
        logging.warning("'augment' is deprecated. Use 'augmentation' instead.")
        if random.randint(0, 1):
            image = np.fliplr(image)
            mask = np.fliplr(mask)
    if augmentation:
        import imgaug
        MASK_AUGMENTERS = ["Sequential", "SomeOf", "OneOf", "Sometimes",
            "Fliplr", "Flipud", "CropAndPad",
            "Affine", "PiecewiseAffine"]

        def hook(images, augmenter, parents, default):
            return augmenter.__class__.__name__ in MASK_AUGMENTERS
        image_shape = image.shape
        mask_shape = mask.shape
        det = augmentation.to_deterministic()
        image = det.augment_image(image)
        mask = det.augment_image(mask.astype(np.uint8),
            hooks=imgaug.HooksImages(activator=hook))
        assert image.shape == image_shape, "Augmentation shouldn't change image size"
        assert mask.shape == mask_shape, "Augmentation shouldn't change mask size"
        mask = mask.astype(np.bool)
        _idx = np.sum(mask, axis=(0, 1)) > 0
        mask = mask[:, :, _idx]
        class_ids = class_ids[_idx]
        bbox = utils.extract_bboxes(mask)
        active_class_ids = np.zeros([dataset.num_classes], dtype=np.int32)

```

```

source_class_ids = dataset.source_class_ids[dataset.image_info[image_id]["source"]]
active_class_ids[source_class_ids] = 1
if use_mini_mask:
    mask = utils.minimize_mask(bbox, mask, config.MINI_MASK_SHAPE)
image_meta = compose_image_meta(image_id, original_shape, image.shape,
                                window, scale, active_class_ids)

return image, image_meta, class_ids, bbox, mask

def build_detection_targets(rpn_rois, gt_class_ids, gt_boxes, gt_masks, config):
    assert rpn_rois.shape[0] > 0
    assert gt_class_ids.dtype == np.int32, "Expected int but got {}".format(
        gt_class_ids.dtype)
    assert gt_boxes.dtype == np.int32, "Expected int but got {}".format(
        gt_boxes.dtype)
    assert gt_masks.dtype == np.bool_, "Expected bool but got {}".format(
        gt_masks.dtype)
    instance_ids = np.where(gt_class_ids > 0)[0]
    assert instance_ids.shape[0] > 0, "Image must contain instances."
    gt_class_ids = gt_class_ids[instance_ids]
    gt_boxes = gt_boxes[instance_ids]
    gt_masks = gt_masks[:, :, instance_ids]
    rpn_roi_area = (rpn_rois[:, 2] - rpn_rois[:, 0]) * \
        (rpn_rois[:, 3] - rpn_rois[:, 1])
    gt_box_area = (gt_boxes[:, 2] - gt_boxes[:, 0]) * \
        (gt_boxes[:, 3] - gt_boxes[:, 1])
    overlaps = np.zeros((rpn_rois.shape[0], gt_boxes.shape[0]))
    for i in range(overlaps.shape[1]):
        gt = gt_boxes[i]
        overlaps[:, i] = utils.compute_iou(
            gt, rpn_rois, gt_box_area[i], rpn_roi_area)
    rpn_roi_iou_argmax = np.argmax(overlaps, axis=1)
    rpn_roi_iou_max = overlaps[np.arange(
        overlaps.shape[0]), rpn_roi_iou_argmax]
    rpn_roi_gt_boxes = gt_boxes[rpn_roi_iou_argmax]
    rpn_roi_gt_class_ids = gt_class_ids[rpn_roi_iou_argmax]
    fg_ids = np.where(rpn_roi_iou_max > 0.5)[0]
    bg_ids = np.where(rpn_roi_iou_max < 0.5)[0]
    fg_roi_count = int(config.TRAIN_ROIS_PER_IMAGE * config.ROI_POSITIVE_RATIO)
    if fg_ids.shape[0] > fg_roi_count:
        keep_fg_ids = np.random.choice(fg_ids, fg_roi_count, replace=False)
    else:
        keep_fg_ids = fg_ids
    remaining = config.TRAIN_ROIS_PER_IMAGE - keep_fg_ids.shape[0]
    if bg_ids.shape[0] > remaining:
        keep_bg_ids = np.random.choice(bg_ids, remaining, replace=False)
    else:
        keep_bg_ids = bg_ids
    keep = np.concatenate([keep_fg_ids, keep_bg_ids])
    remaining = config.TRAIN_ROIS_PER_IMAGE - keep.shape[0]
    if remaining > 0:
        if keep.shape[0] == 0:
            bg_ids = np.where(rpn_roi_iou_max < 0.5)[0]
            assert bg_ids.shape[0] >= remaining
            keep_bg_ids = np.random.choice(bg_ids, remaining, replace=False)
            assert keep_bg_ids.shape[0] == remaining
            keep = np.concatenate([keep, keep_bg_ids])
        else:
            keep_extra_ids = np.random.choice(
                keep_bg_ids, remaining, replace=True)
            keep = np.concatenate([keep, keep_extra_ids])
    assert keep.shape[0] == config.TRAIN_ROIS_PER_IMAGE, \
        "keep doesn't match ROI batch size {}, {}".format(
            keep.shape[0], config.TRAIN_ROIS_PER_IMAGE)
    rpn_roi_gt_boxes[keep_bg_ids, :] = 0
    rpn_roi_gt_class_ids[keep_bg_ids] = 0
    rois = rpn_rois[keep]
    roi_gt_boxes = rpn_roi_gt_boxes[keep]
    roi_gt_class_ids = rpn_roi_gt_class_ids[keep]
    roi_gt_assignment = rpn_roi_iou_argmax[keep]
    bboxes = np.zeros((config.TRAIN_ROIS_PER_IMAGE,
                        config.NUM_CLASSES, 4), dtype=np.float32)
    pos_ids = np.where(roi_gt_class_ids > 0)[0]
    bboxes[pos_ids, roi_gt_class_ids[pos_ids]] = utils.box_refinement(

```

```

    rois[pos_ids], roi_gt_boxes[pos_ids, :4])
    bboxes /= config.BBOX_STD_DEV
    masks = np.zeros((config.TRAIN_ROIS_PER_IMAGE, config.MASK_SHAPE[0], con-
fig.MASK_SHAPE[1], config.NUM_CLASSES),
                      dtype=np.float32)
    for i in pos_ids:
        class_id = roi_gt_class_ids[i]
        assert class_id > 0, "class id must be greater than 0"
        gt_id = roi_gt_assignment[i]
        class_mask = gt_masks[:, :, gt_id]

        if config.USE_MINI_MASK:
            placeholder = np.zeros(config.IMAGE_SHAPE[:2], dtype=bool)
            gt_y1, gt_x1, gt_y2, gt_x2 = gt_boxes[gt_id]
            gt_w = gt_x2 - gt_x1
            gt_h = gt_y2 - gt_y1
            placeholder[gt_y1:gt_y2, gt_x1:gt_x2] = \
                np.round(utils.resize(class_mask, (gt_h, gt_w))).astype(bool)
            class_mask = placeholder
        y1, x1, y2, x2 = rois[i].astype(np.int32)
        m = class_mask[y1:y2, x1:x2]
        mask = utils.resize(m, config.MASK_SHAPE)
        masks[i, :, :, class_id] = mask

    return rois, roi_gt_class_ids, bboxes, masks

```